

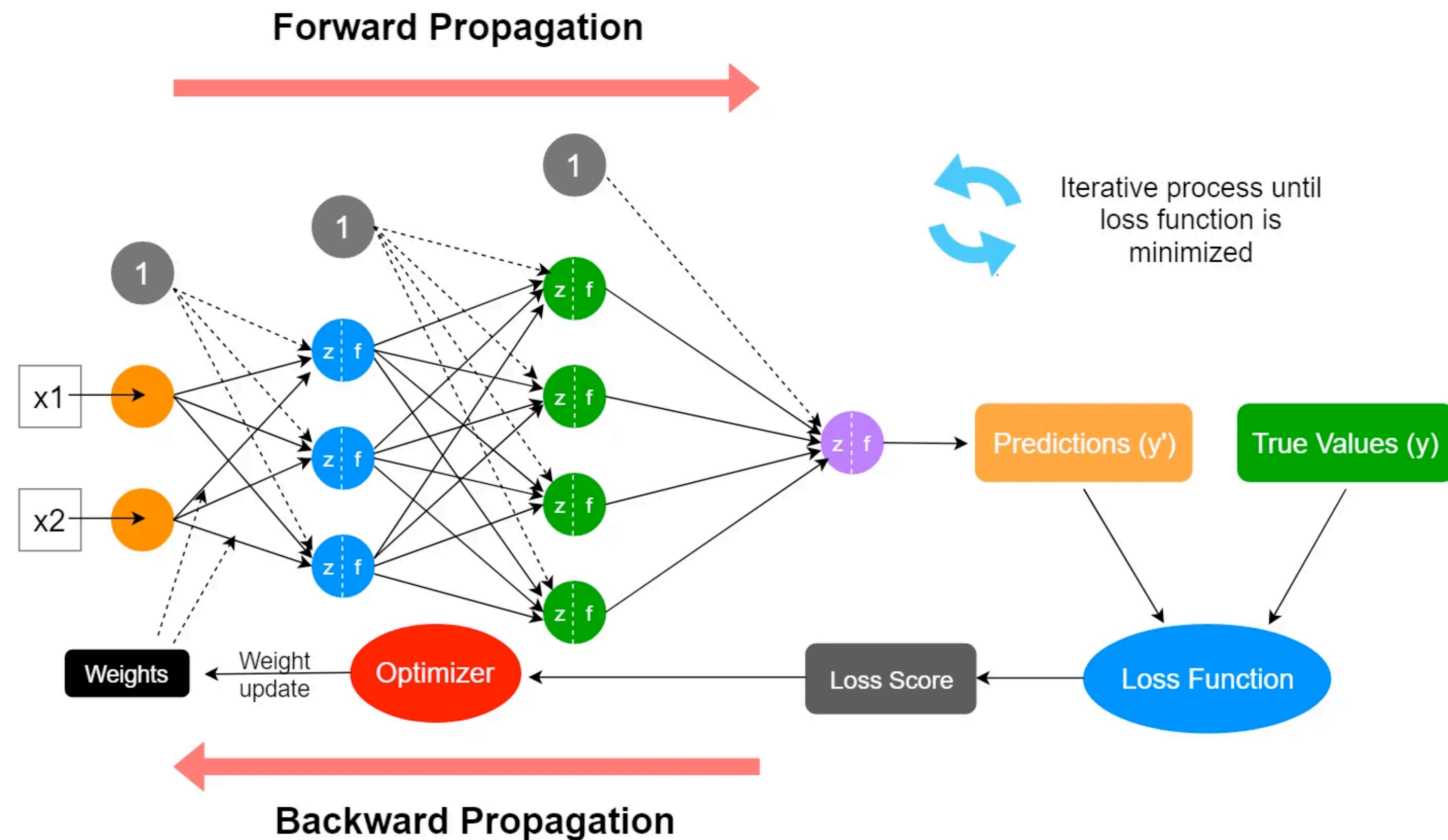
Deep Learning

Optimiser l'apprentissage

Théo Mariotte - 06/02/2026

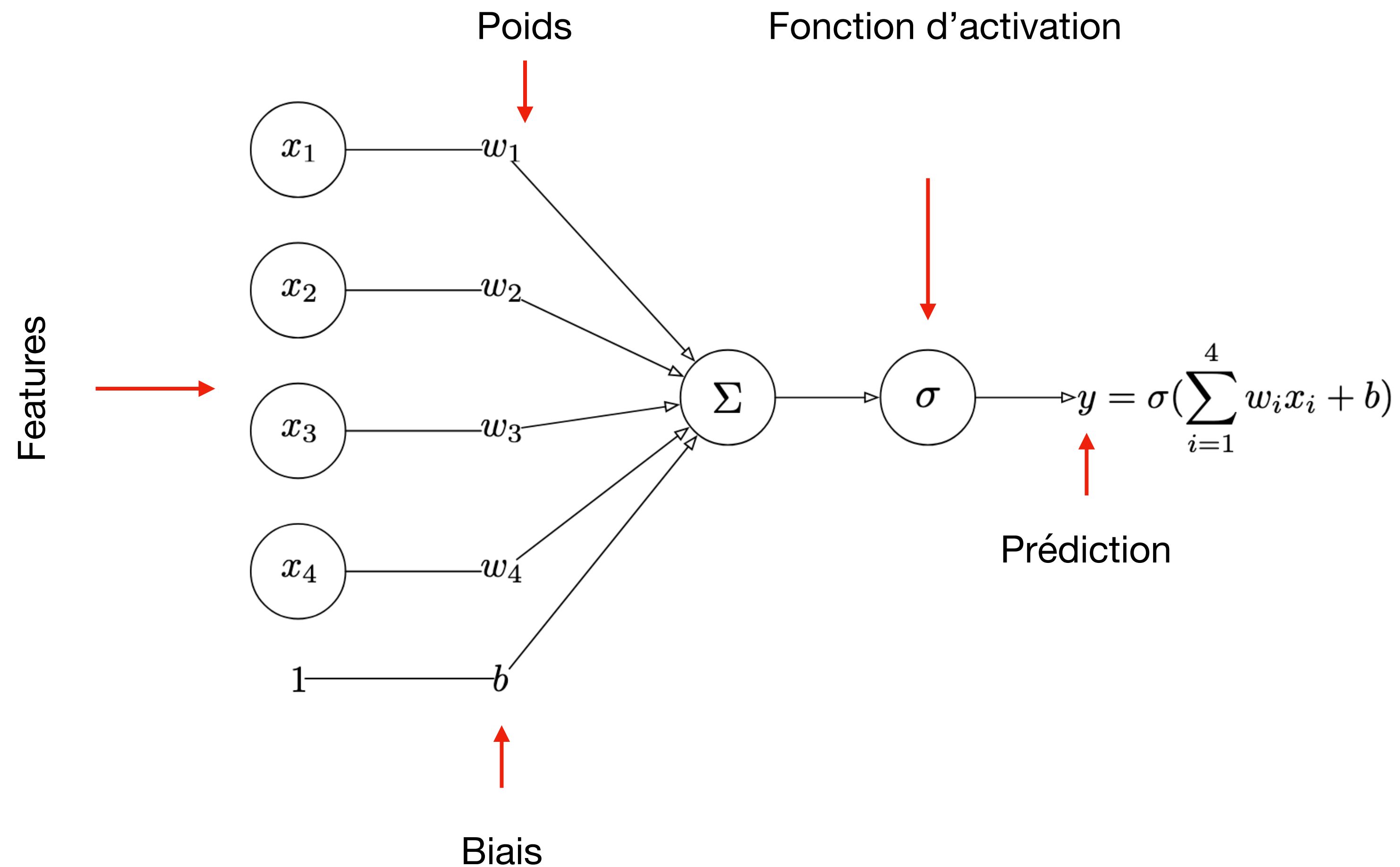
Quelques rappels

Apprentissage d'un réseau de neurones



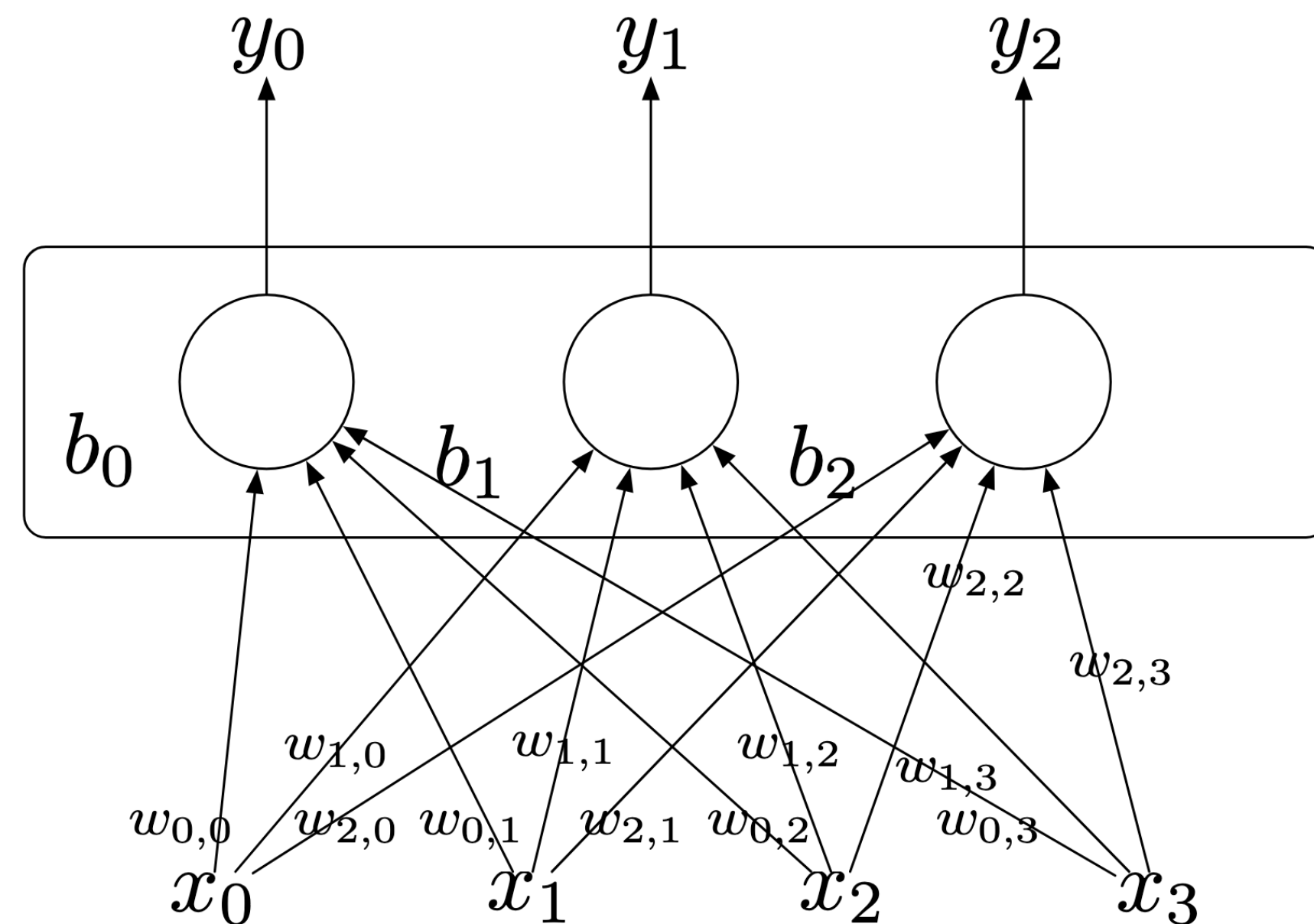
Ce que vous savez déjà

1 seul neurone



Ce que vous savez déjà

Réseau de neurones à 1 couche de 3 neurones



Fonction d'activation

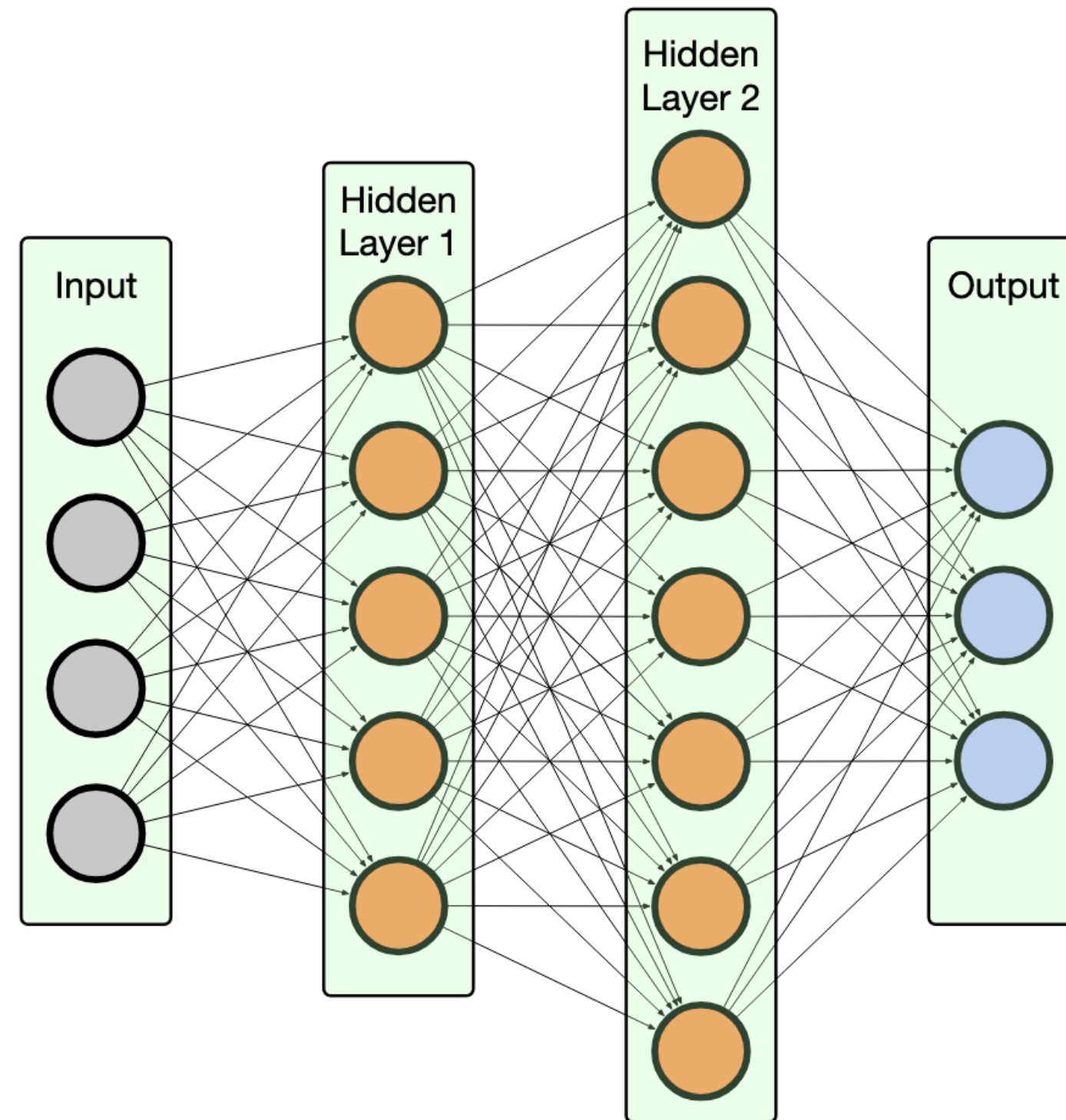
$$\begin{matrix} \uparrow \\ \text{Prédiction} \end{matrix} \begin{bmatrix} y_0 \\ y_1 \\ y_2 \end{bmatrix} = \sigma \begin{matrix} \downarrow \\ \text{Poids} \end{matrix} \begin{pmatrix} \begin{bmatrix} w_{0,0} & w_{0,1} & w_{0,2} & w_{0,3} \\ w_{1,0} & w_{1,1} & w_{1,2} & w_{1,3} \\ w_{2,0} & w_{2,1} & w_{2,2} & w_{2,3} \end{bmatrix} \times \begin{matrix} \uparrow \\ \text{Features} \end{matrix} \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix} + \begin{matrix} \uparrow \\ \text{Biais} \end{matrix} \begin{bmatrix} b_0 \\ b_1 \\ b_2 \end{bmatrix} \end{pmatrix}$$

Ce que vous savez déjà



Augmenter la profondeur du réseau avec des couches cachées

Depth of the Network →



$$\mathbf{Y} = \sigma \left(\begin{bmatrix} w_{11}^{(3)} & w_{12}^{(3)} & w_{13}^{(3)} \\ w_{21}^{(3)} & w_{22}^{(3)} & w_{23}^{(3)} \end{bmatrix} \sigma \left(\begin{bmatrix} w_{11}^{(2)} & w_{12}^{(2)} & w_{13}^{(2)} \\ w_{21}^{(2)} & w_{22}^{(2)} & w_{23}^{(2)} \\ w_{31}^{(2)} & w_{32}^{(2)} & w_{33}^{(2)} \end{bmatrix} \sigma \left(\begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \\ w_{31} & w_{32} & w_{33} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix} \right) + \begin{bmatrix} b_1^{(2)} \\ b_2^{(2)} \\ b_3^{(2)} \end{bmatrix} \right) + \begin{bmatrix} b_1^{(3)} \\ b_2^{(3)} \end{bmatrix} \right)$$

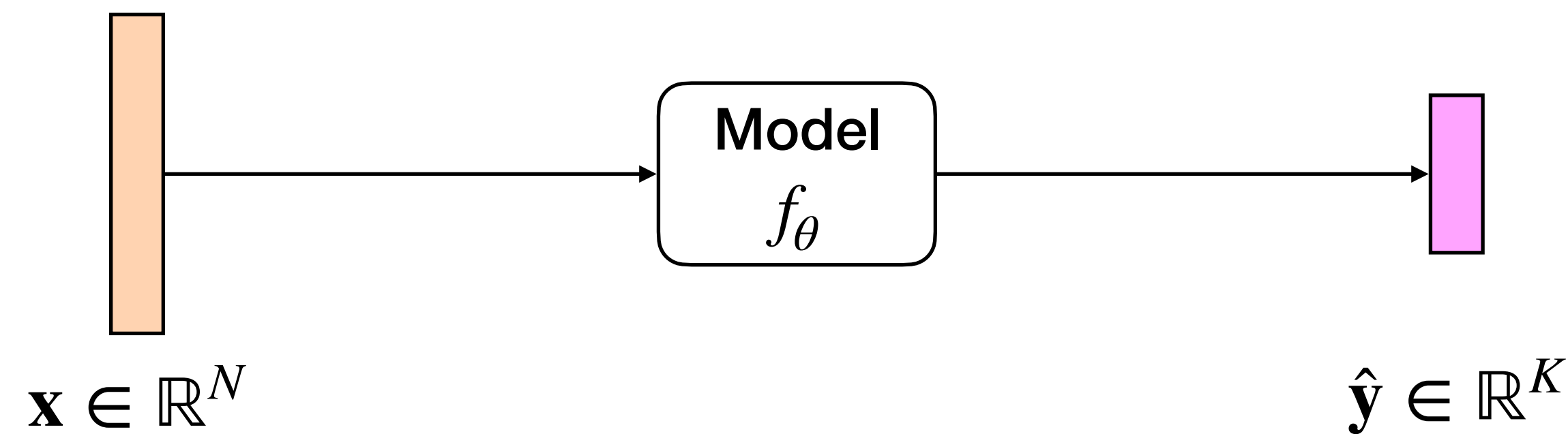
$$\mathbf{Y} = \sigma (\mathbf{W}_3 \sigma (\mathbf{W}_2 \sigma (\mathbf{W}_1 \mathbf{X} + \mathbf{b}_1) + \mathbf{b}_2) + \mathbf{b}_3) .$$

$$\mathbf{Y} = f_{\mathbf{w}_1} \circ f_{\mathbf{w}_2} \circ \dots \circ f_{\mathbf{w}_L}(\mathbf{X})$$

Ce que vous savez déjà

Passage des données

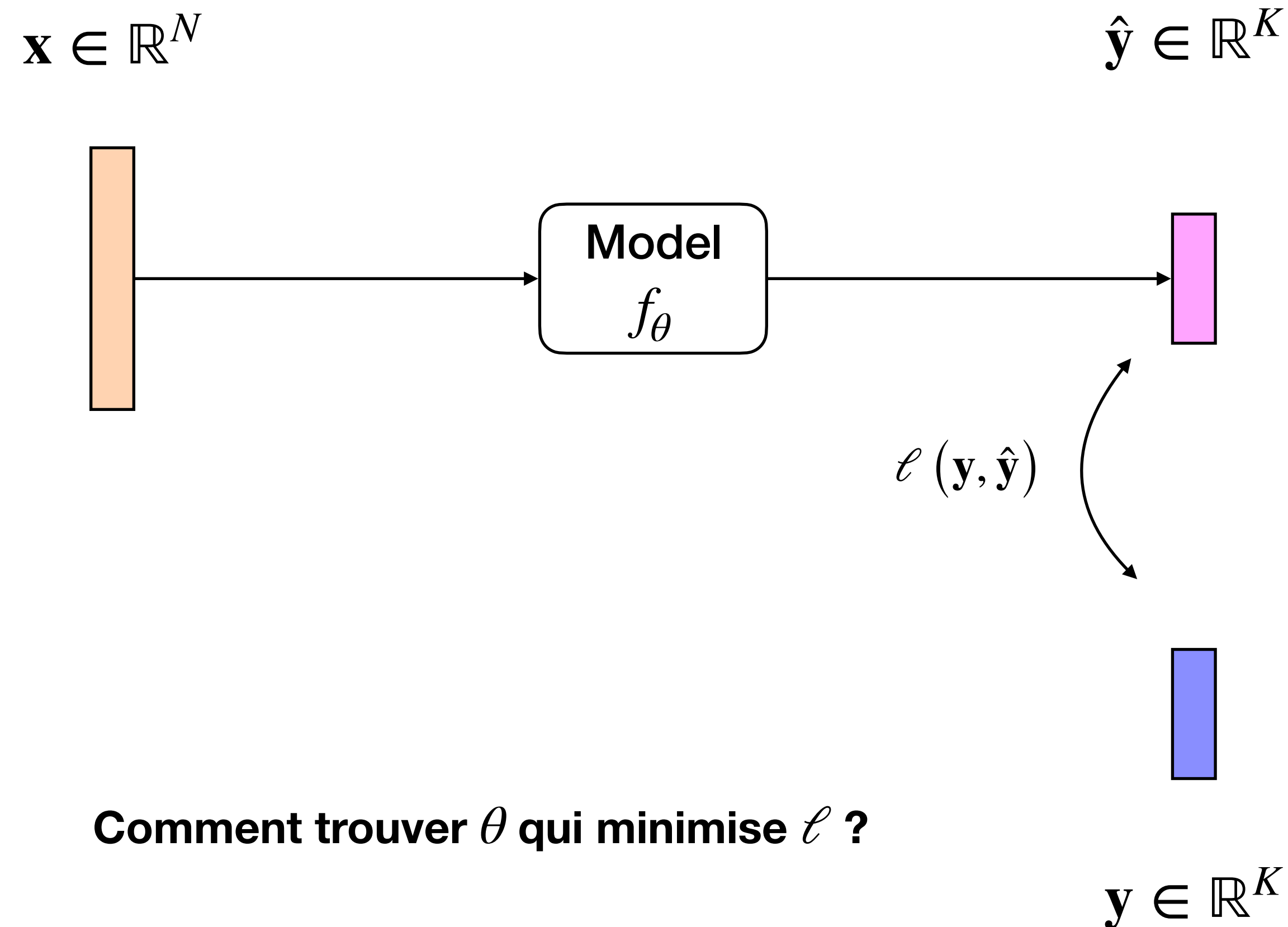
$$f_{\theta} = f_{\mathbf{w}_1} \circ f_{\mathbf{w}_2} \circ \dots \circ f_{\mathbf{w}_L} : \mathbb{R}^N \mapsto \mathbb{R}^K$$



- Prédiction en calculant $\hat{\mathbf{y}} = f_{\theta}(\mathbf{x})$
- θ représente l'ensemble des poids (paramètres) du réseau
- Avec une initialisation aléatoire, la prédiction est aléatoire
- Besoin d'optimiser les poids θ pour minimiser un objectif (*loss function*)

Ce que vous savez déjà

Mais comment ça s'utilise ?



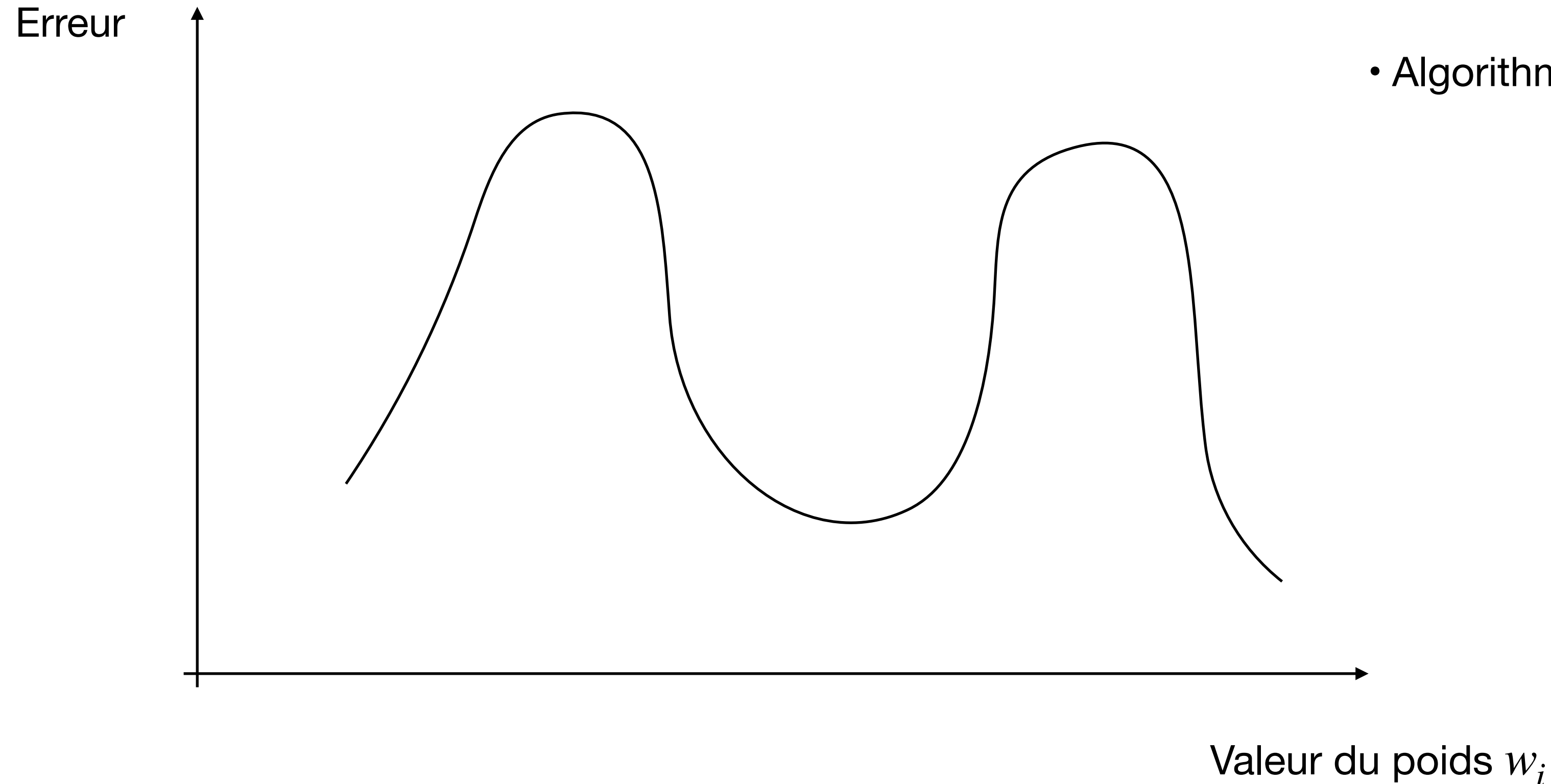
$$f_\theta = f_{\mathbf{w}_1} \circ f_{\mathbf{w}_2} \circ \dots \circ f_{\mathbf{w}_L} : \mathbb{R}^N \mapsto \mathbb{R}^K$$

- On définit une fonction de coût $\ell(\mathbf{y}, \hat{\mathbf{y}}) : \mathbb{R}^k \mapsto \mathbb{R}$
- ℓ mesure l'écart entre la prédiction et la référence (annotation)
- On cherche $\hat{\theta} = \arg \min_{\theta} \ell(\hat{\mathbf{y}}, \mathbf{y})$
- Il existe des fonctions de coût adaptées à différentes tâches :
 - *Cross Entropy* : classification multi-classe
 - *Binary Cross Entropy* : classification multi-label
 - *Mean Squared Error (MSE)* : régression

Ce que vous savez déjà

Optimisation des paramètres

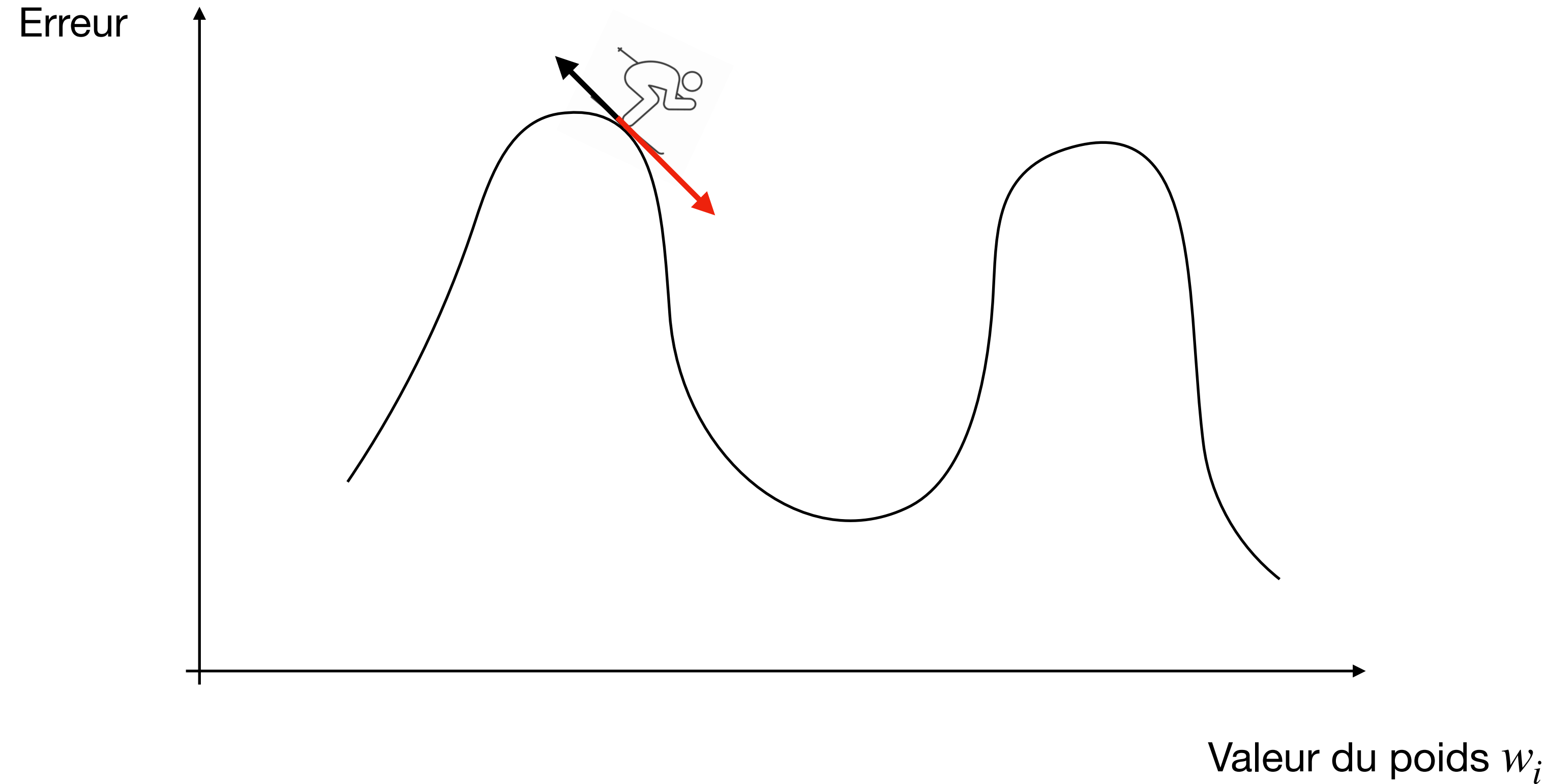
- L'erreur varie en fonction de la valeur des poids
- Le coût en fonction des paramètres s'appelle **l'espace de recherche**
- On cherche un jeu de paramètres qui minimise la fonction ci-contre
- Algorithme de **descente de gradient**



Ce que vous savez déjà

Optimisation des paramètres

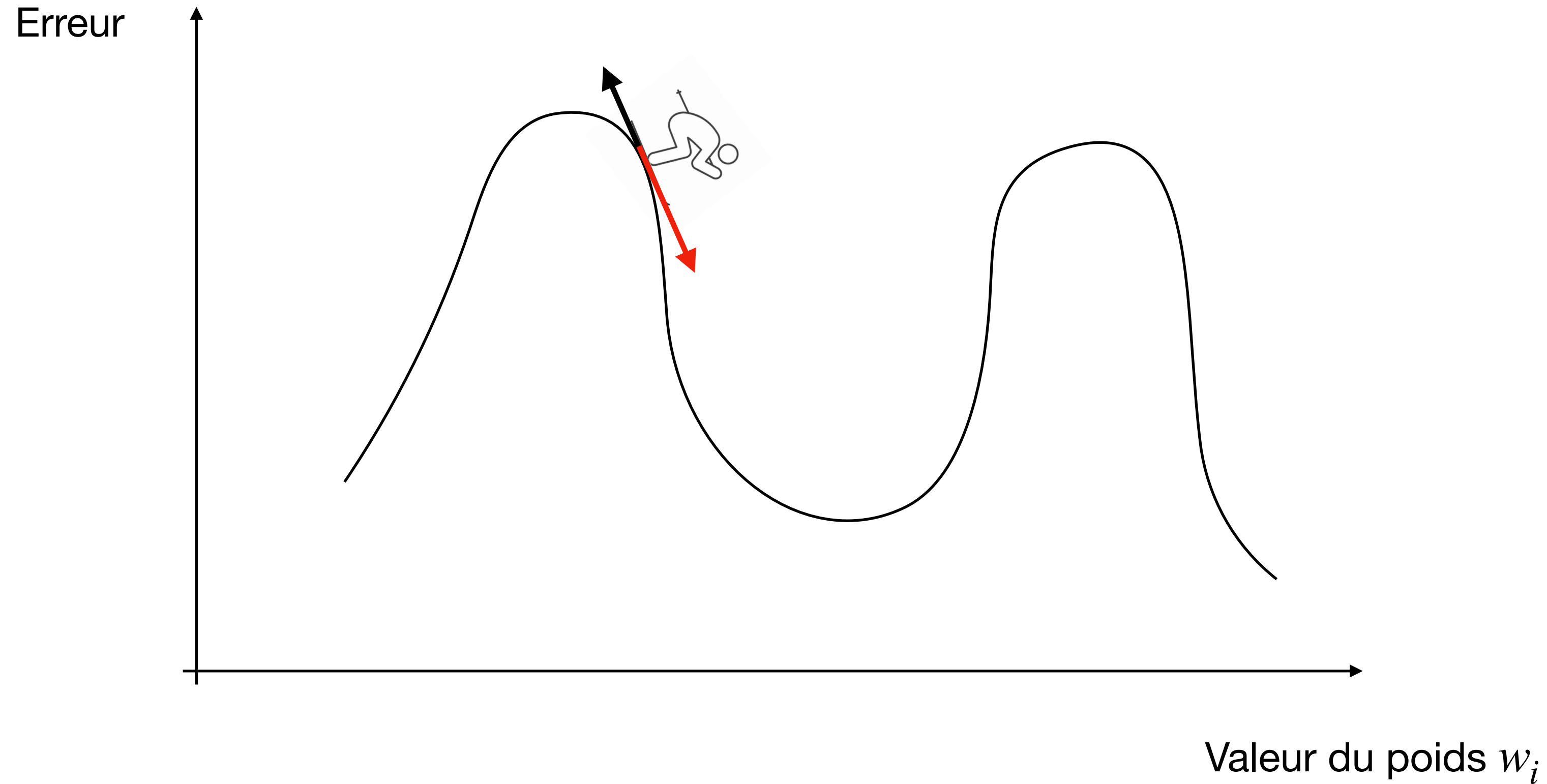
- Point d'initialisation aléatoire
- On suit la direction inverse du gradient



Ce que vous savez déjà

Optimisation des paramètres

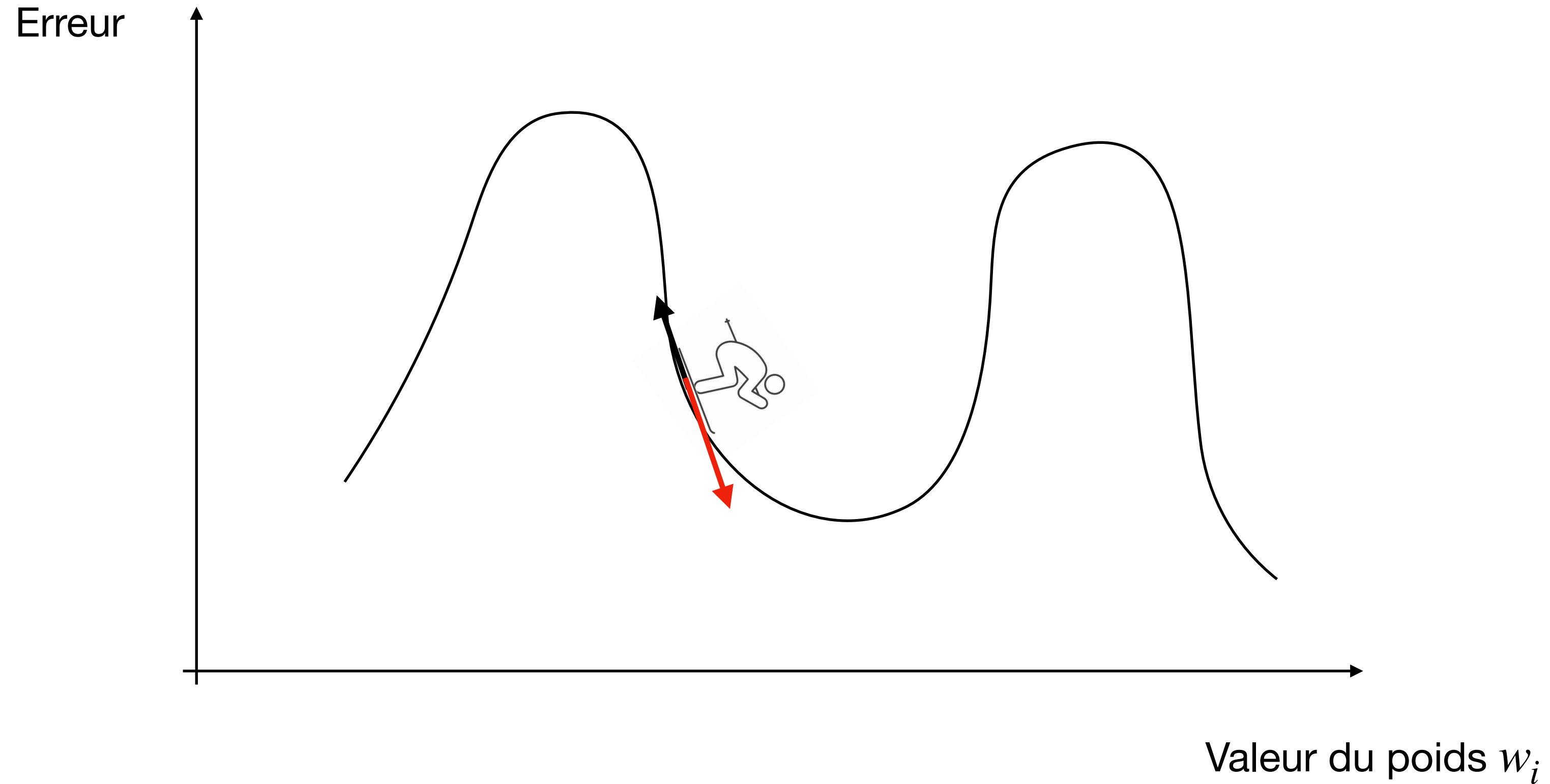
- A chaque étape on calcule le gradient et on suit sa direction opposée



Ce que vous savez déjà

Optimisation des paramètres

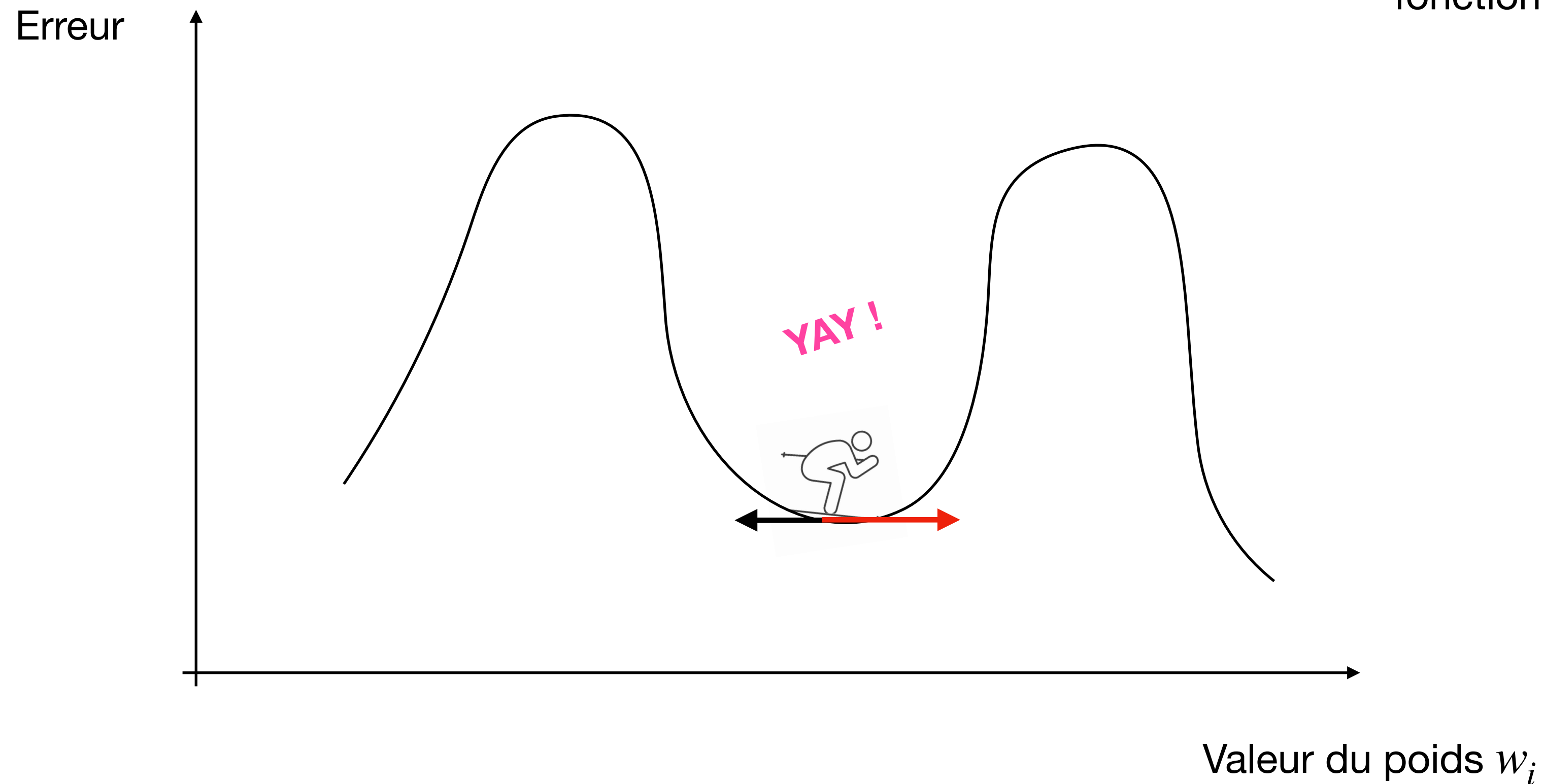
- Zwiip on se rapproche du fond de la vallée



Ce que vous savez déjà

Optimisation des paramètres

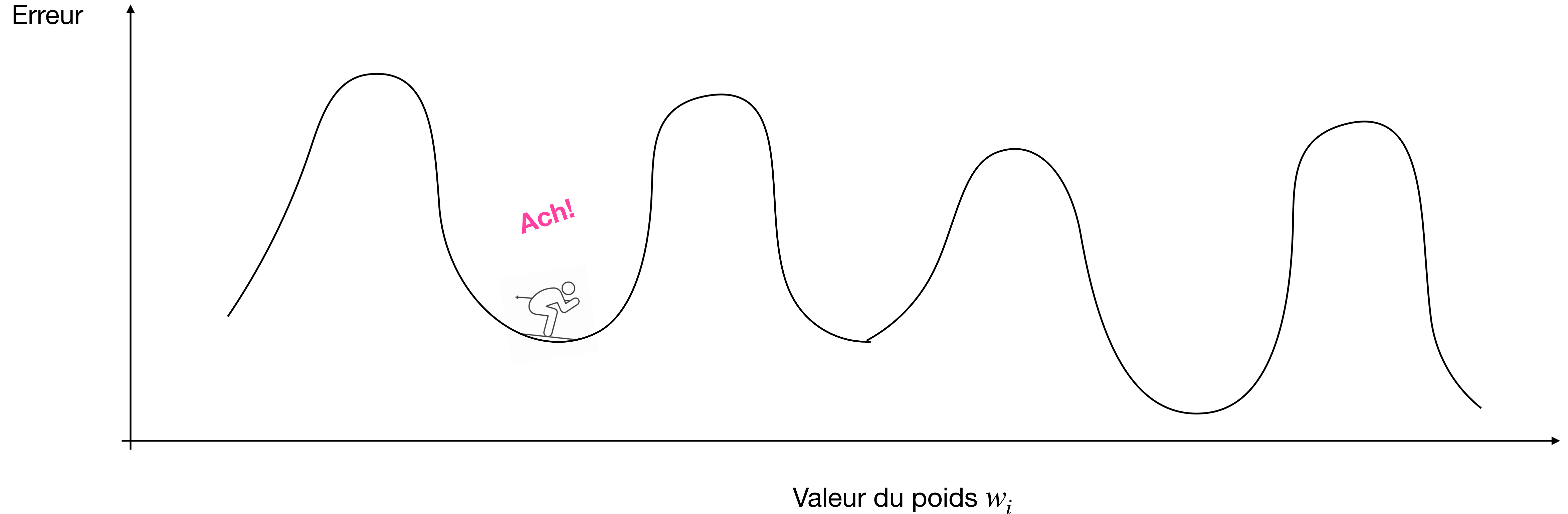
- Quand le gradient devient tout petit, on a convergé !
- Le point d'arrêt définit les **paramètres optimaux** de notre modèle pour la fonction de coût considérée



Ce que vous savez déjà

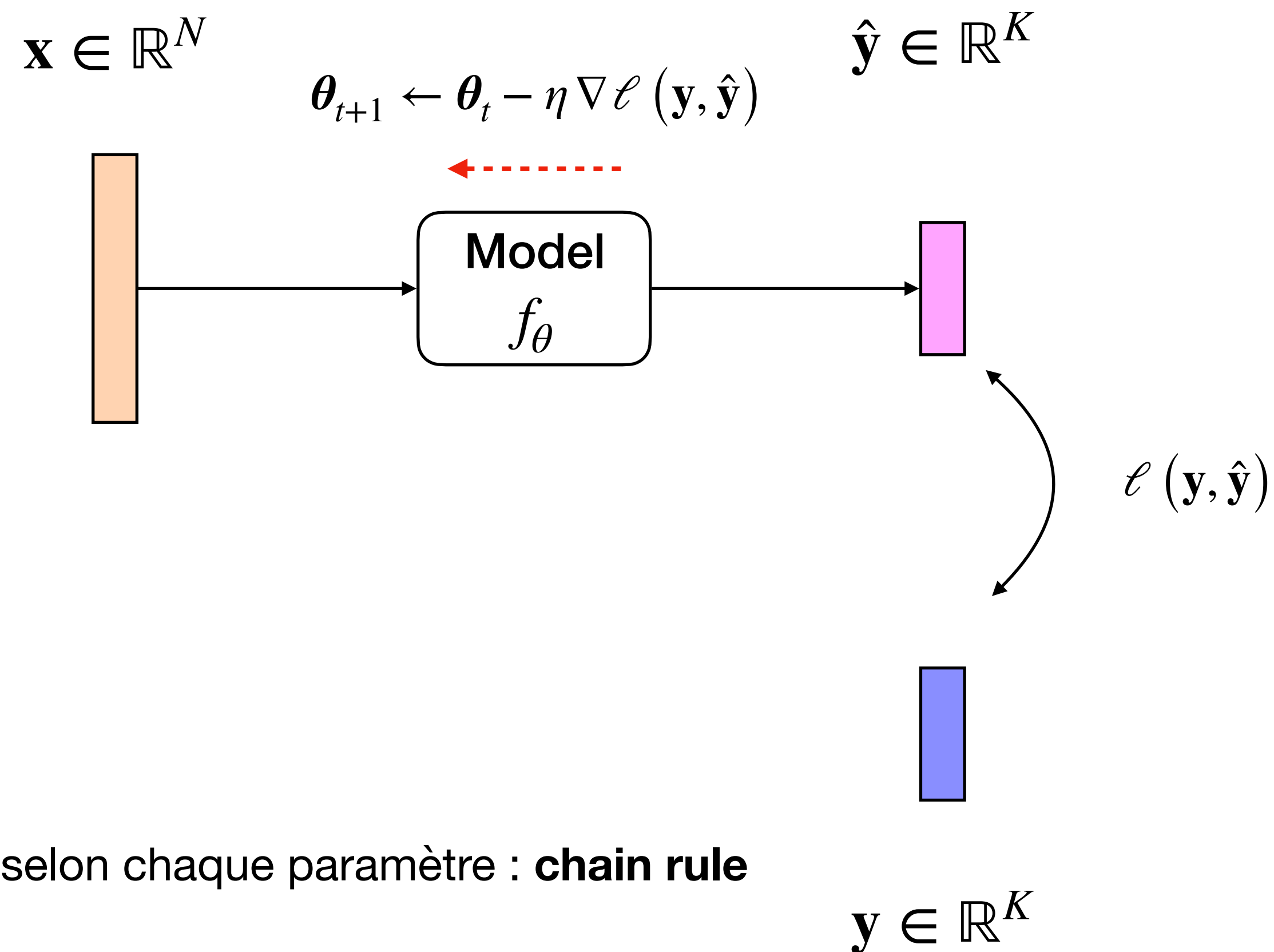
Optimisation des paramètres

Attention : vous trouvez un minimum **LOCAL** qui dépend de votre point d'initialisation !



Ce que vous savez déjà

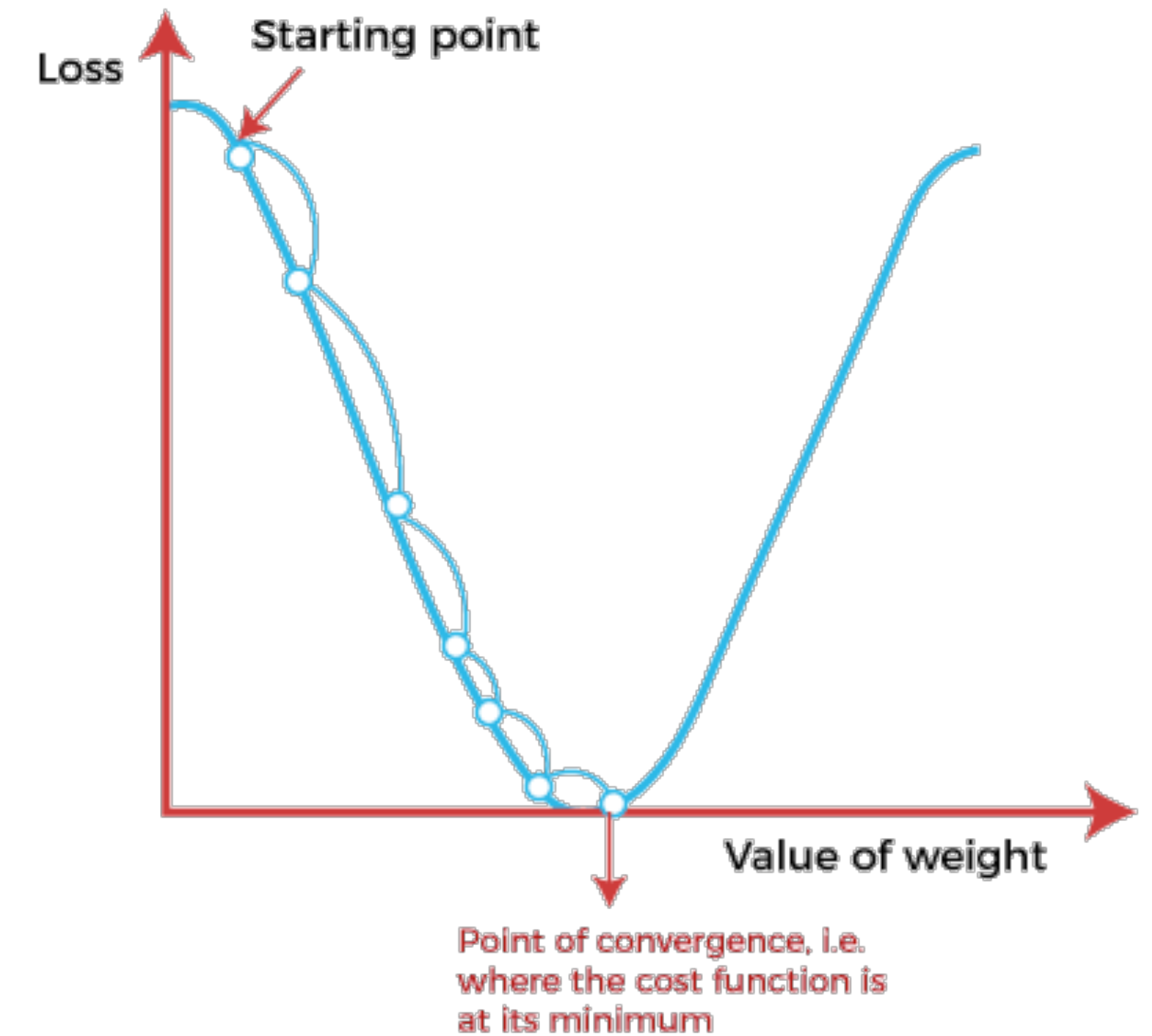
Mais comment ça s'utilise ?



Gradient selon chaque paramètre : **chain rule**

Implémentation efficace : **backpropagation**

$$f_\theta = f_{\mathbf{w}_1} \circ f_{\mathbf{w}_2} \circ \dots \circ f_{\mathbf{w}_L} : \mathbb{R}^N \mapsto \mathbb{R}^K$$



Descente de gradient pour minimiser le coût (loss)

Ce que vous savez déjà

Les limites

- Objectif du *machine learning* : trouver un modèle qui représente les données
- Il faut minimiser le risque :

$$J(\theta) := \mathbb{E}_{\mathbf{x}, \mathbf{y} \sim \mathbb{P}(\mathbf{x}, \mathbf{y})} [\ell(f_{\theta}(\mathbf{x}), \mathbf{y})]$$

Ce que vous savez déjà

Les limites

- Objectif du *machine learning* : trouver un modèle qui représente les données

- Il faut minimiser le risque :

$$J(\theta) := \mathbb{E}_{\mathbf{x}, \mathbf{y} \sim \mathbb{P}(\mathbf{x}, \mathbf{y})} [\ell(f_{\theta}(\mathbf{x}), \mathbf{y})]$$

- En pratique, on a pas accès à $\mathbb{P}$, mais seulement un **échantillonnage** : $\mathcal{D} = \{\mathbf{x}_i, \mathbf{y}_i\}_{i=1}^n$ n : taille du dataset

- On peut minimiser un estimateur, appelé le **risque empirique**

$$\mathbf{f} = J_{\text{emp}}(\theta) := \frac{1}{n} \sum_{i=1}^n \ell(f_{\theta}(\mathbf{x}_i), \mathbf{y}_i)$$

Ce que vous savez déjà

Les limites

- Objectif du *machine learning* : trouver un modèle qui représente les données
- Il faut minimiser le risque :

$$J(\theta) := \mathbb{E}_{\mathbf{x}, \mathbf{y} \sim \mathbb{P}(\mathbf{x}, \mathbf{y})} [\ell(f_{\theta}(\mathbf{x}), \mathbf{y})]$$

- En pratique, on a pas accès à $\mathbb{P}$, mais seulement un échantillonnage : $\mathcal{D} = \{\mathbf{x}_i, \mathbf{y}_i\}_{i=1}^n$

n : taille du dataset

- On peut minimiser un estimateur, appelé le **risque empirique**

$$\mathbf{f} = J_{\text{emp}}(\theta) := \frac{1}{n} \sum_{i=1}^n \ell(f_{\theta}(\mathbf{x}_i), \mathbf{y}_i)$$

- En pratique, **on doit optimiser le modèle avec le gradient** $\nabla \mathbf{f}$ (sur tout le jeu de données)
 - Avec des grand jeux de données, ça devient très coûteux
 - Le calcul du gradient exact devient donc un problème !

If exact computation is expensive, replace it with a cheaper estimate

Ce que vous savez déjà

Par exemple



- ImageNet est un jeu de données très célèbre pour la reconnaissance d'images
- Il contient **14 Millions d'images** pour l'apprentissage
- Pour faire une itération de la descente de gradient, il faudrait donc faire **14M de *forward* dans le modèle**, et autant de calcul de *loss*
- Quel autre problème peut apparaître selon vous?

Ce que vous savez déjà

Par exemple



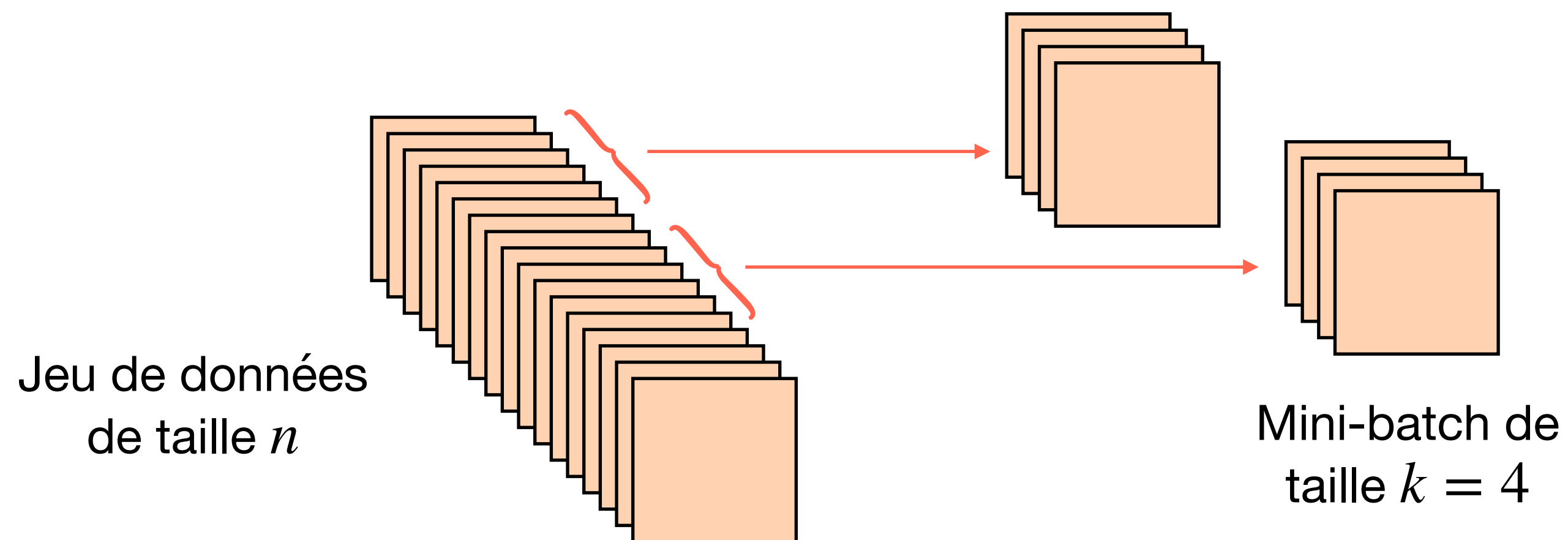
- ImageNet est un jeu de données très célèbre pour la reconnaissance d'images
- Il contient **14 Millions d'images** pour l'apprentissage
- Pour faire une itération de la descente de gradient, il faudrait donc faire **14M de *forward* dans le modèle**, et autant de calcul de *loss*
- **Quel autre problème peut apparaître selon vous?**

Moyenne de la loss sur beaucoup de données = perte d'information utile à l'apprentissage !

Apprentissage profond

Stochastic Gradient Descent

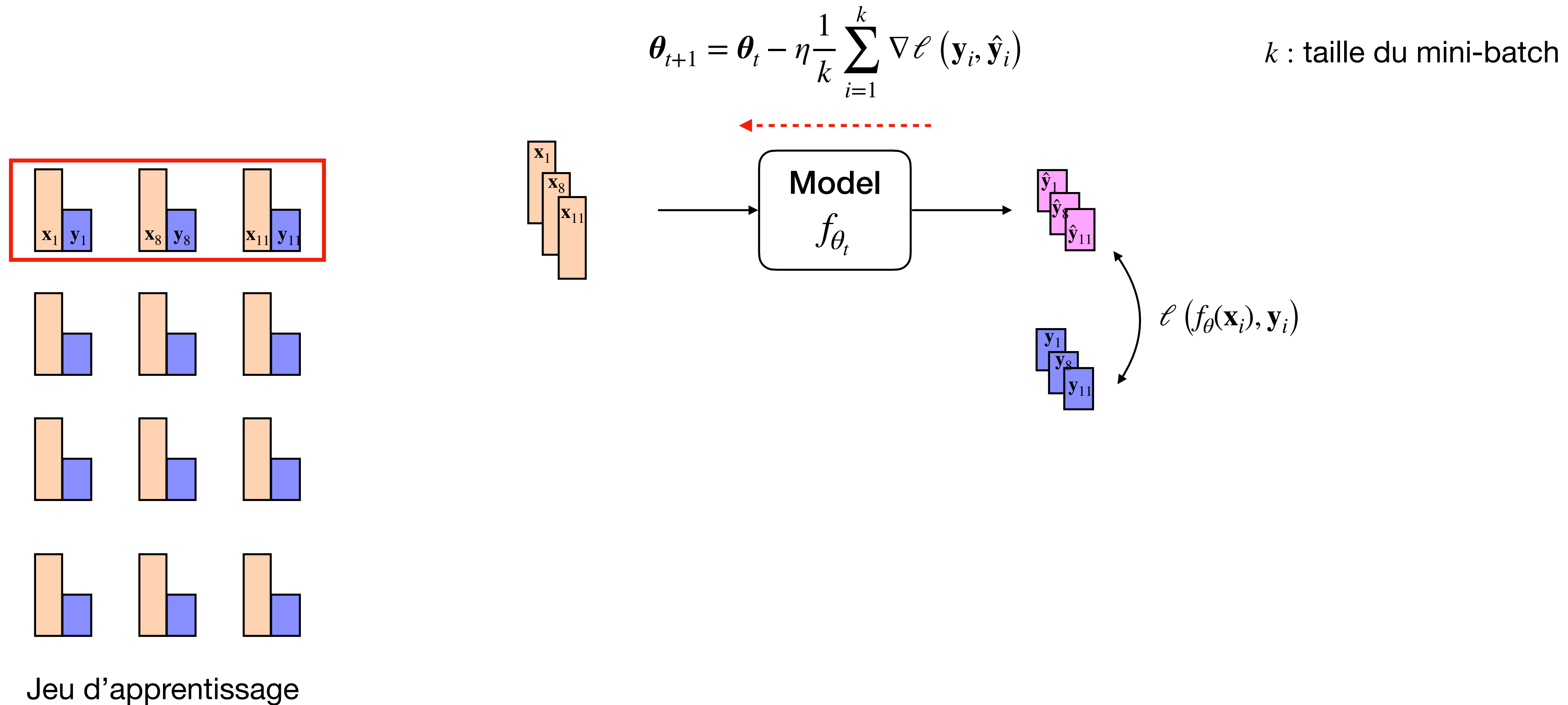
- **Principe** : plutôt que de calculer le gradient sur toutes les données avant d'optimiser les poids, on va le calculer sur des sous-ensembles du jeu de données
 - On remplace $\nabla \mathbf{f}$ par une estimation $\tilde{\nabla} \mathbf{f}$
 - $\tilde{\nabla} \mathbf{f}$ est calculé sur un sous-ensemble de données appelé **mini-batch**
 - Remarque : $\nabla \mathbf{f} = \mathbb{E} [\tilde{\nabla} \mathbf{f}]$



$$\tilde{\nabla} \mathbf{f} = \frac{1}{4} \sum_{i=1}^4 \nabla \ell(\mathbf{y}_i, \hat{\mathbf{y}}_i)$$

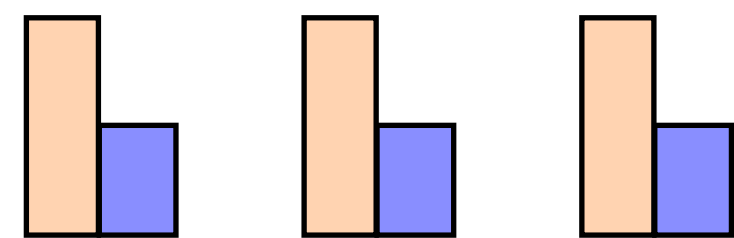
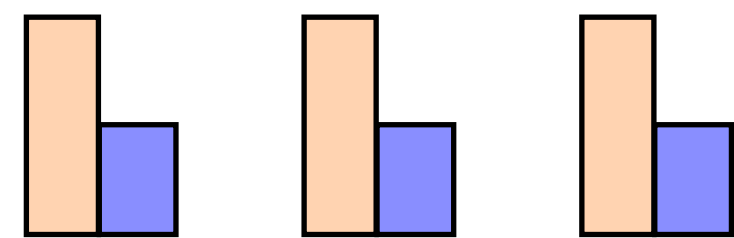
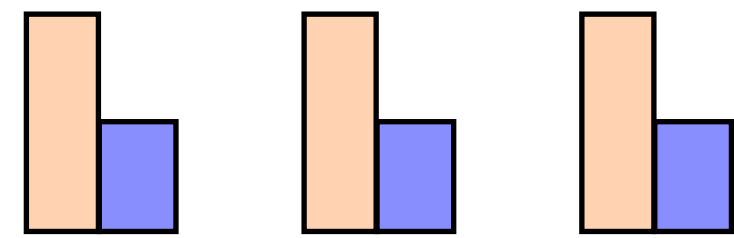
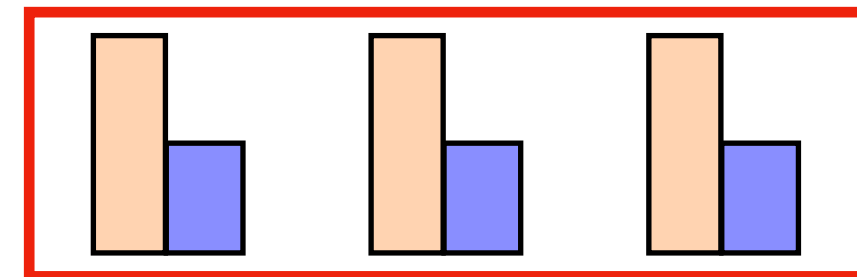
Apprentissage profond

SGD: illustration

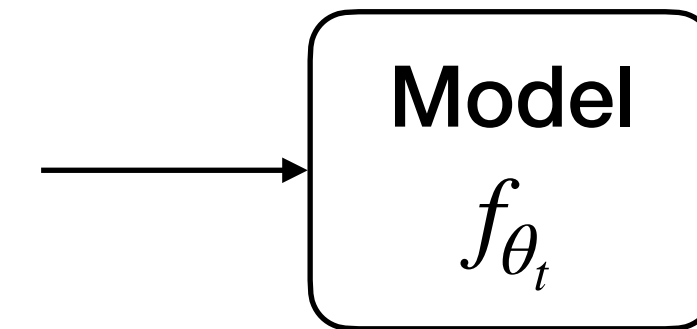
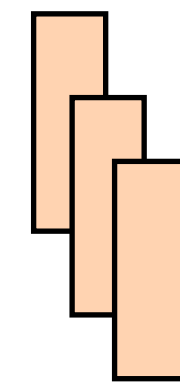


Apprentissage profond

SGD: illustration



Jeu d'apprentissage



$\ell(f_{\theta}(\mathbf{x}_i), \mathbf{y}_i)$

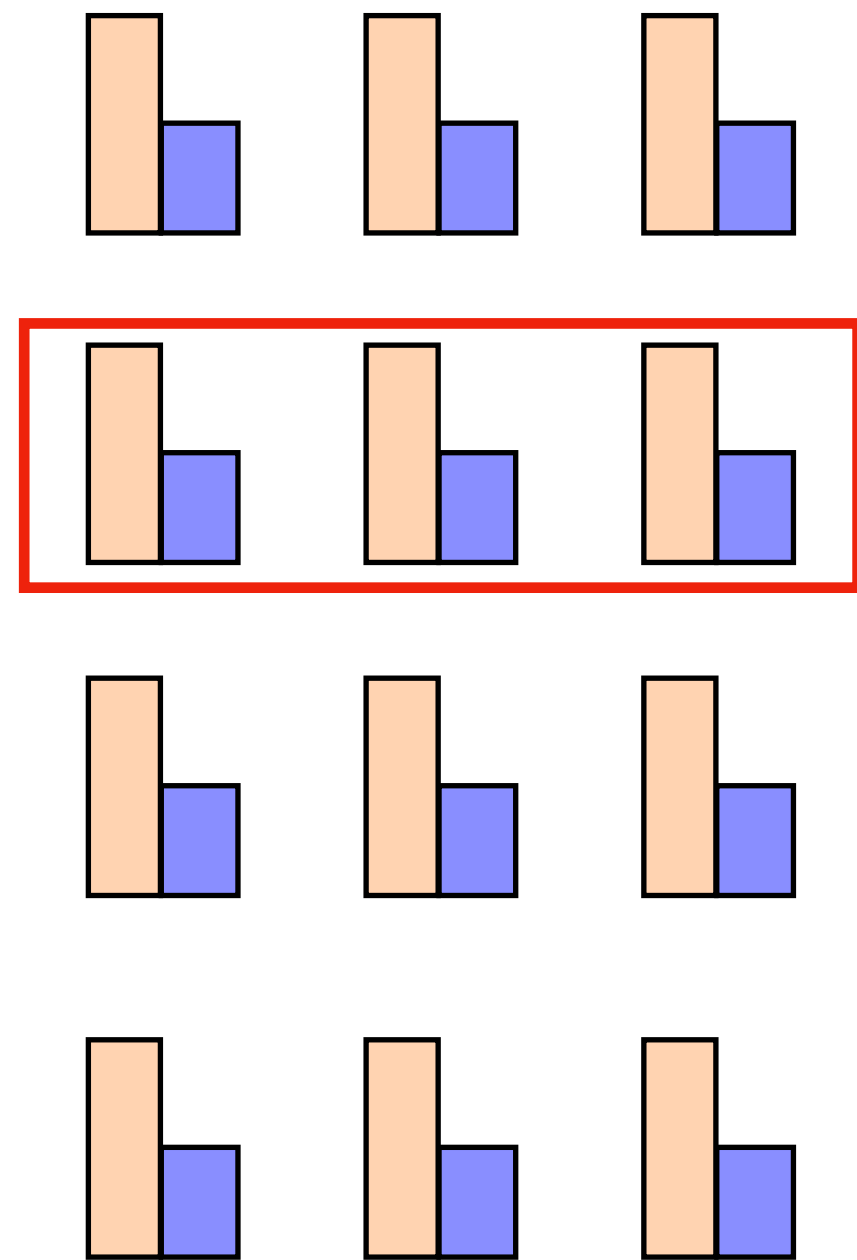
$$\theta_{t+1} = \theta_t - \eta \frac{1}{k} \sum_{i=1}^k \nabla \ell(\mathbf{y}_i, \hat{\mathbf{y}}_i)$$

Moyenne des gradients
sur les éléments du
mini-batch

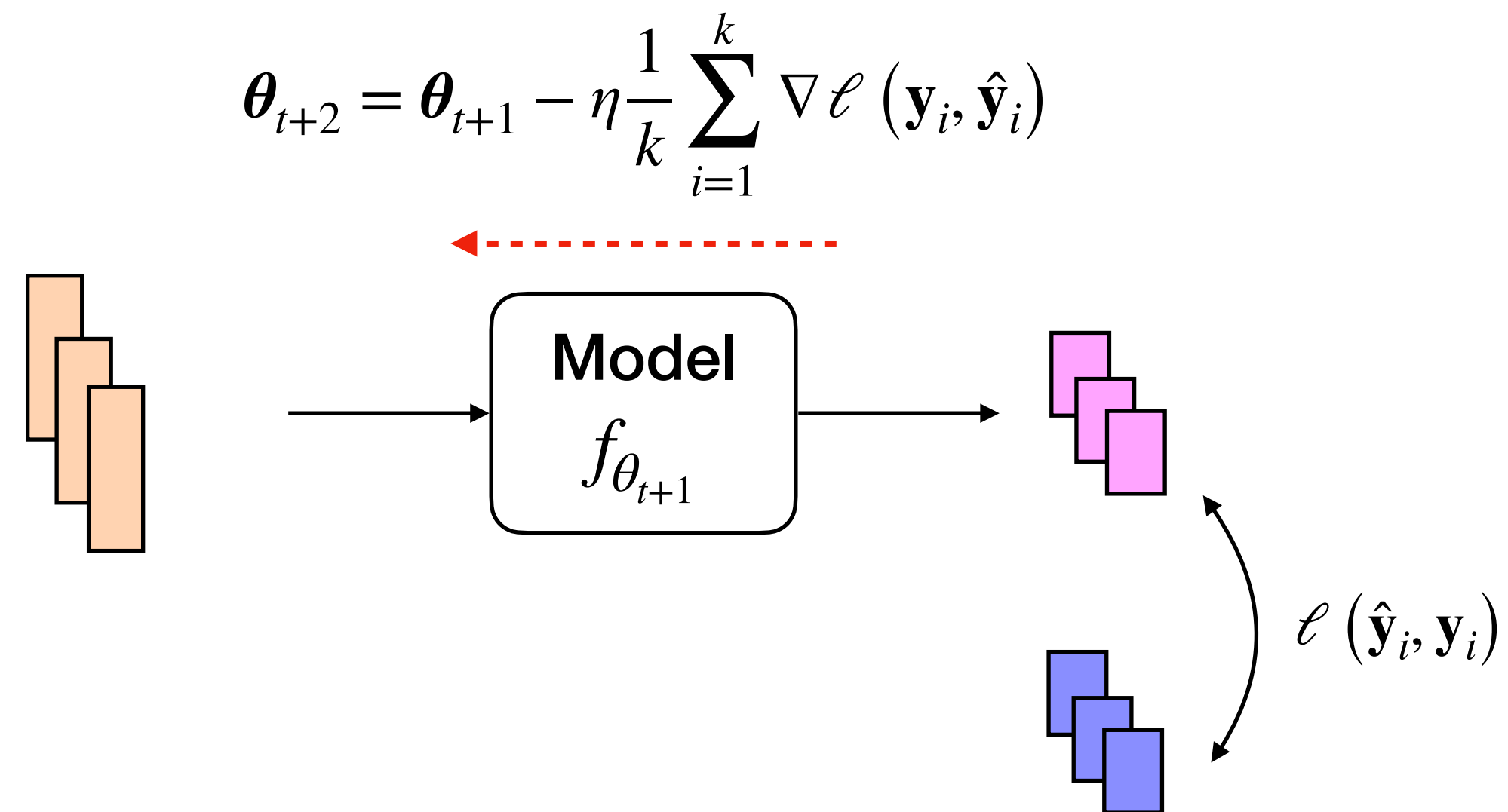
k : taille du mini-batch

Apprentissage profond

SGD: illustration

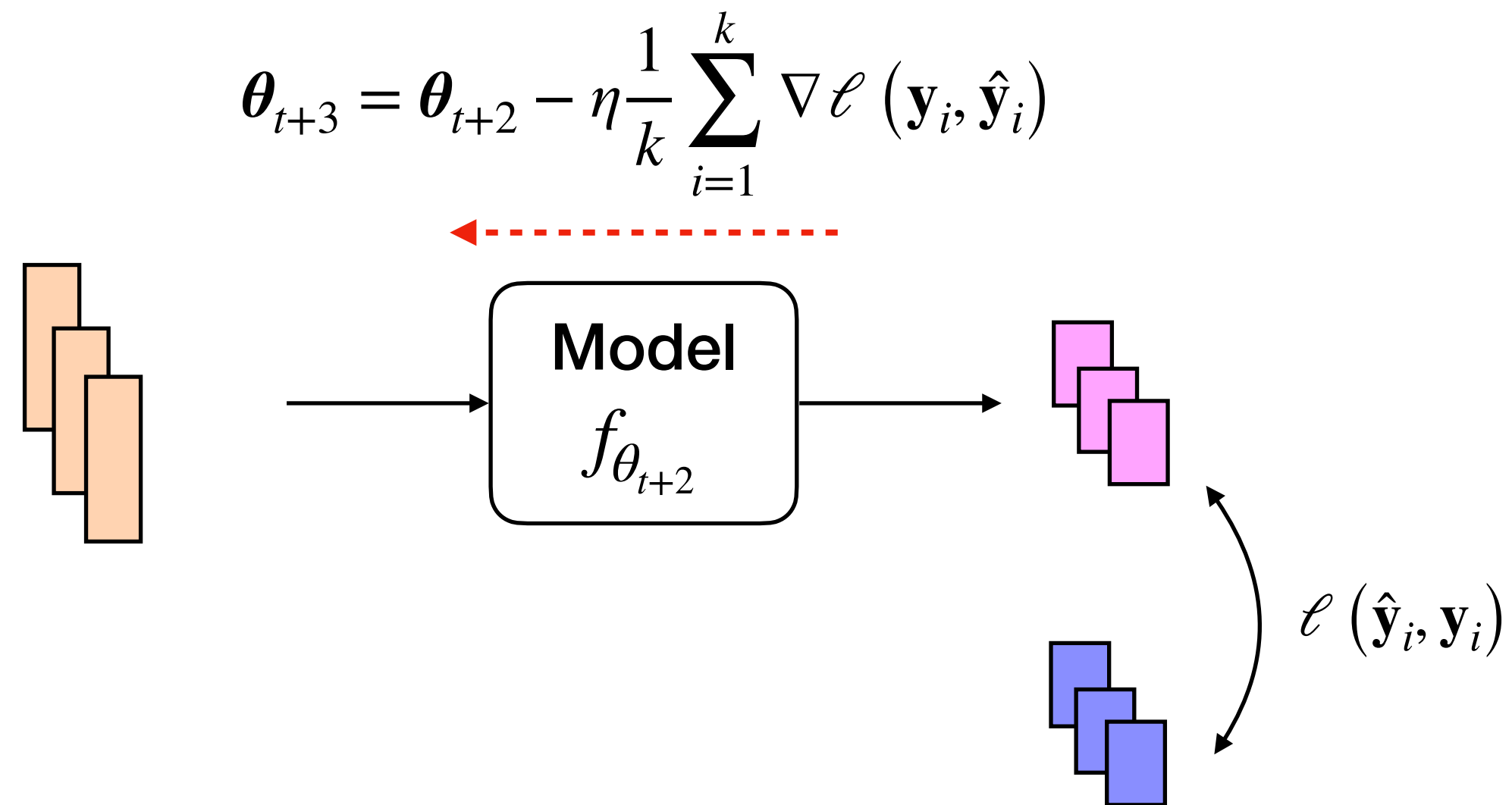
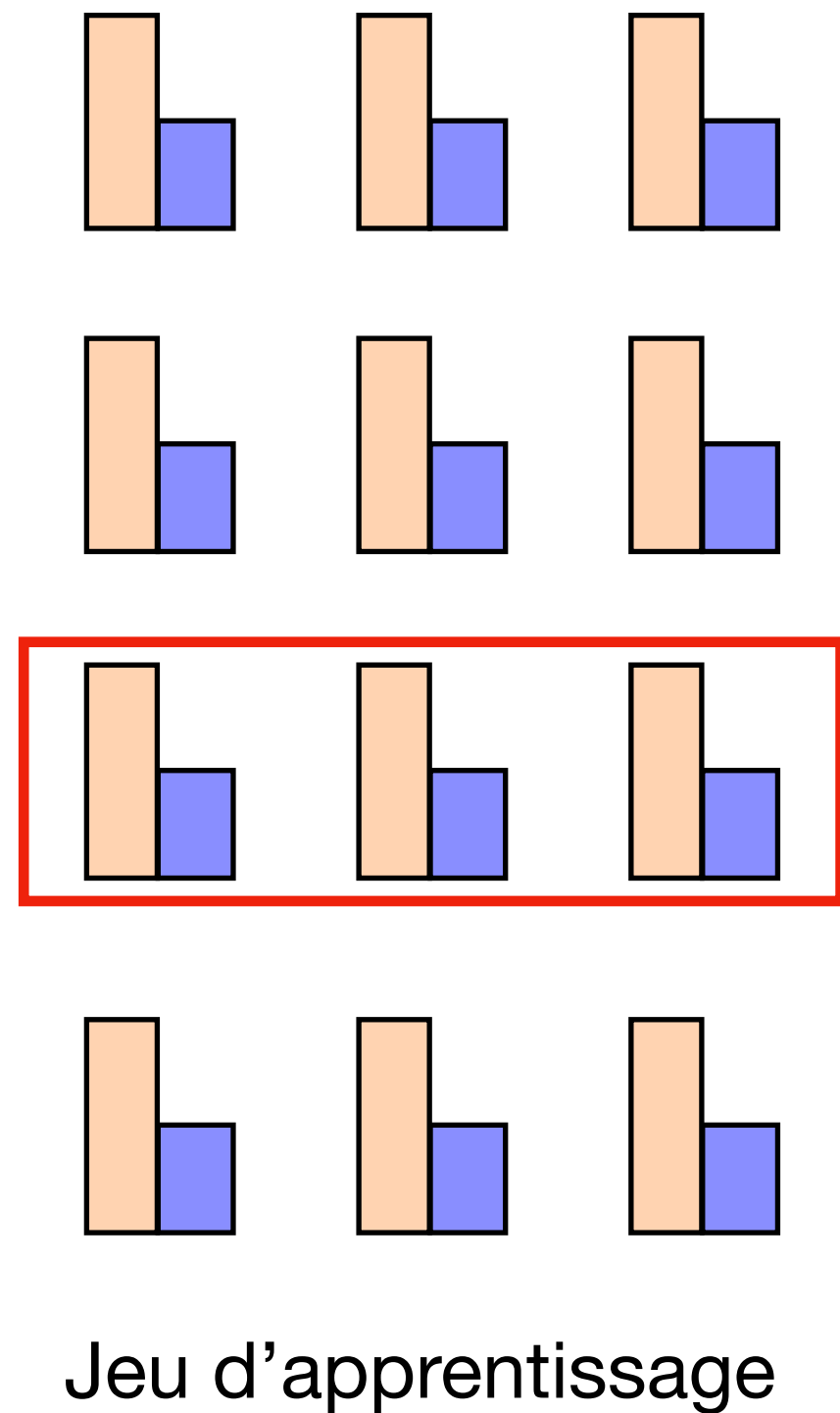


Jeu d'apprentissage



Apprentissage profond

SGD: illustration



Stochastic Gradient Descent

Résumé

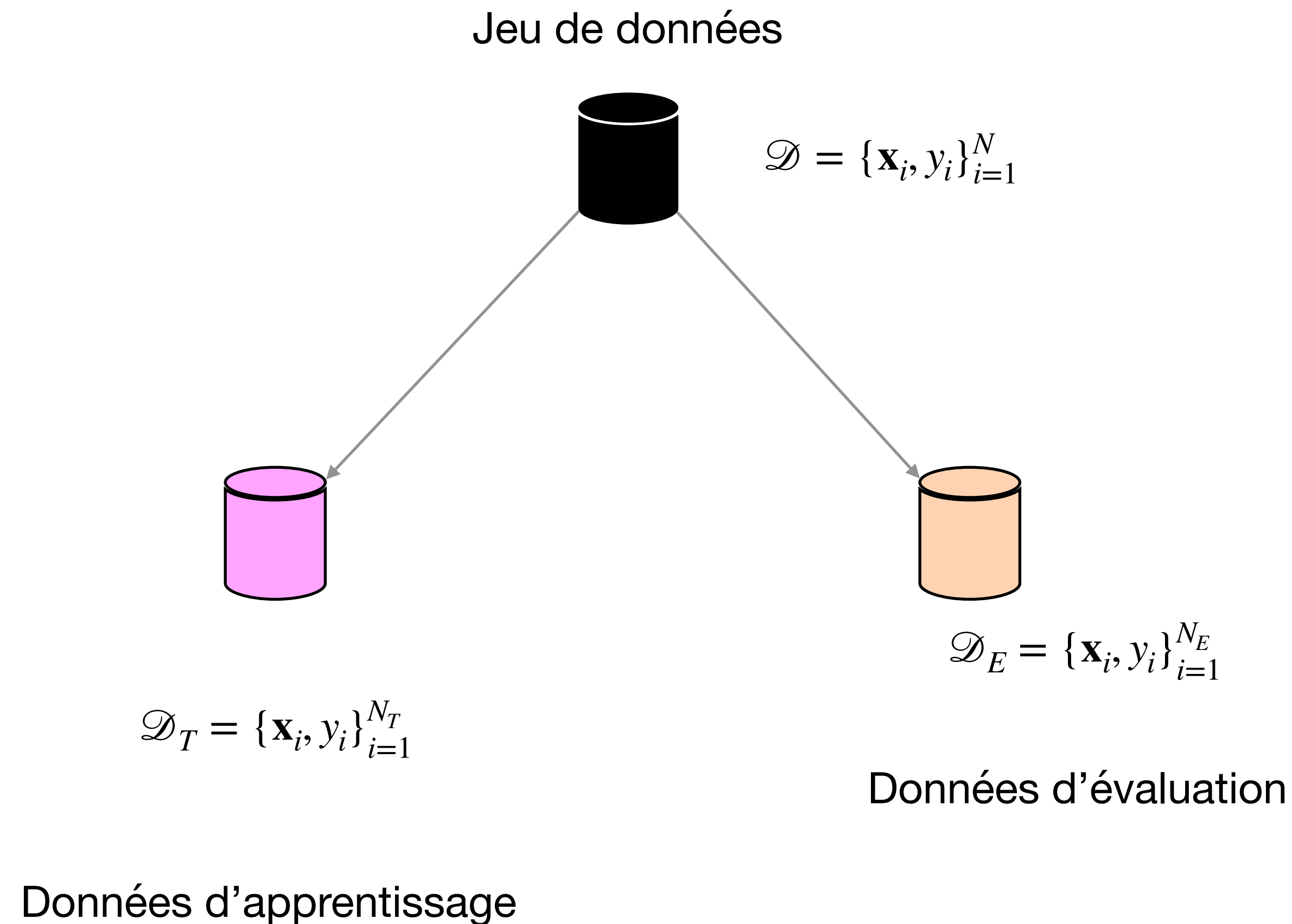
- En machine learning, on cherche à minimiser le risque empirique $\mathbf{f} = J_{\text{emp}}(\theta) := \frac{1}{n} \sum_{i=1}^n \ell(f_{\theta}(\mathbf{x}_i), \mathbf{y}_i)$
- Estimer cette quantité sur un grand jeu de données est très coûteux
- On estime donc le risque empirique (et son gradient) sur des sous ensembles (**mini-batch**)
- La mise à jour des poids se fait en moyennant le gradient de la *loss* sur le mini-batch courant

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \frac{1}{k} \sum_{i=1}^k \nabla \ell(\mathbf{y}_i, \hat{\mathbf{y}}_i)$$

- On observe que si $k \rightarrow n$, la descente de gradient minimise le risque empirique
- Besoin de protocoles de chargement des données !

Gestion des données pour le *Deep Learning*

Préparation des données

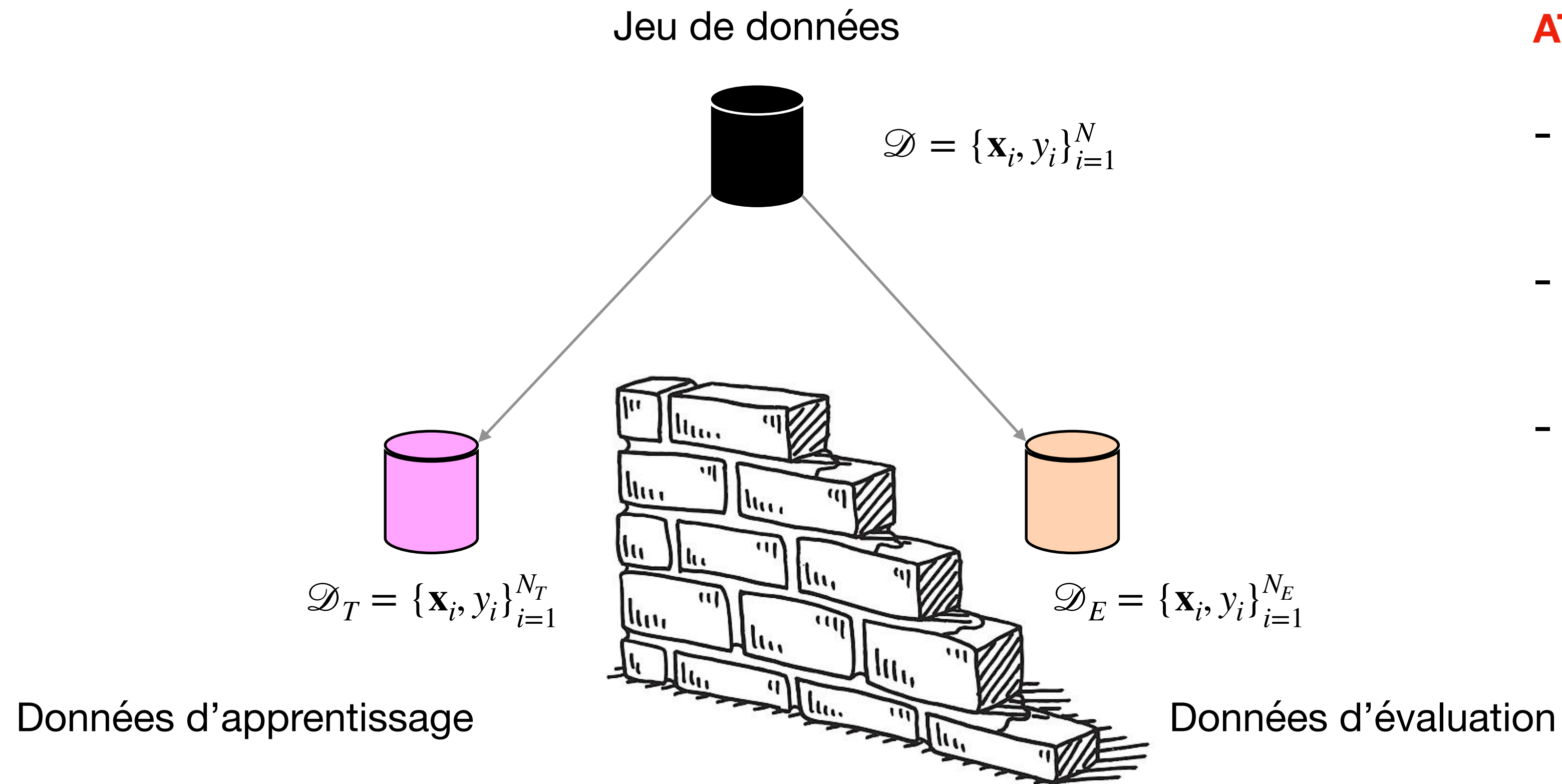


ATTENTION

- Toujours séparer les données d'apprentissage et d'évaluation
- Il ne doit JAMAIS y avoir de fuites entre des jeux de données !
- Vérifiez toujours l'organisation des données avant de lancer un apprentissage

Gestion des données pour le *Deep Learning*

Préparation des données



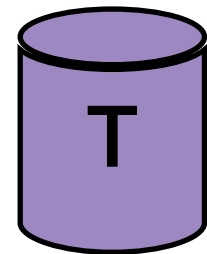
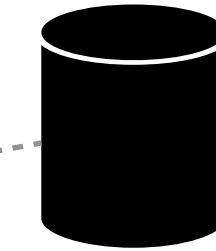
ATTENTION

- Toujours séparer les données d'apprentissage et d'évaluation
- Il ne doit JAMAIS y avoir de fuites entre des jeux de données !
- Vérifiez toujours l'organisation des données avant de lancer un apprentissage

Gestion des données pour le *Deep Learning*

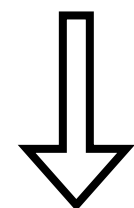
Séparation des jeux de données

$$\mathcal{D} = \{\mathbf{x}_i, y_i\}_{i=1}^N$$



$$\mathcal{D}_T = \{\mathbf{x}_i, y_i\}_{i=1}^{N_T}$$

Jeu d'apprentissage

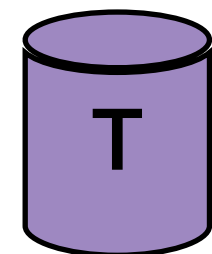
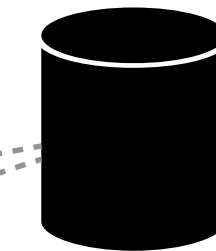


Le seul utilisé pour mettre à
jour les poids θ

Gestion des données pour le *Deep Learning*

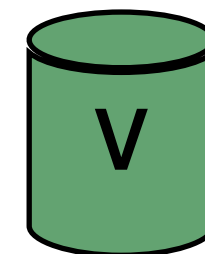
Séparation des jeux de données

$$\mathcal{D} = \{\mathbf{x}_i, y_i\}_{i=1}^N$$



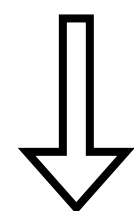
$$\mathcal{D}_T = \{\mathbf{x}_i, y_i\}_{i=1}^{N_T}$$

Jeu d'apprentissage

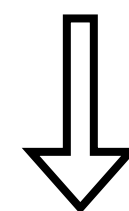


$$\mathcal{D}_V = \{\mathbf{x}_i, y_i\}_{i=1}^{N_V}$$

Jeu de **validation**



Le seul utilisé pour mettre à
jour les poids θ

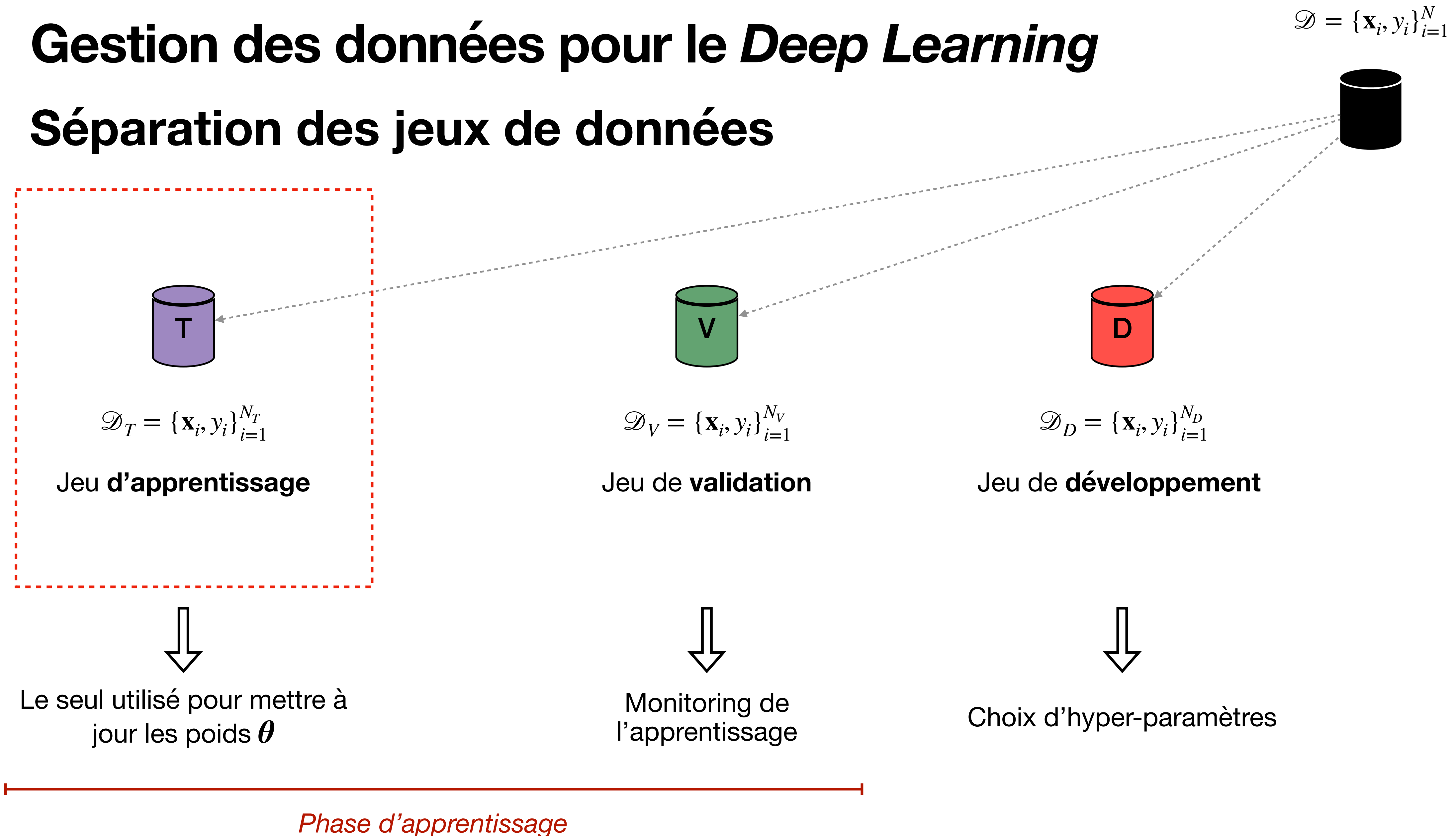


Monitoring de
l'apprentissage

Phase d'apprentissage

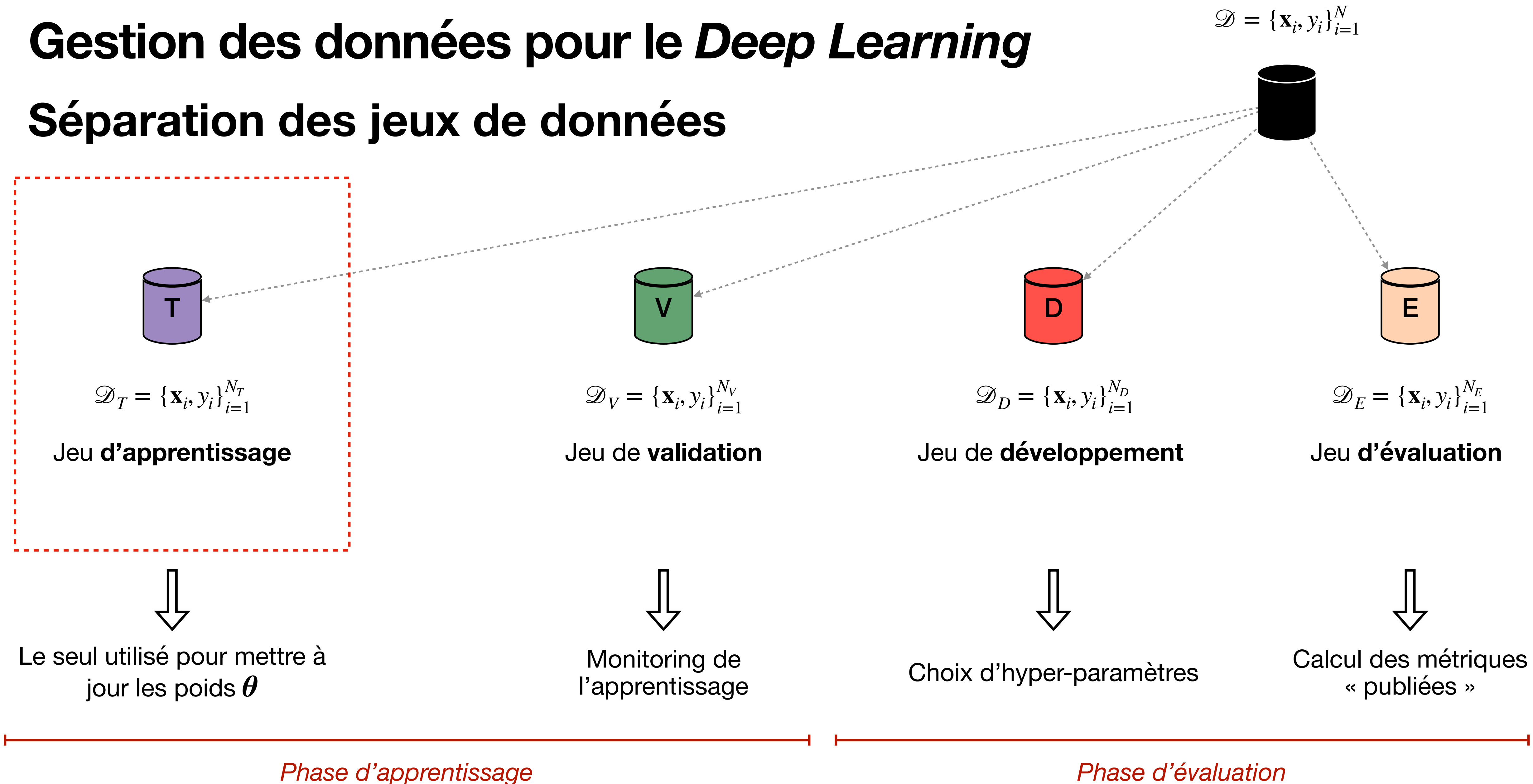
Gestion des données pour le *Deep Learning*

Séparation des jeux de données



Gestion des données pour le *Deep Learning*

Séparation des jeux de données



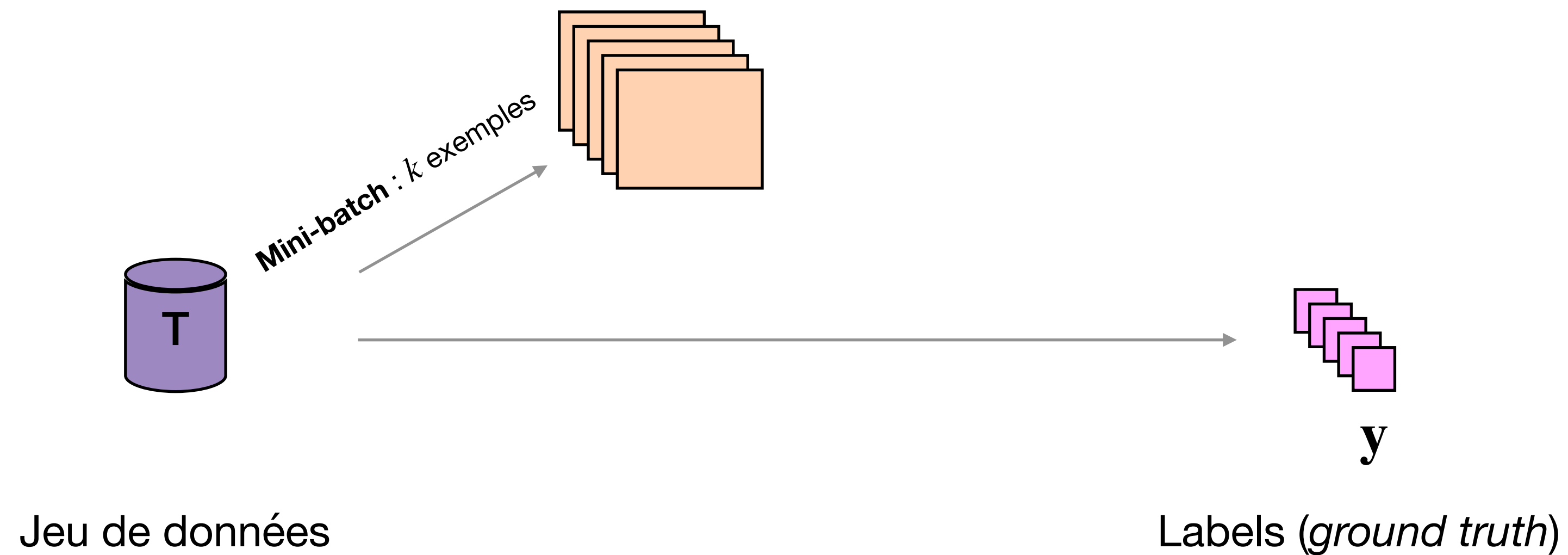
Boucle d'apprentissage

Glossaire

- **Mini-batch** : sous ensemble du jeu d'apprentissage utilisé pour mettre à jour les poids du modèle par SGD
- **Epoch** : fait de faire passer l'ensemble des données d'apprentissage (tous les mini-batch) pour optimiser le modèle
- **Loss** : fonction d'erreur utilisée pour quantifier l'erreur du modèle
- **Optimiseur** : algorithme utilisé pour mettre à jour les poids du modèle
- **Learning rate** : pas d'avancement de la descente de gradient
- **Forward step** : passage des données dans le modèle
- **Backward step** : mise à jour des poids du modèle (optimisation)

Apprentissage d'un modèle de *Deep Learning*

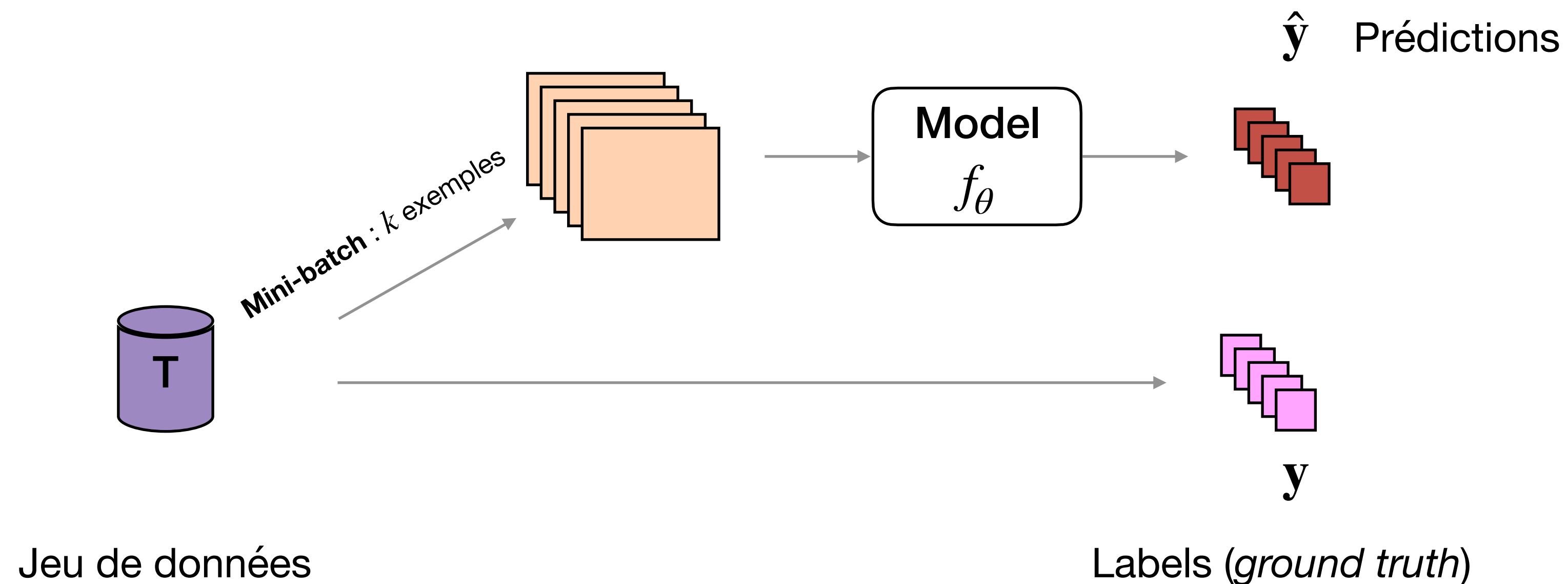
Construction d'un batch



```
for data, label in dataloader:
```

Apprentissage d'un modèle de *Deep Learning*

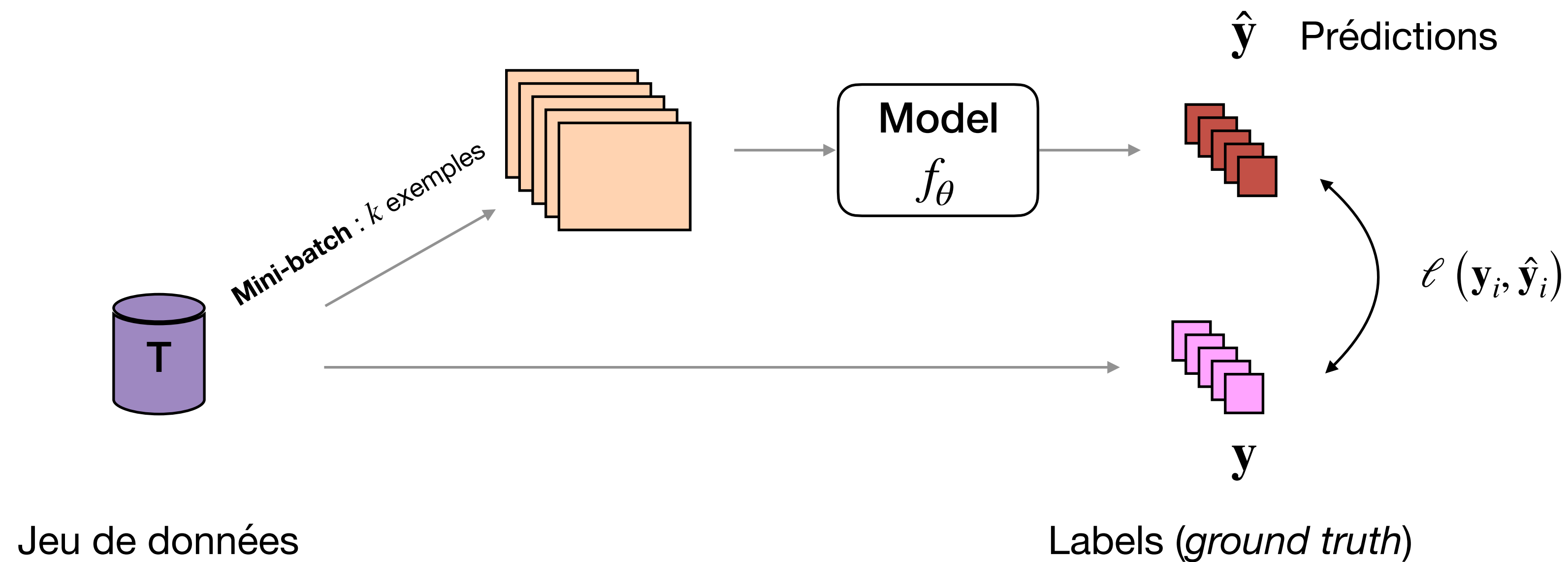
Prédiction pour chaque élément du batch (*forward step*)



```
pred = model(data)
```

Apprentissage d'un modèle de *Deep Learning*

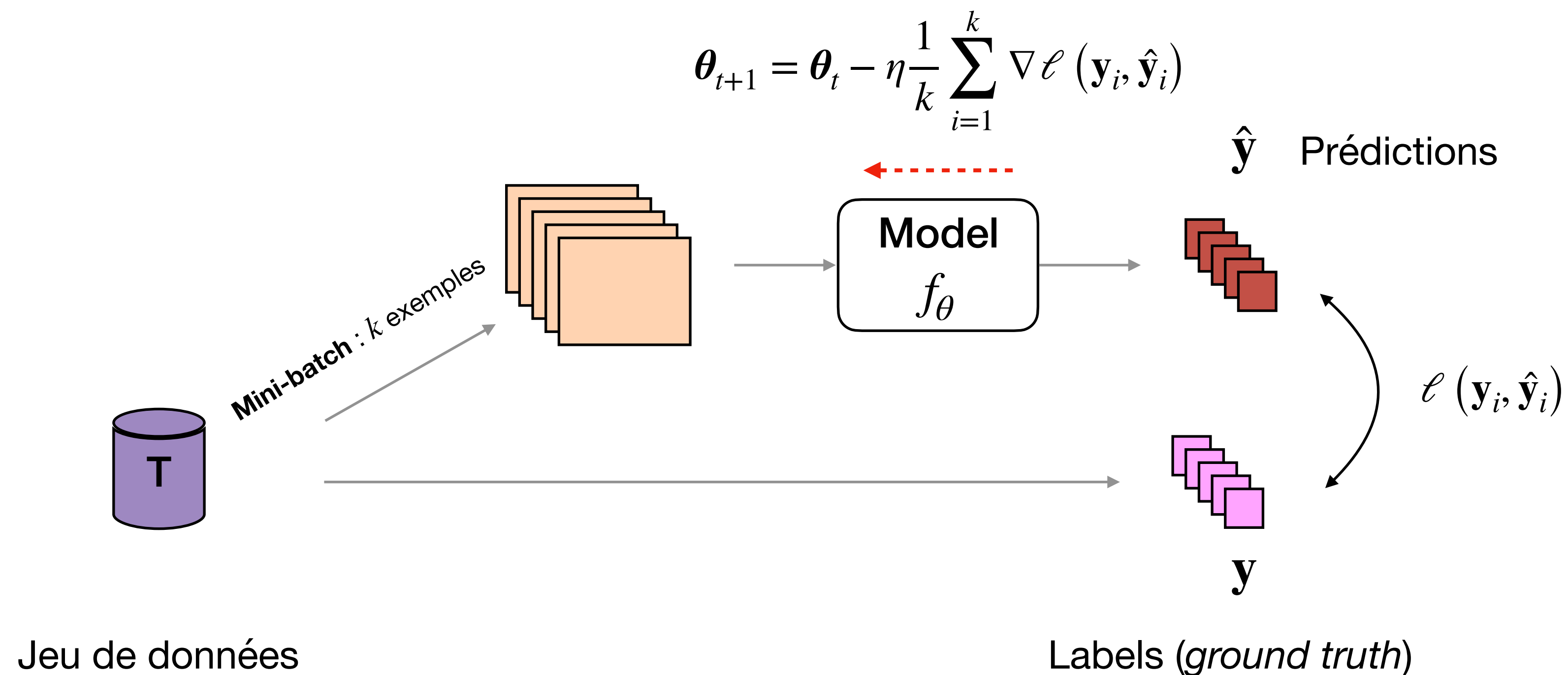
Calcul de l'erreur pour chaque élément (*loss*)



```
loss = my_loss(pred, label)
```

Apprentissage d'un modèle de *Deep Learning*

Mise à jour des poids du modèle (SGD)



```
my_loss.backward()  
optimizer.step()
```

Boucle d'apprentissage

Putting all together

```
import torch
my_loss = OneLoss()
model = MyModel()
optimizer = torch.optim.SGD(model.parameters, lr=0.001)
dataset = MyDataset()
train_loader = torch.utils.data.DataLoader(dataset, batch_size=32)
n_epoch = 100
for epoch in range(n_epoch):
    for data, label in train_loader:
        optimizer.zero_grad()
        pred = model(data)
        loss = my_loss(pred, label)
        loss.backward()
        optimizer.step()
```

Boucle d'apprentissage

Putting all together

```
import torch
my_loss = OneLoss()
model = MyModel()
optimizer = torch.optim.SGD(model.parameters, lr=0.001)
dataset = MyDataset()
train_loader = torch.utils.data.DataLoader(dataset, batch_size=32)
n_epoch = 100
for epoch in range(n_epoch):
    for data, label in train_loader:
        optimizer.zero_grad()
        pred = model(data)
        loss = my_loss(pred, label)
        loss.backward()
        optimizer.step()
```

Boucle d'apprentissage

Putting all together

```
import torch
my_loss = OneLoss()
model = MyModel()
optimizer = torch.optim.SGD(model.parameters, lr=0.001)
dataset = MyDataset()
train_loader = torch.utils.data.DataLoader(dataset, batch_size=32)
n_epoch = 100
for epoch in range(n_epoch):
    for data, label in train_loader:
        optimizer.zero_grad()
        pred = model(data)
        loss = my_loss(pred, label)
        loss.backward()
        optimizer.step()
```

Boucle d'apprentissage

Putting all together

```
import torch
my_loss = OneLoss()
model = MyModel()
optimizer = torch.optim.SGD(model.parameters, lr=0.001)
dataset = MyDataset()
train_loader = torch.utils.data.DataLoader(dataset, batch_size=32)
n_epoch = 100
for epoch in range(n_epoch):
    for data, label in train_loader:
        optimizer.zero_grad()
        pred = model(data)
        loss = my_loss(pred, label)
        loss.backward()
        optimizer.step()
```

Boucle d'apprentissage

Putting all together

```
import torch
my_loss = OneLoss()
model = MyModel()
optimizer = torch.optim.SGD(model.parameters, lr=0.001)
dataset = MyDataset()
train_loader = torch.utils.data.DataLoader(dataset, batch_size=32)
n_epoch = 100
for epoch in range(n_epoch):
    for data, label in train_loader:
        optimizer.zero_grad()
        pred = model(data)
        loss = my_loss(pred, label)
        loss.backward()
        optimizer.step()
```

Boucle d'apprentissage

Putting all together

```
import torch
my_loss = OneLoss()
model = MyModel()
optimizer = torch.optim.SGD(model.parameters, lr=0.001)
dataset = MyDataset()
train_loader = torch.utils.data.DataLoader(dataset, batch_size=32)
n_epoch = 100
for epoch in range(n_epoch):
    for data, label in train_loader:
        optimizer.zero_grad()
        pred = model(data)
        loss = my_loss(pred, label)
        loss.backward()
        optimizer.step()
```

Boucle d'apprentissage

Putting all together

```
import torch
my_loss = OneLoss()
model = MyModel()
optimizer = torch.optim.SGD(model.parameters, lr=0.001)
dataset = MyDataset()
train_loader = torch.utils.data.DataLoader(dataset, batch_size=32)
n_epoch = 100
for epoch in range(n_epoch):
    for data, label in train_loader:
        optimizer.zero_grad()
        pred = model(data)
        loss = my_loss(pred, label)
        loss.backward()
        optimizer.step()
```

Boucle d'apprentissage

Putting all together

```
import torch
my_loss = OneLoss()
model = MyModel()
optimizer = torch.optim.SGD(model.parameters, lr=0.001)
dataset = MyDataset()
train_loader = torch.utils.data.DataLoader(dataset, batch_size=32)
n_epoch = 100
for epoch in range(n_epoch):
    for data, label in train_loader:
        optimizer.zero_grad()
        pred = model(data)
        loss = my_loss(pred, label)
        loss.backward()
        optimizer.step()
```

Boucle d'apprentissage

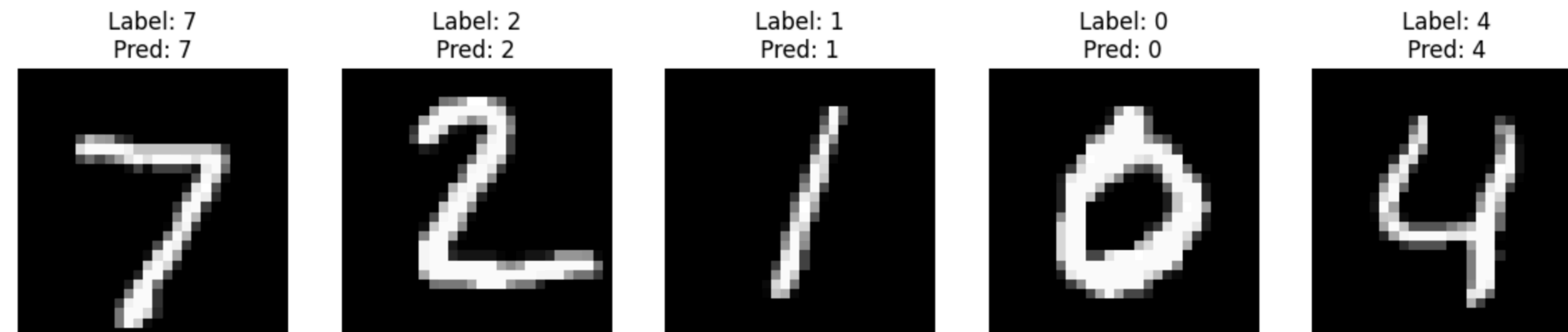
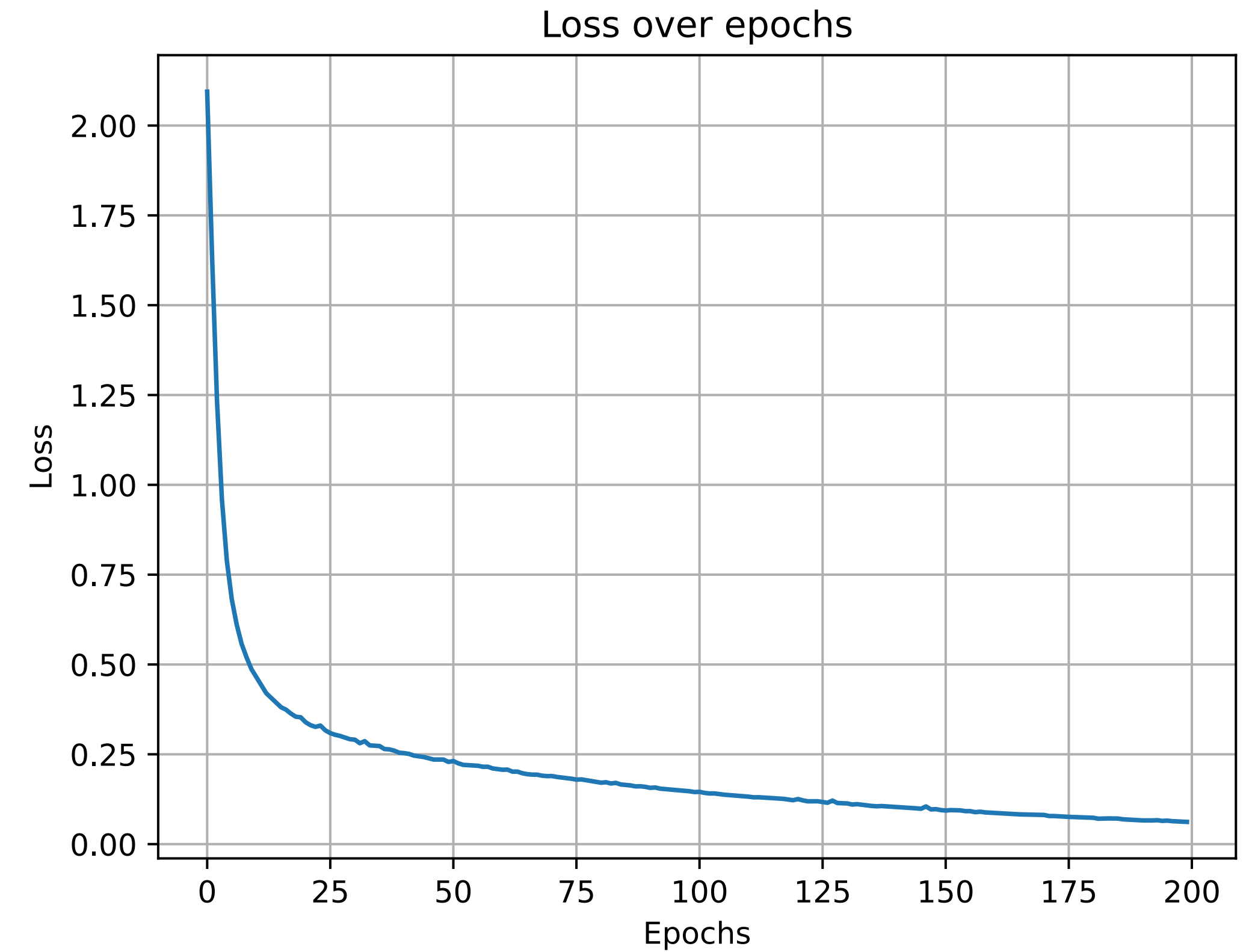
Un exemple avec MNIST

- Identifier les différents éléments de la boucle d'apprentissage
- Discuter les paramètres du Dataloader
- Discuter les limites

Boucle d'apprentissage

Putting all together

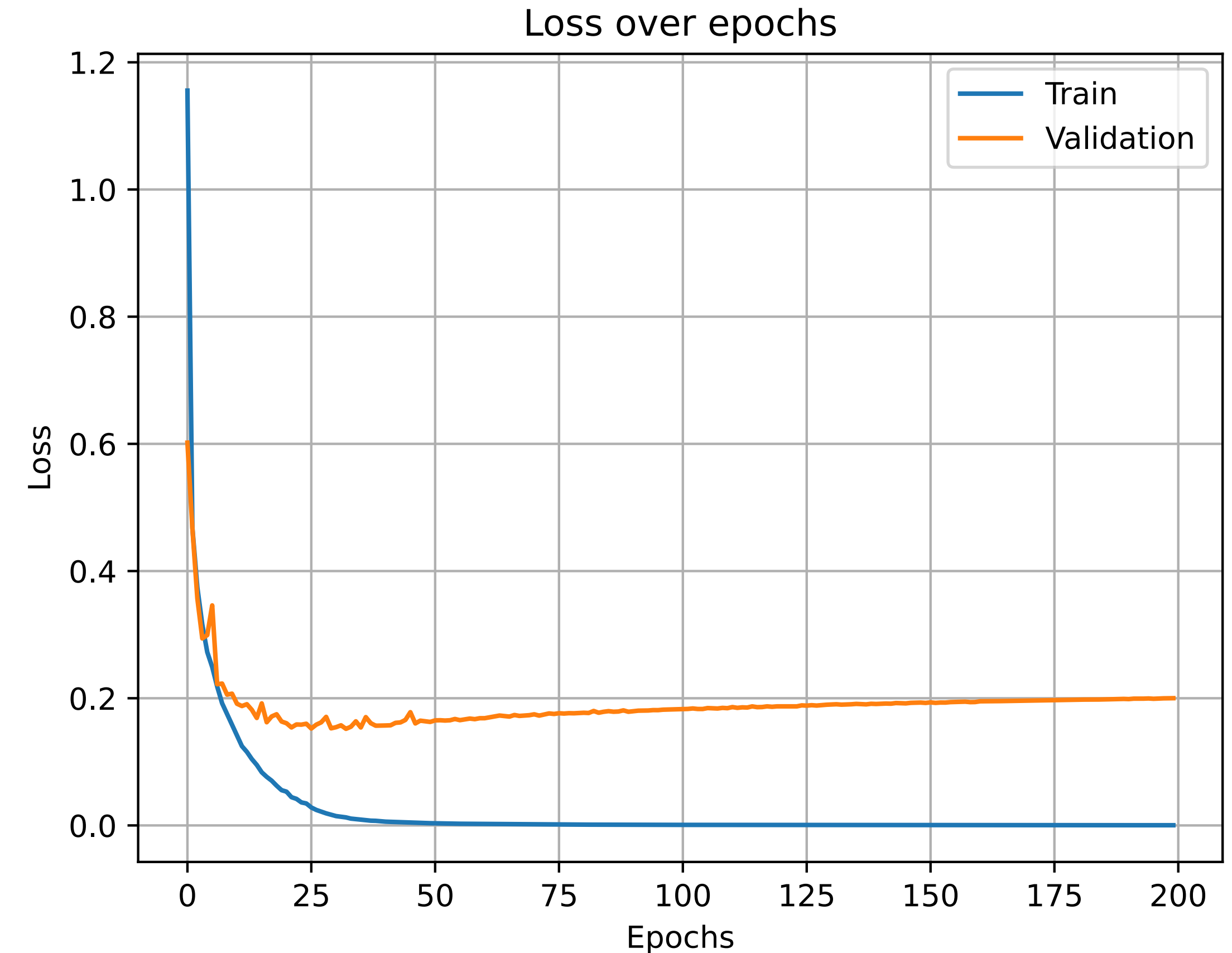
- Magnifique, ma loss converge !
- J'ai une accuracy de 91,1% !
- Quelles limites ?



Boucle d'apprentissage

Importance de la validation

- La loss de validation permet d'estimer le comportement du modèle sur des données non vues lors de l'apprentissage
- Quand peut-on arrêter l'apprentissage ?
- La validation est une bonne pratique pour éviter le sur-apprentissage de vis modèles
- Utiliser le *early stopping* !



Boucle d'apprentissage

Boucle complète avec validation

```
import tqdm
import copy
log_loss = []
train_loss_epoch = []
val_loss_epoch = []
train_dataset, val_dataset = random_split(train_dataset,
[train_size, val_size])
train_loader = torch.utils.data.Dataloader(train_dataset,
batch_size=16, shuffle = True, drop_last=True)
val_loader = torch.utils.data.Dataloader(train_dataset,
batch_size=16, shuffle = False)

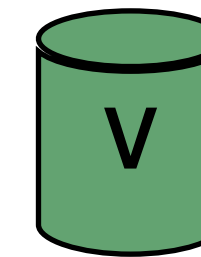
for epoch in tqdm.tqdm(range(num_epochs)):
    avg_loss = 0
    for batch_idx, (data, label) in enumerate(train_loader):
        data, label = data, label

        # Forward pass
        pred = model(data)
        loss = criterion(pred, label)

        # Backward pass
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    avg_loss += loss.item()
    log_loss.append(loss.item())
    avg_loss /= len(train_loader)
    train_loss_epoch.append(avg_loss)
```

On ajoute ensuite la validation

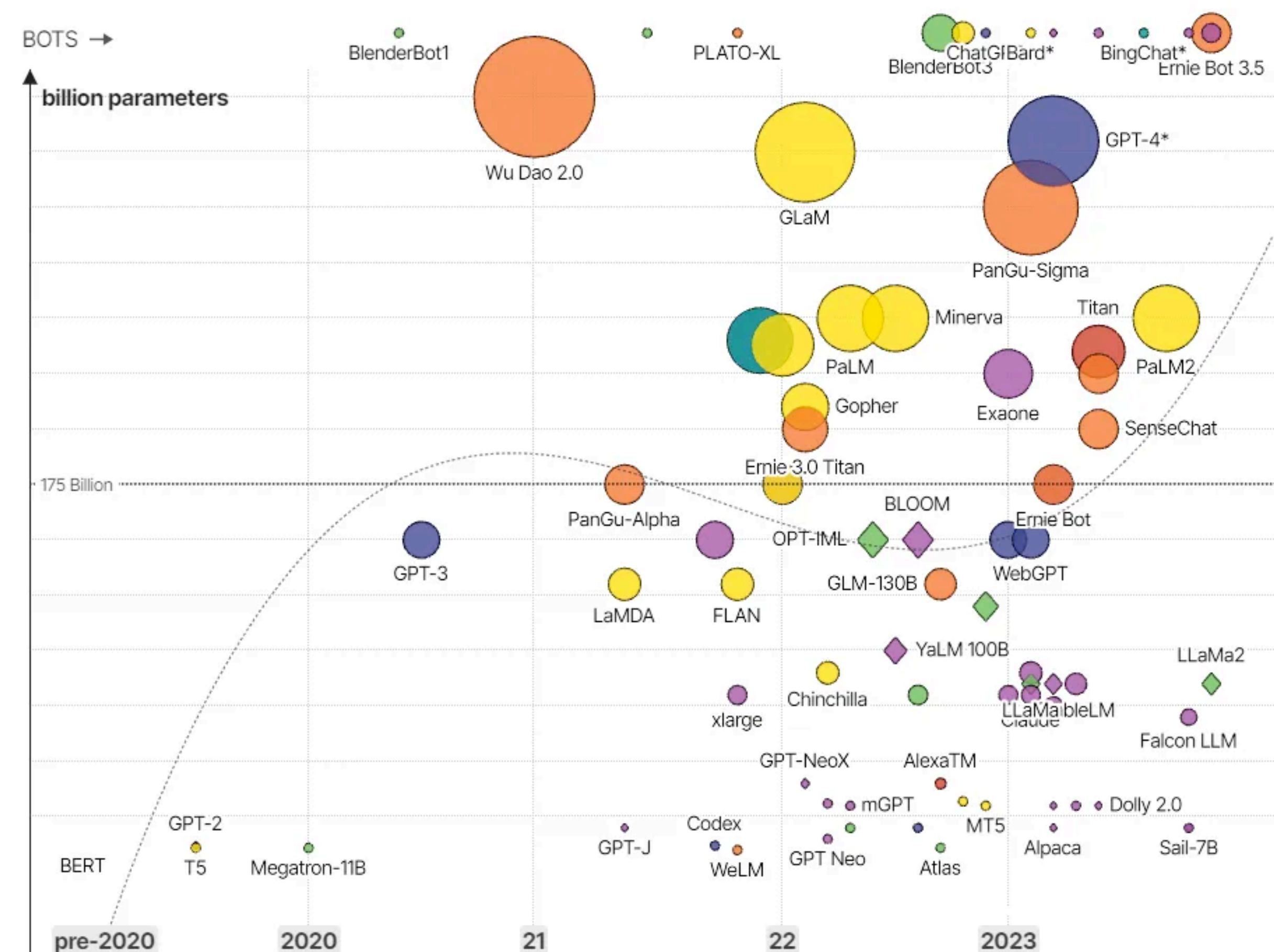


```
model.eval()
val_loss = 0
with torch.no_grad():
    for data, label in val_loader:
        pred = model(data)
        loss = criterion(pred, label)
        val_loss += loss.item()
val_loss /= len(val_loader)
val_loss_epoch.append(val_loss)
model.train()
```

Optimisation de l'apprentissage

Contexte

- Réseaux de neurones profonds = beaucoup de paramètres
- Problème d'optimisation complexe, non-convexe, à hautes dimensions
- Pas de modèle théorique permettant de choisir les meilleurs hyper-paramètres d'apprentissage
- Expérience d'ingénierie = méthodes empiriques pour améliorer l'efficacité de l'optimisation
- **ATTENTION:** pas de fondement théorique, donc pas de garantie que ces techniques fonctionnent dans toutes les situations !



David McCandless, Tom Evans, Paul Barton
Information is Beautiful // UPDATED 27th Jul 23

source: news reports, [LifeArchitect.ai](https://lifeai.org/)
* = parameters undisclosed // see [the data](https://lifeai.org/)

<https://medium.com/@adria.cabello/the-evolution-of-language-models-a-journey-through-time-3179f72ae7eb>

Optimisation de l'apprentissage

Exemple de questions auxquelles nous répondrons

- Mon espace d'optimisation est trop grand, je n'arrive pas à trouver une solution optimale et l'apprentissage est long
- L'apprentissage est instable
- Je résous une tâche connue dans la littérature, et mon réseau ne converge pas !
- Mon modèle ne généralise pas à d'autres données :(
- Quels hyper-paramètres choisir ?

Optimisation de l'apprentissage

Sommaire

1. Algorithmes d'optimisation

1. Importance du learning rate
2. Schedulers
3. Warm-up
4. Optimiseurs

2. Modification de l'espace de recherche

1. Connections résiduelles
2. Choix des valeurs

3. Stabilisation de l'apprentissage

1. Régler la taille de batch
2. Accumuler les gradients
3. Gradient clipping

Optimisation de l'apprentissage

Sommaire

4. Réutiliser l'existant

1. Principe du transfer learning
2. Finetuning
3. Foundation models

5. Améliorer la généralisation

1. Weight-decay
2. Drop-out
3. Augmentation de données

6. Fin de l'apprentissage

1. Early stopping
2. Model checkpointing

Optimisation de l'apprentissage

Sommaire

1. Algorithmes d'optimisation

1. Importance du learning rate
2. Schedulers
3. Warm-up
4. Optimiseurs

2. Modification de l'espace de recherche

1. Connexions résiduelles
2. Choix des valeurs

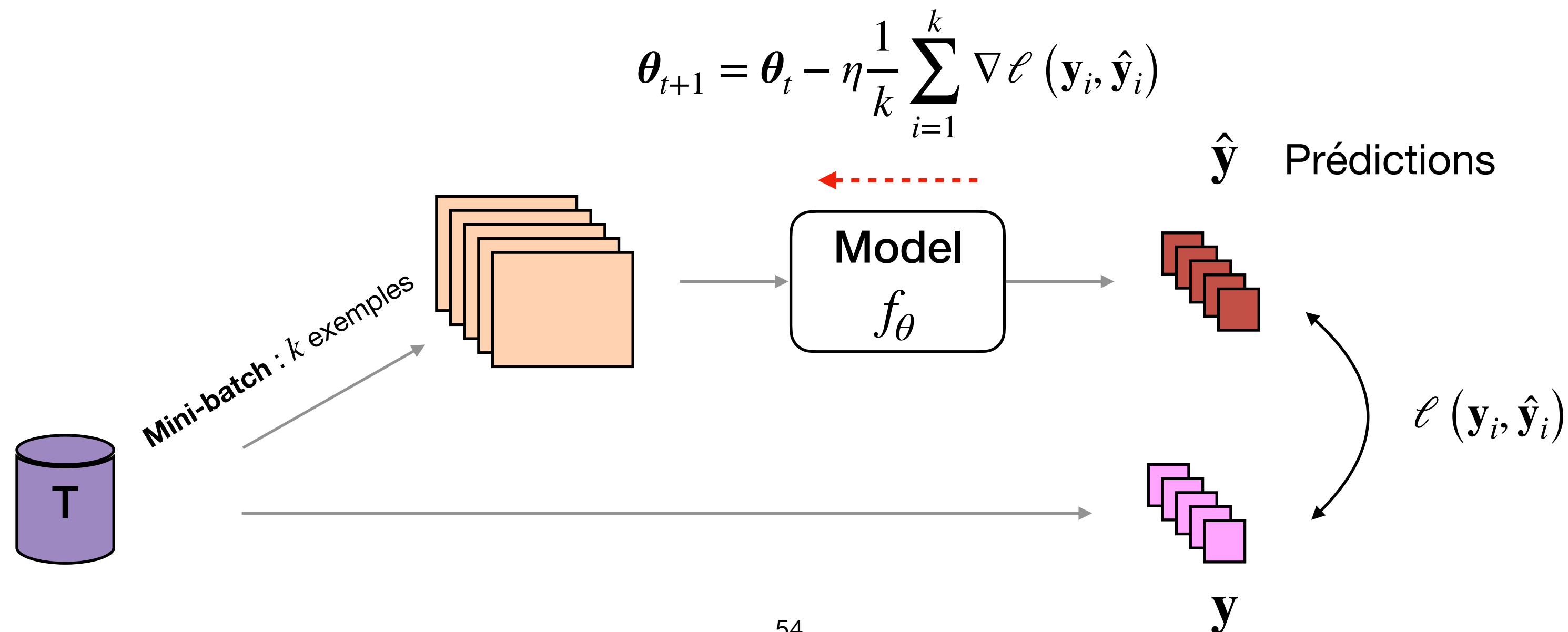
3. Stabilisation de l'apprentissage

1. Régler la taille de batch
2. Accumuler les gradients
3. Gradient clipping

Optimisation des paramètres

Introduction

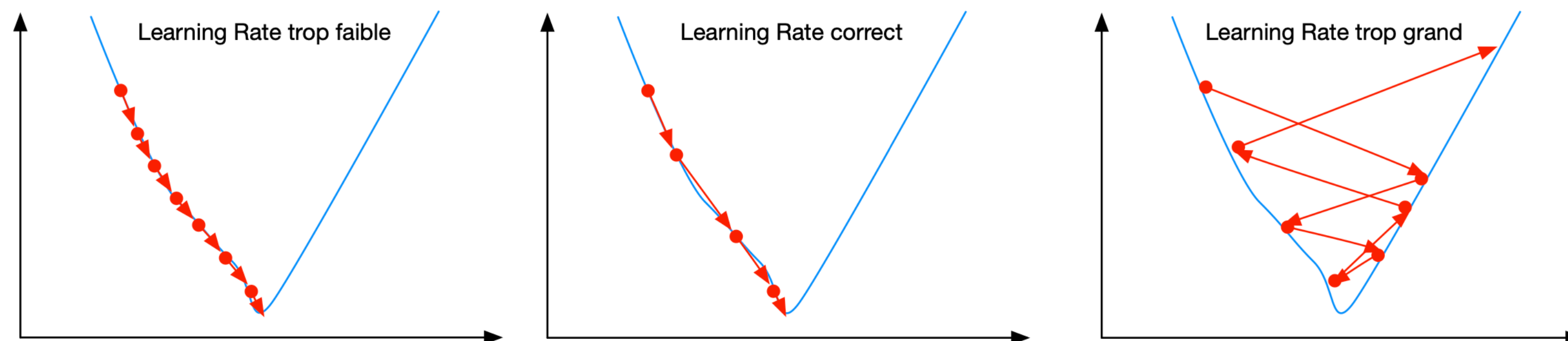
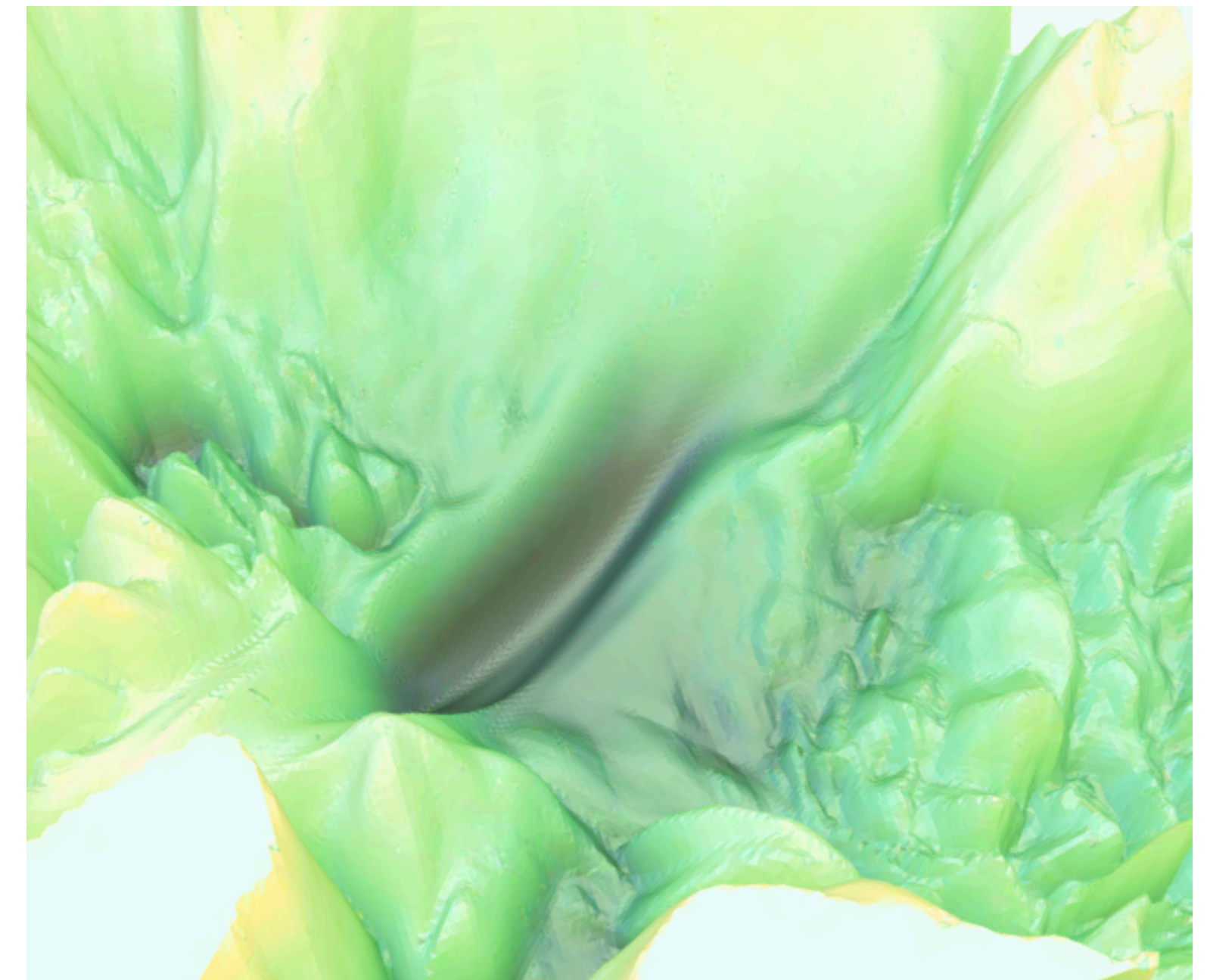
- **Deep learning** = trouver un modèle qui minimise une fonction objectif (*loss*) à partir d'un jeu de données fini
- Stochastic Gradient Descent (SDG) = une méthode d'optimisation permettant de résoudre ce problème
- Paramètres très importants : *learning rate* η et *batch size* k
- Objectif : découvrir des techniques de gestion du learning rate et d'autres méthodes d'optimisation (*optimizers*)



Optimisation des paramètres

Importance du learning rate

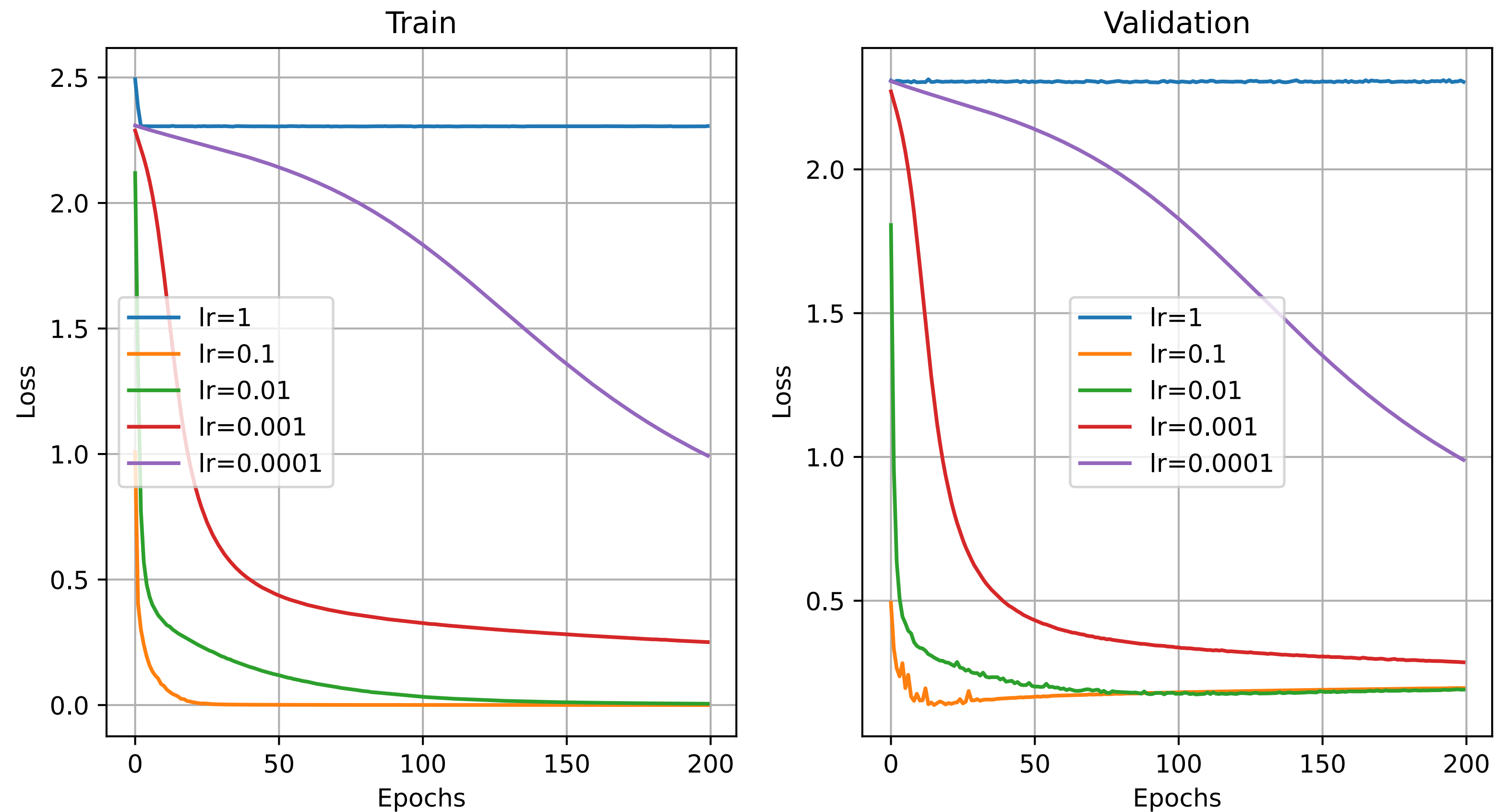
- L'espace des paramètres d'un réseau profond est très complexe
- Le site <https://losslandscape.com/explorer> permet de visualiser l'espace des paramètres (projection 2D)
- Le *learning rate* définit l'amplitude des déplacements dans cet espace, et donc le minimum local trouvé



Le choix du *learning rate* est primordial lors de l'optimisation de votre modèle

Optimisation des paramètres

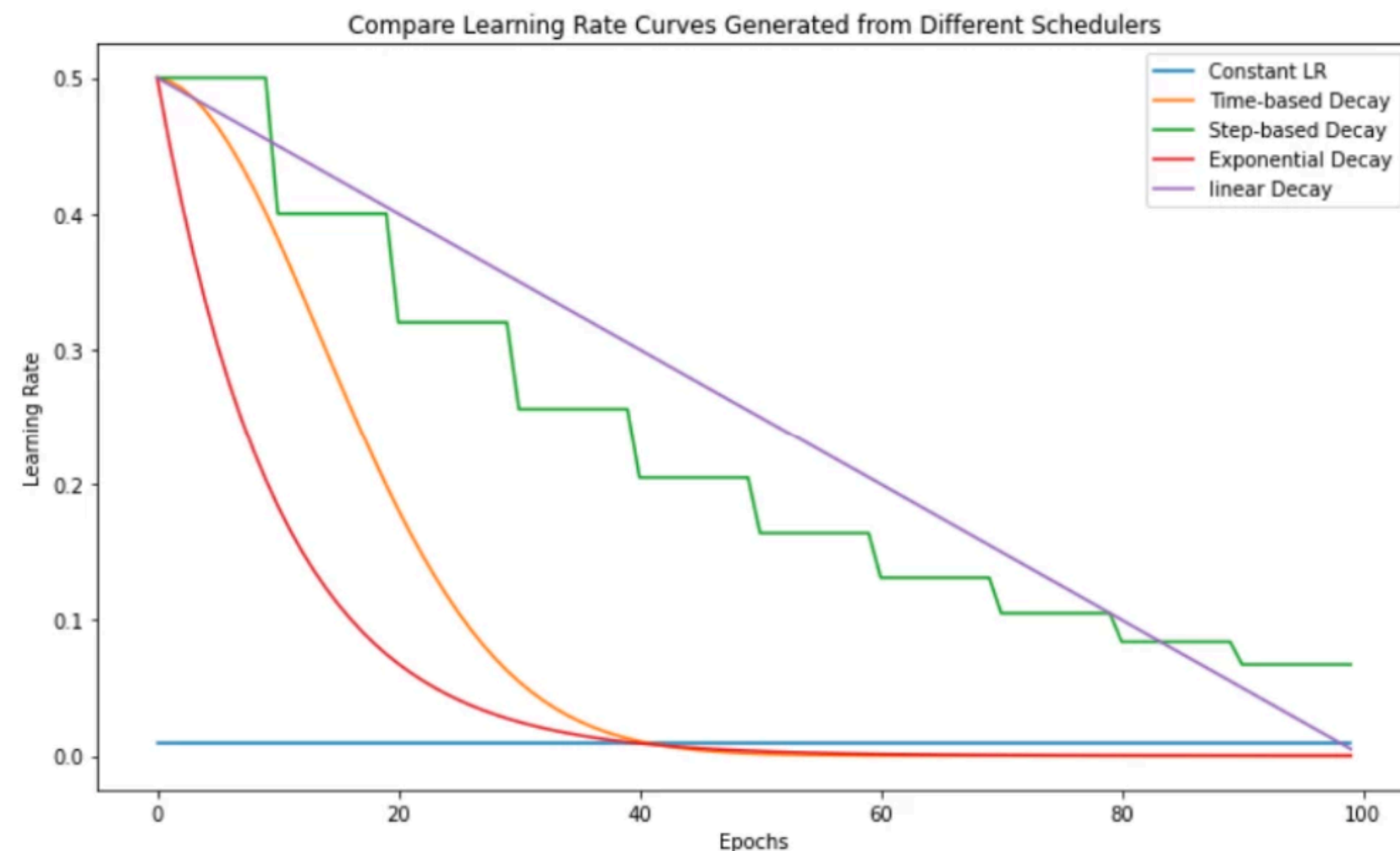
Impact du learning rate : exemple sur MNIST



Optimisation des paramètres

Modifier le learning rate pendant l'apprentissage

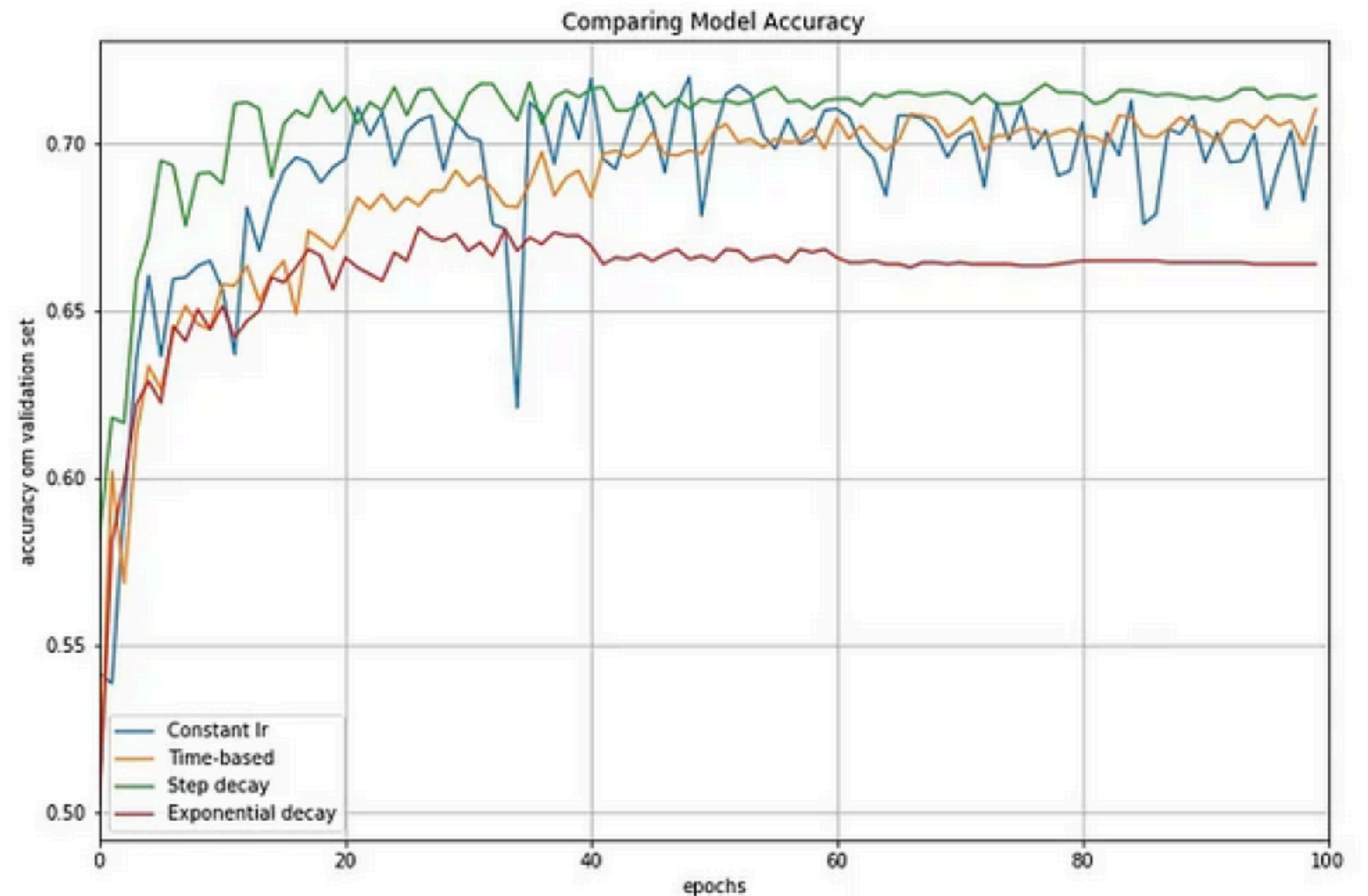
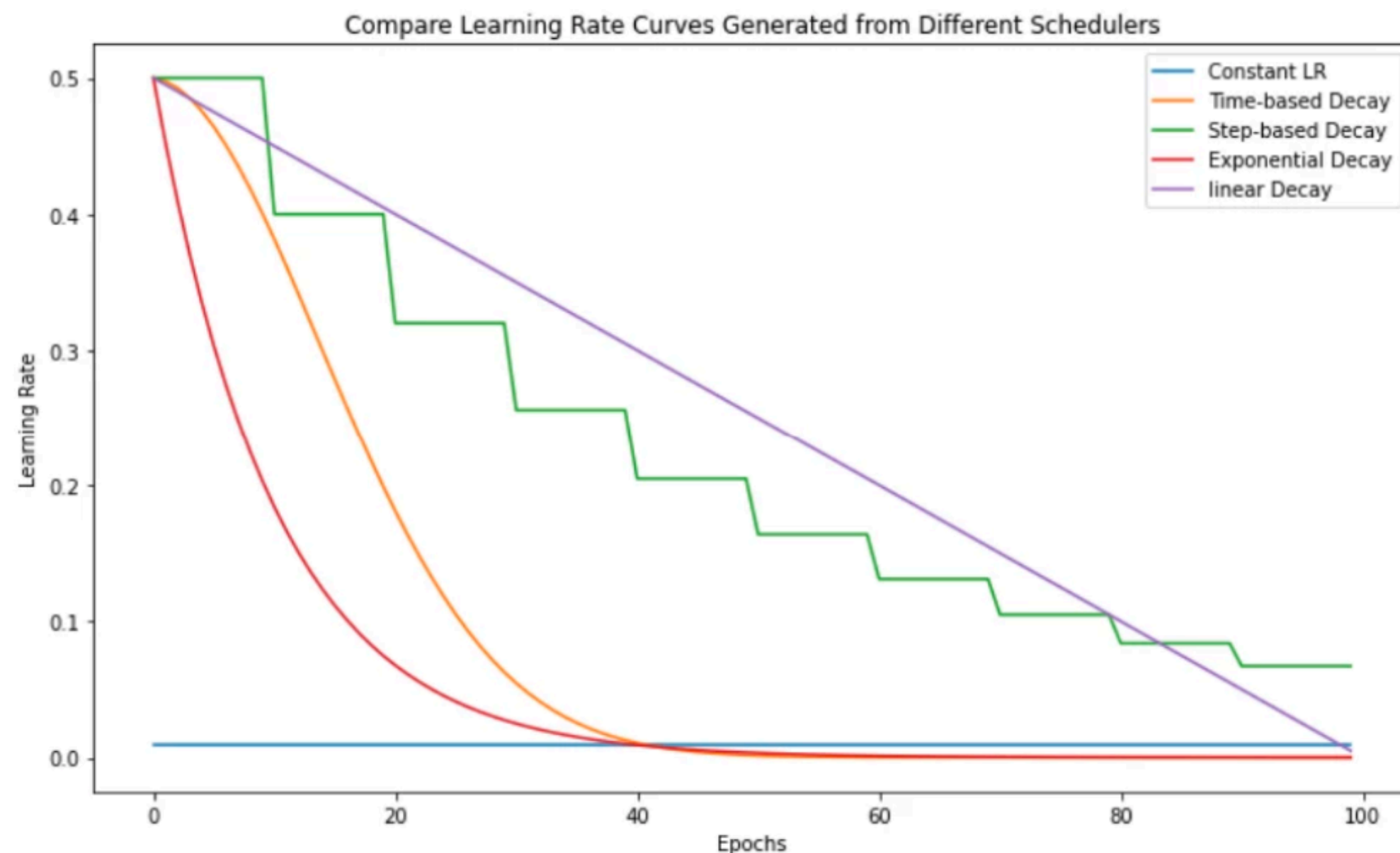
- **Intuition** : on peut améliorer le minimum lorsque la loss n'évolue plus
- **Idée** : réduire le *learning rate* pour avoir un pas plus fin
- Plusieurs possibilités de réduction



Optimisation des paramètres

Modifier le learning rate pendant l'apprentissage

- **Intuition** : on peut améliorer le minimum lorsque la loss n'évolue plus
- **Idée** : réduire le *learning rate* pour avoir un pas plus fin
- Plusieurs possibilités de réduction

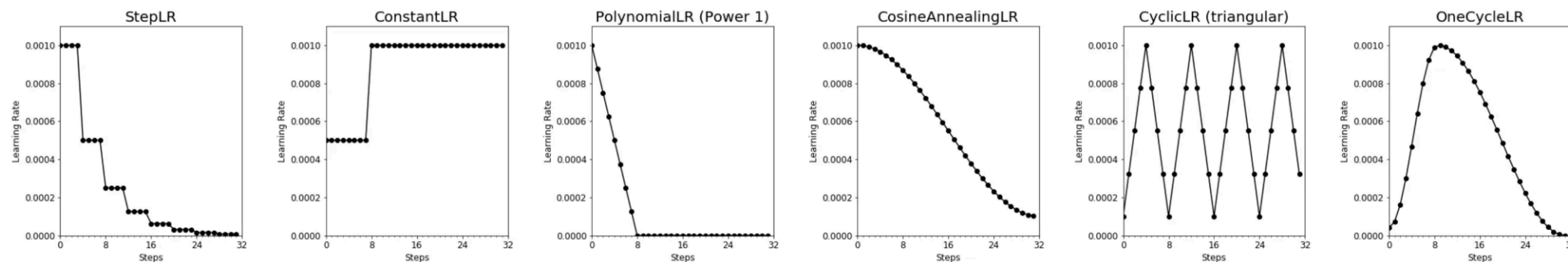


Que peut-on dire de cet exemple ?

Optimisation des paramètres

Schedulers

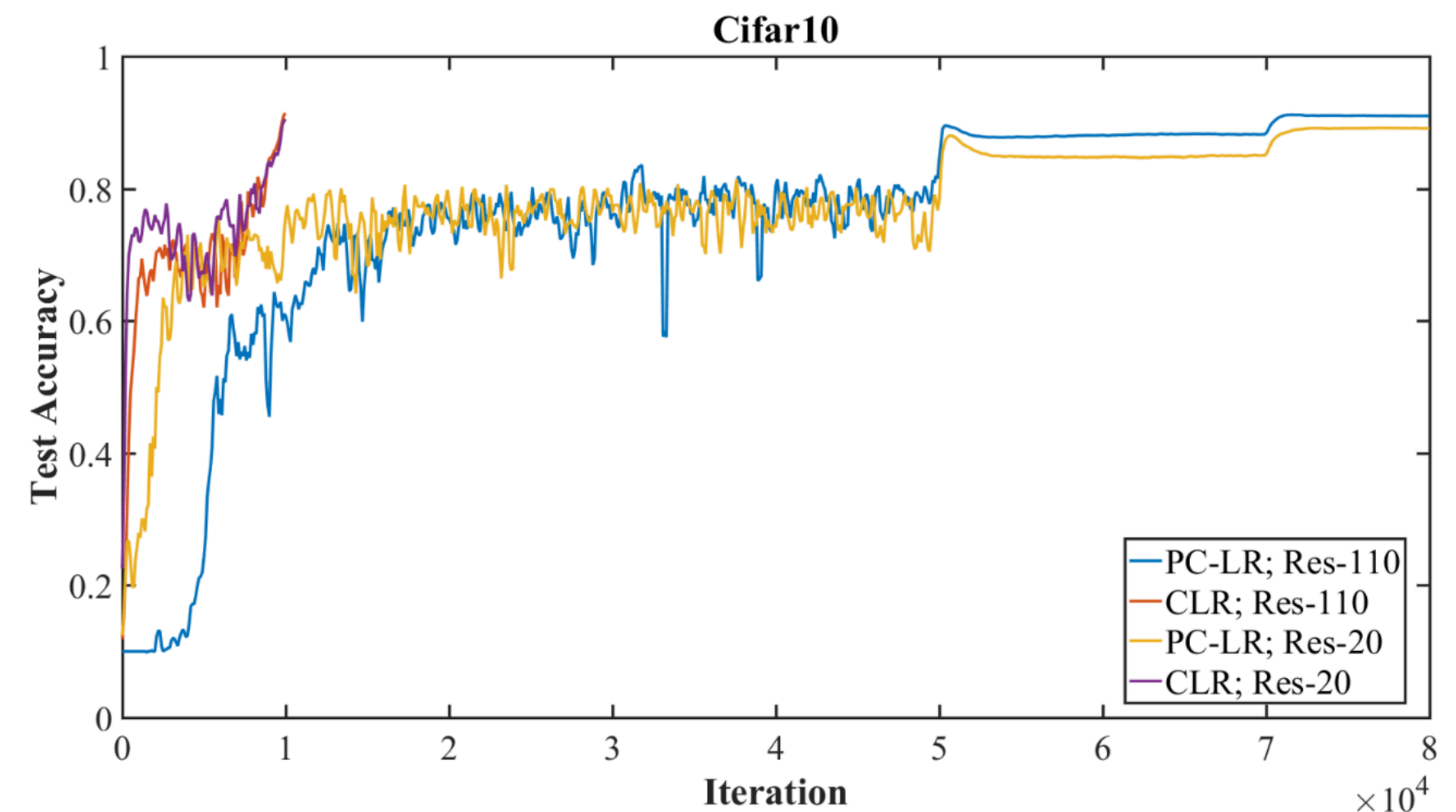
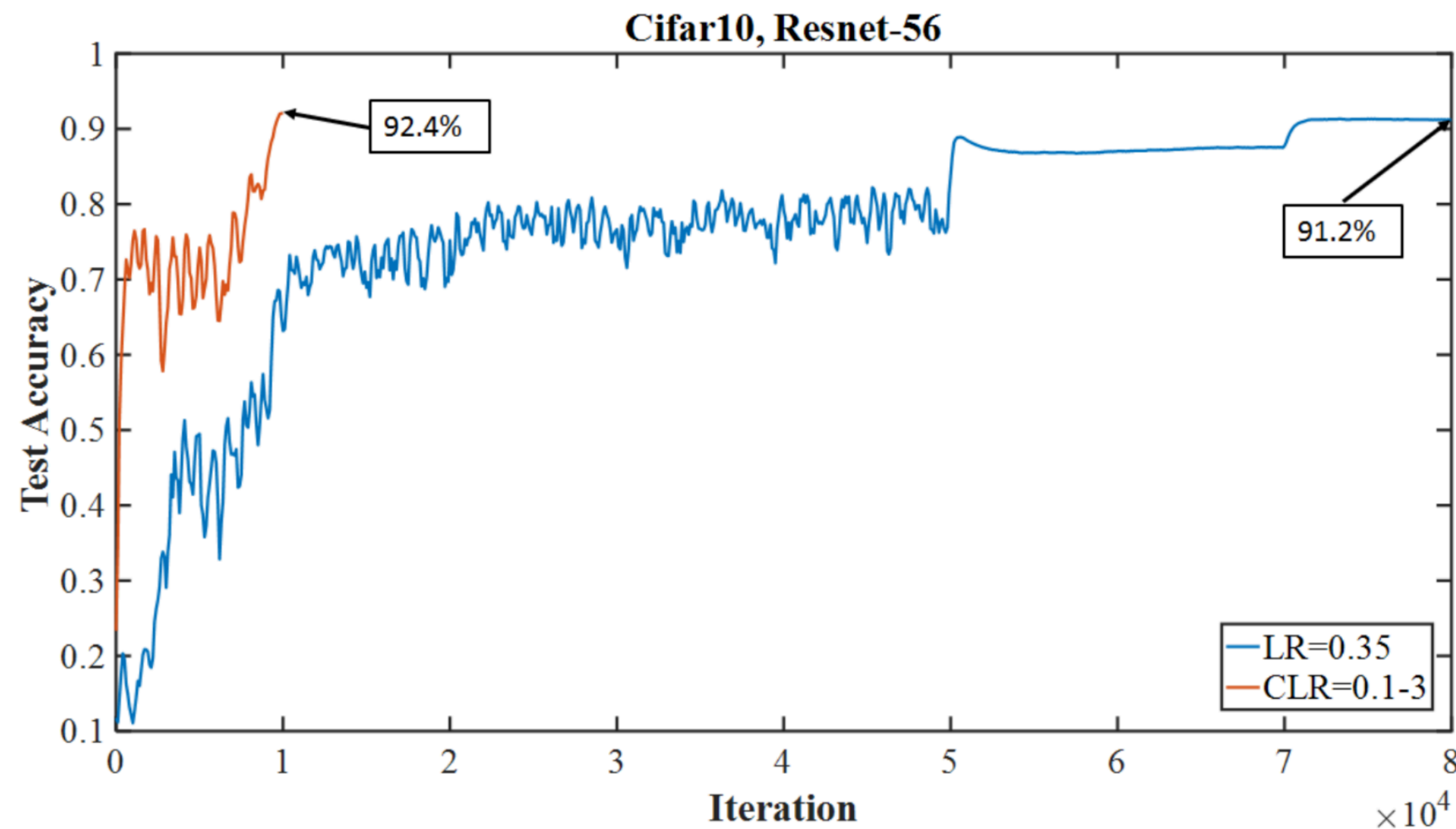
- **Scheduler** = schéma de variation du learning rate au cours de l'apprentissage
- De nombreux schémas existent
- Bien réglés, ces schémas peuvent accélérer la convergence des modèles
- Possibilité d'explorer plusieurs minima locaux dans l'espace des paramètres



Optimisation des paramètres

Schedulers : exemple Smith et al. (2019)

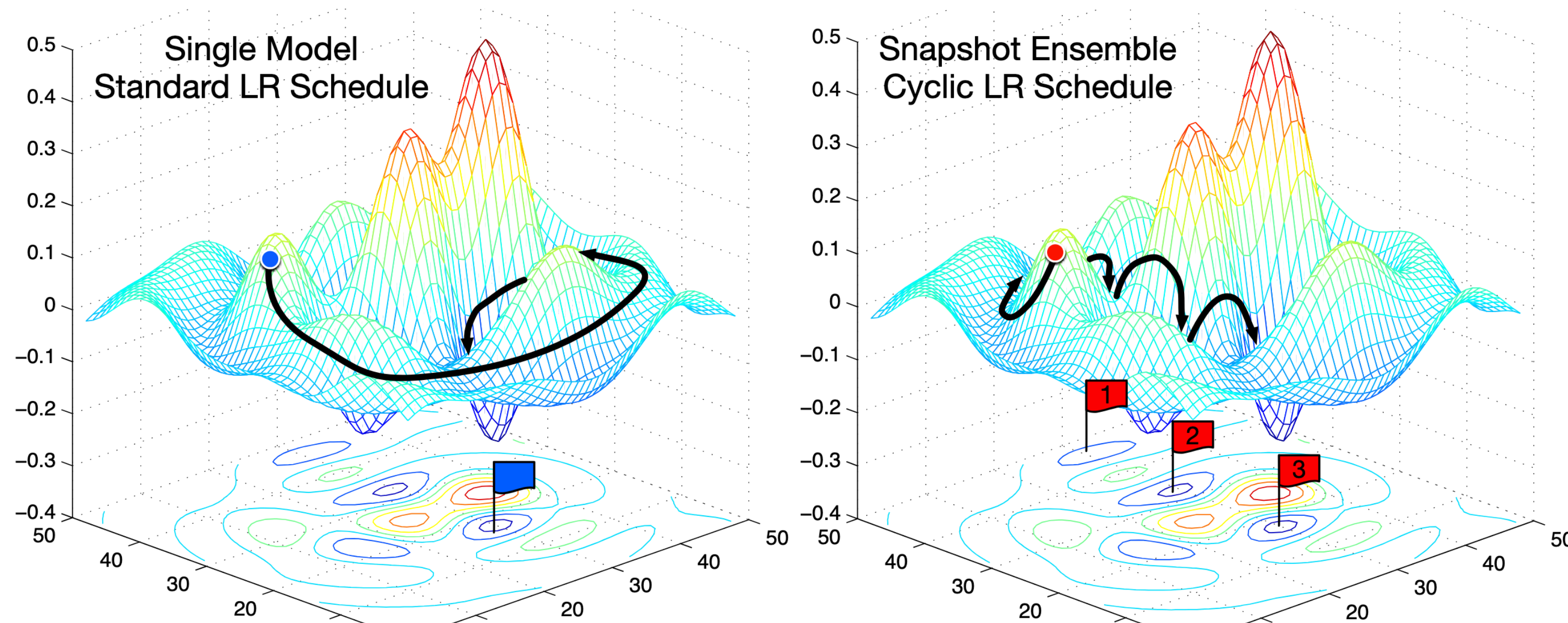
- **Superconvergence** : combinaison d'un grand *learning rate* initial et d'un scheduler cyclique
- Accélère drastiquement la convergence de modèles de classification d'images



Optimisation des paramètres

Schedulers : exemple Huang et al. (2017)

- Exploration de plusieurs minima locaux
 - Permet de trouver plus rapidement un modèle optimal
 - Permet d'apprendre plusieurs modèles optimisés au cours d'un apprentissage (minima différents)



*Huang et al. (2017) SNAPSHOT ENSEMBLES:
TRAIN 1, GET M FOR FREE, ICLR 2017*

Optimisation des paramètres

Schedulers : implémentation PyTorch

- La majorité des scheduler sont existents déjà dans PyTorch

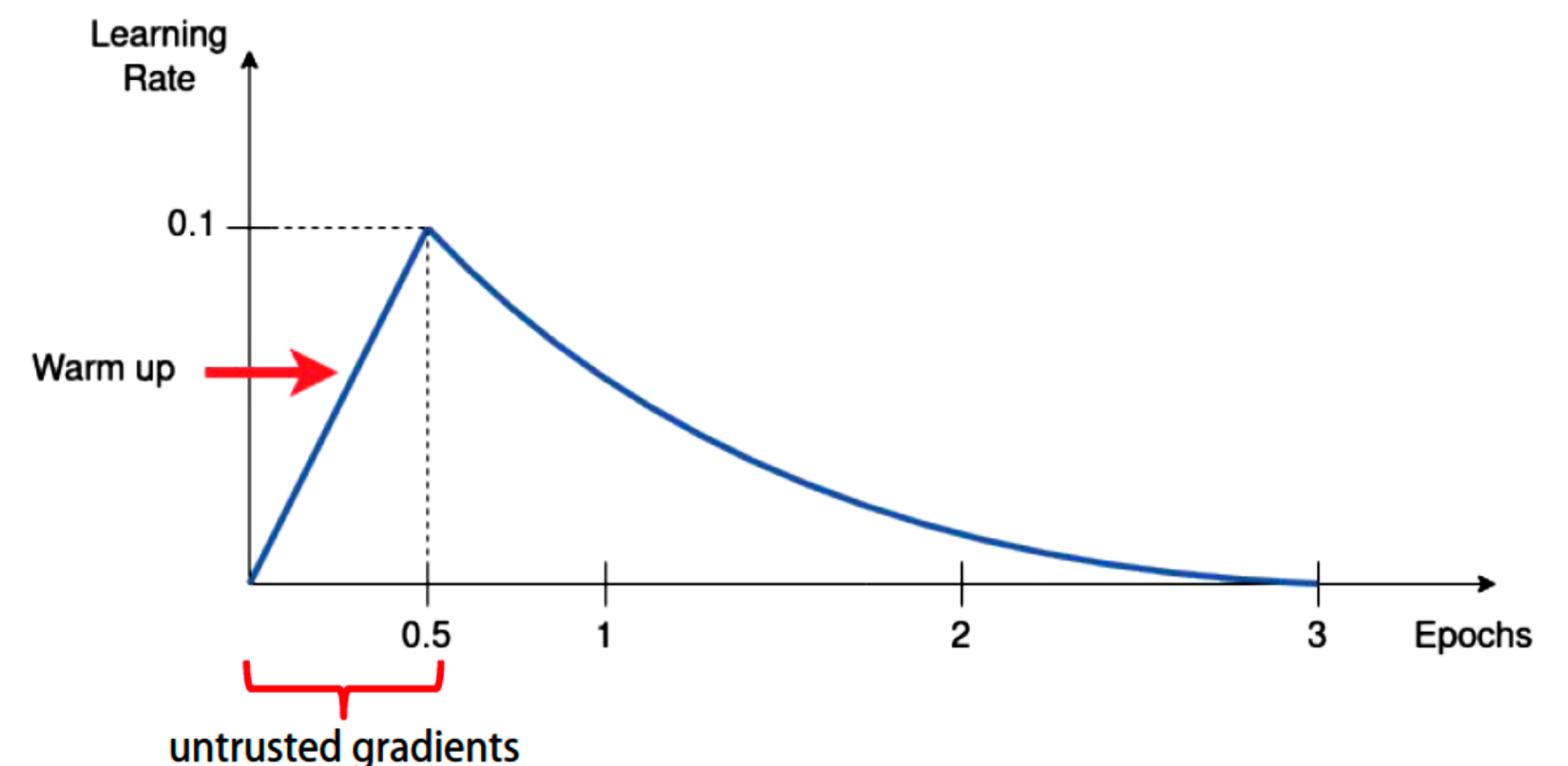
```
>>> scheduler = StepLR(optimizer, step_size=30, gamma=0.1)
>>> for epoch in range(100):
>>>     train(...)
>>>     validate(...)
>>>     scheduler.step()
```

- Le *learning rate* se met à jour automatiquement en fonction de l'étape d'apprentissage
- Il est important de **logger** ce paramètre afin de s'assurer du bon fonctionnement du scheduler !!!
- Toutes les informations sur les schedulers se trouvent ici : <https://pytorch.org/docs/stable/optim.html>
- La mise à jour du scheduler dépend du type (parfois après `optimizer.step()`)

Algorithmes d'optimisation

Warm-up

- Les DNN sont sensibles à l'initialisation
- Les poids initiaux conditionnent fortement la convergence du modèle vers un bon minimum
- **Poids aléatoires = erreur élevée = gradient élevé = fort impact sur la mise à jour des poids**
- *Warm up* = échauffer son modèle
 - On utilise un *learning rate* plus petit au début puis on augmente pour limiter l'influence de l'initialisation
 - Scheduler cyclique = intègre déjà cette étape
- S'applique à tout scheduler dans Pytorch¹

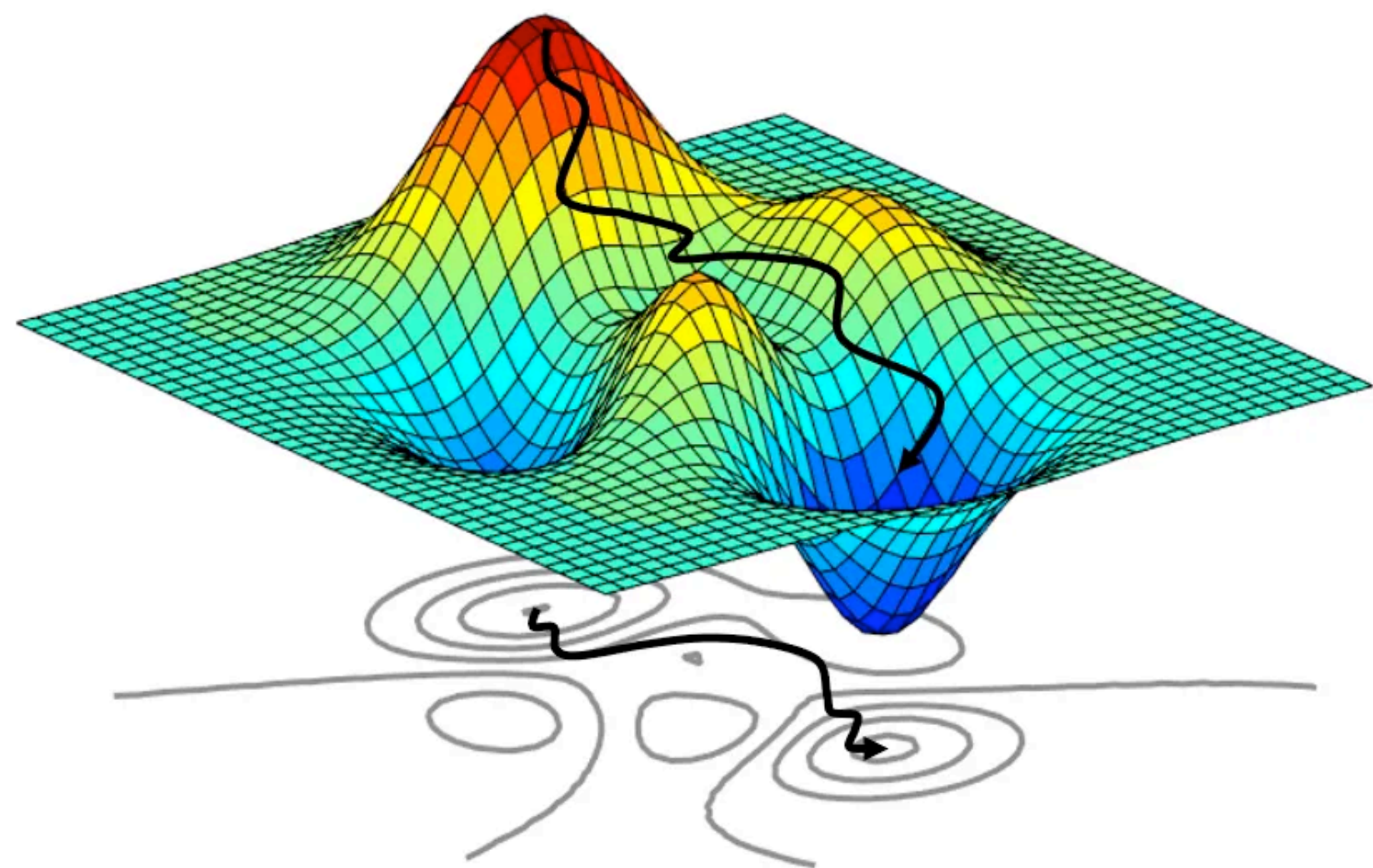


¹ https://pytorch.org/ignite/generated/ignite.handlers.param_scheduler.create_lr_scheduler_with_warmup.html

Algorithmes d'optimisation

Le monde des optimiseurs

- **Fonction de l'optimiseur** : mettre à jour les poids du modèle pour minimiser le risque empirique
- On a déjà vu SGD mais il existe d'autres algorithmes



Algorithms	
Adadelta	Implements Adadelta algorithm.
Adafactor	Implements Adafactor algorithm.
Adagrad	Implements Adagrad algorithm.
Adam	Implements Adam algorithm.
AdamW	Implements AdamW algorithm.
SparseAdam	SparseAdam implements a masked version of the Adam algorithm suitable for sparse gradients.
Adamax	Implements Adamax algorithm (a variant of Adam based on infinity norm).
ASGD	Implements Averaged Stochastic Gradient Descent.
LBFGS	Implements L-BFGS algorithm.
NAdam	Implements NAdam algorithm.
RAdam	Implements RAdam algorithm.
RMSprop	Implements RMSprop algorithm.
Rprop	Implements the resilient backpropagation algorithm.
SGD	Implements stochastic gradient descent (optionally with momentum).

Optimiseurs

Notion de momentum

- La descente de gradient met à jours les poids avec la relation suivante :

$$\theta_{t+1} = \theta_t - \eta g_t$$

- Le terme **momentum** vient de la mécanique (moment d'inertie)
- La nouvelle mise à jour des poids s'écrit :

$$v_t = \beta v_{t-1} + g_t$$

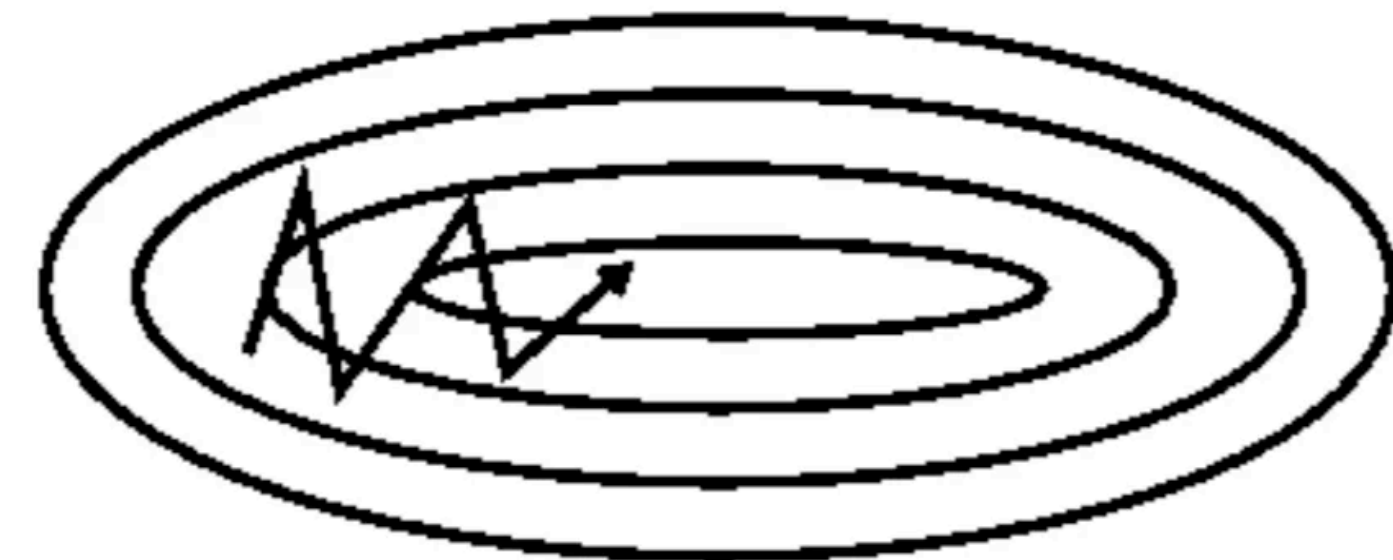
$$\theta_t = \theta_{t-1} - \eta v_t$$

Exponential weighted
moving average

- v = **momentum**, $\beta < 1$ = pondération
- La descente pointe désormais dans la direction d'une **moyenne des gradients passés**
- Ce terme permet limiter la sensibilité aux mauvaises mises à jour



(a) SGD without momentum



(b) SGD with momentum

Optimiseurs

AdaGrad

- SGD applique un ***learning rate constant*** à tous les paramètres
- **Problème** : certains paramètres associés à des données plus rares sont mises à jour moins souvent
- **Objectif** : adapter la formule de mise à jour des paramètres pour qu'elle soit adaptive
- **Adaptive Gradient (AdaGrad)** : ajuster le learning rate pour chaque paramètres en fonction de la norme du gradient

Exemple:

Si on veut entraîner un modèle de langage, certains mots sont moins fréquents que d'autres. Pour obtenir une bonne accuracy, il faudra modéliser correctement tous les mots. Adapter le gradient permet d'éviter que seuls les mots fréquents guident l'apprentissage.

Optimiseurs

AdaGrad

- **Principe** : on souhaite **ajuster le learning rate pour chaque paramètre**, de façon inversement proportionnelle au gradient
- **Motivation** : si des paramètres reçoivent des gradients élevés, on diminue le learning rate pour que ceux recevant de faibles gradient contribuent aussi à l'optimisation.
- Rappel de l'équation de mise à jour : $\theta_{t+1} = \theta_t - \eta g_t$
- Pour chaque paramètre, on accumule le carré des gradients passés

$$r_t = r_{t-1} + g_t^2$$

- La mise à jour des paramètres se fait avec la nouvelle équation suivante

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{r_t}} g_t$$

Optimiseurs

AdaGrad

- **Principe** : on souhaite **ajuster le learning rate pour chaque paramètre**, de façon inversement proportionnelle au gradient
- **Motivation** : si des paramètres reçoivent des gradients élevés, on diminue le learning rate pour que ceux recevant de faibles gradient contribuent aussi à l'optimisation.
- Rappel de l'équation de mise à jour : $\theta_{t+1} = \theta_t - \eta g_t$
- Pour chaque paramètre, on accumule le carré des gradients passés

$$r_t = r_{t-1} + g_t^2$$

- La mise à jour des paramètres se fait avec la nouvelle équation suivante

$$\theta_{t+1}^i = \theta_t^i - \frac{\eta}{\sqrt{r_t^i}} g_t^i$$

On divise le learning rate par la racine des gradients au carré accumulés

Optimiseurs

AdaGrad

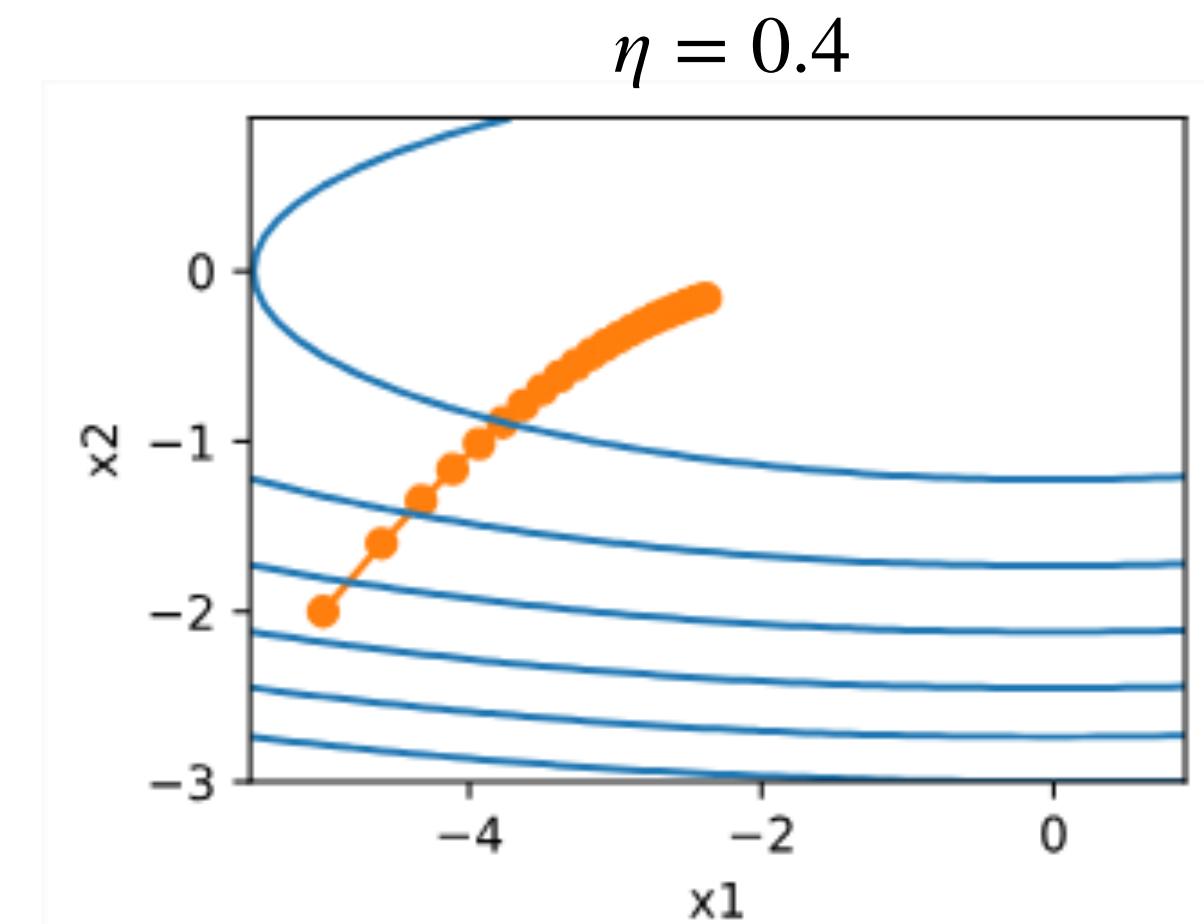
- **Principe** : on souhaite **ajuster le learning rate pour chaque paramètre**, de façon inversement proportionnelle au gradient
- **Motivation** : si des paramètres reçoivent des gradients élevés, on diminue le learning rate pour que ceux recevant de faibles gradient contribuent aussi à l'optimisation.

- Rappel de l'équation de mise à jour : $\theta_{t+1} = \theta_t - \eta g_t$
- Pour chaque paramètre, on accumule le carré des gradients passés

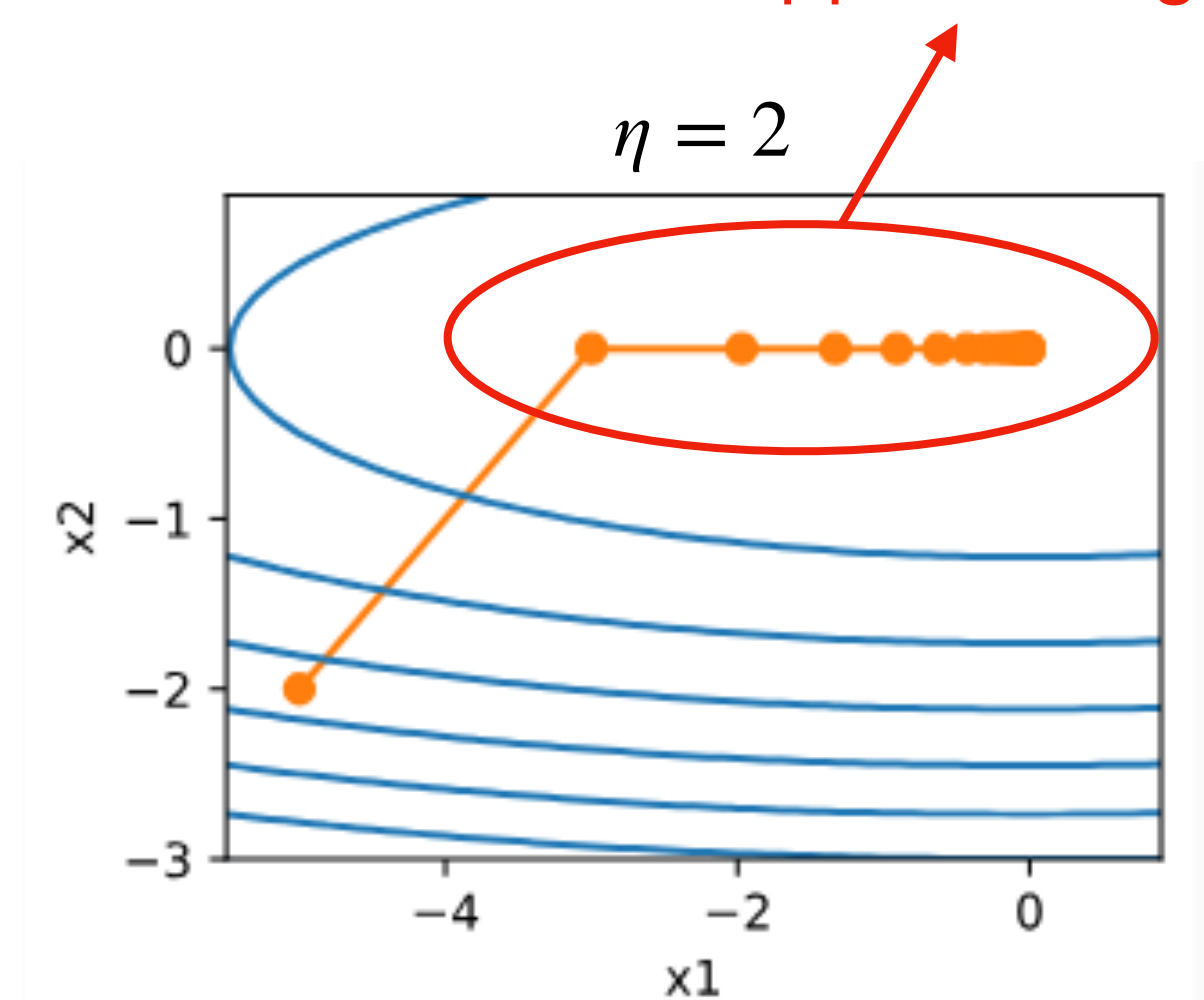
$$r_t = r_{t-1} + g_t^2$$

- La mise à jour des paramètres se fait avec la nouvelle équation suivante

$$\theta_{t+1}^i = \theta_t^i - \frac{\eta}{\sqrt{r_t^i}} g_t^i$$



Diminution progressive du learning rate au cours de l'apprentissage



Optimiseurs

RMSProp

- **Problème avec AdaGrad** : la décroissance du learning rate en fonction des itérations est prédéfinie ($t^{-\frac{1}{2}}$)
 - On peut accumuler des erreurs
 - Pas de normalisation, donc l'état $r_t = r_{t-1} + g_t^2$ augmente linéairement sans limite
- RMSProp consiste à pondérer chaque terme afin d'en contrôler l'importance dans la mise à jour des paramètres
- L'accumulation des gradients prend la forme suivante, avec $\lambda = [0,1]$ (**Note**: ça ressemble un peu au momentum) :

$$r_t = (1 - \lambda)r_{t-1} + \lambda g_t^2$$

- Le paramètre λ permet de donner plus d'importance au gradient courant (leaky average)
- La formule de mise à jour reste identique:

$$\theta_{t+1}^i = \theta_t^i - \frac{\eta}{\sqrt{r_t^i}} g_t^i$$

Optimiseurs

Adam

- Algorithme d'optimisation proposé par Kingma et Ba (2015)
- Méthode la plus utilisée pour l'optimisation de réseaux de neurones profonds
- Inspirée de RMSProp en ajoutant la notion de momentum
- A donné lieu à d'autres approches : AdaMax, AdamW...
- Usuellement : $(\beta_1, \beta_2) = (0.9, 0.999)$
- Le poids associé aux anciens gradient est plus important dans le momentum que dans l'adaptation du *learning rate*

Algorithm 1: *Adam*, our proposed algorithm for stochastic optimization. See section 2 for details, and for a slightly more efficient (but less clear) order of computation. g_t^2 indicates the elementwise square $g_t \odot g_t$. Good default settings for the tested machine learning problems are $\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\epsilon = 10^{-8}$. All operations on vectors are element-wise. With β_1^t and β_2^t we denote β_1 and β_2 to the power t .

Require: α : Stepsize

Require: $\beta_1, \beta_2 \in [0, 1)$: Exponential decay rates for the moment estimates

Require: $f(\theta)$: Stochastic objective function with parameters θ

Require: θ_0 : Initial parameter vector

$m_0 \leftarrow 0$ (Initialize 1st moment vector)

$v_0 \leftarrow 0$ (Initialize 2nd moment vector)

$t \leftarrow 0$ (Initialize timestep)

while θ_t not converged **do**

$t \leftarrow t + 1$

$g_t \leftarrow \nabla_{\theta} f_t(\theta_{t-1})$ (Get gradients w.r.t. stochastic objective at timestep t)

$m_t \leftarrow \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$ (Update biased first moment estimate)

$v_t \leftarrow \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$ (Update biased second raw moment estimate)

$\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$ (Compute bias-corrected first moment estimate)

$\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$ (Compute bias-corrected second raw moment estimate)

$\theta_t \leftarrow \theta_{t-1} - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$ (Update parameters)

end while

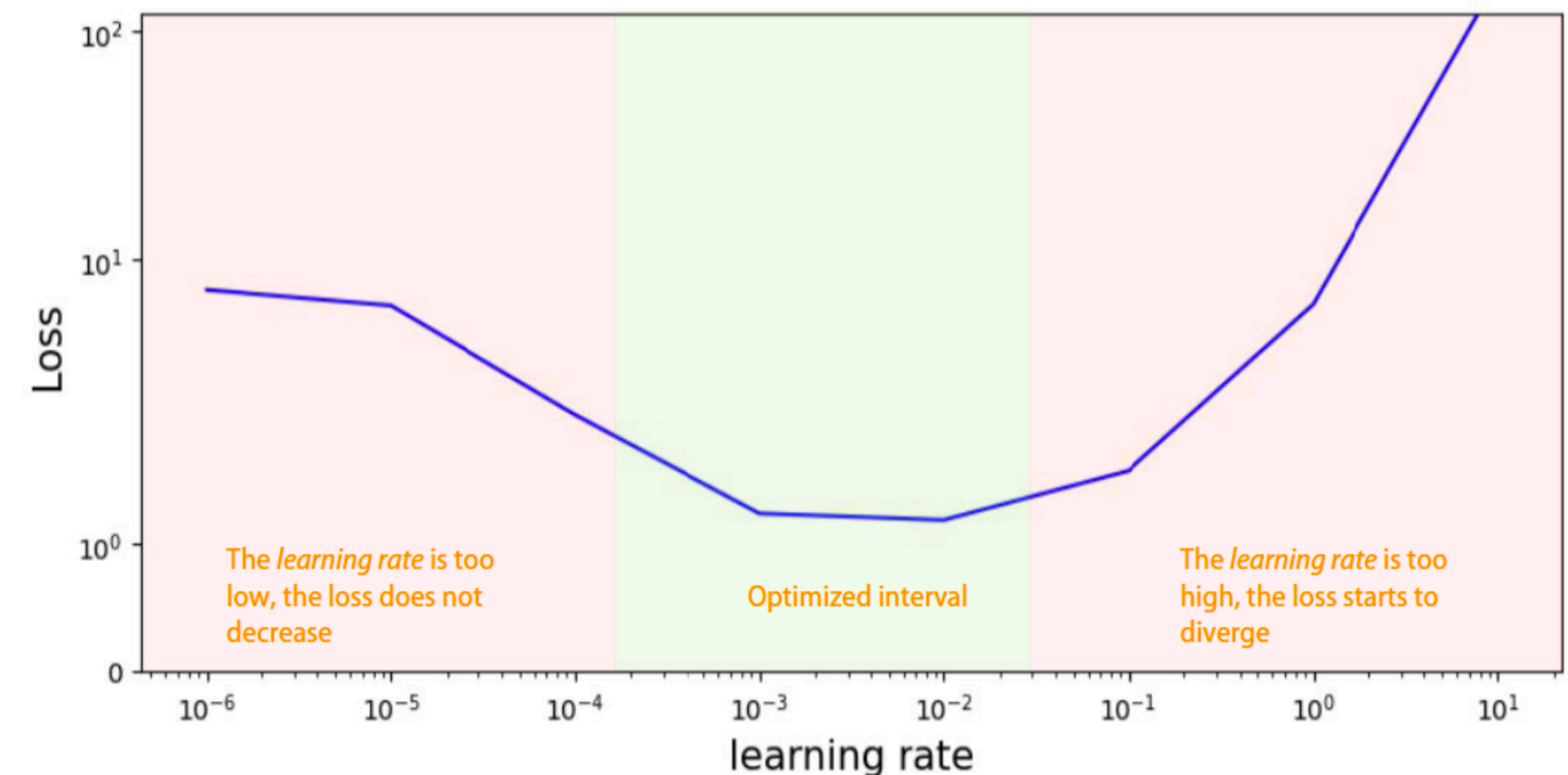
return θ_t (Resulting parameters)

<https://arxiv.org/pdf/1412.6980.pdf>

Algorithmes d'optimisation

Note sur le choix du *learning rate*

- Le choix du learning rate est **primordial**
- Quels que soient les schedulers et optimiseurs utilisés, il faut choisir la bonne initialisation
- Bonne pratique :
 - Tester plusieurs valeurs sur des entraînements courts (sous ensemble de train, une epoch...)
 - Tracer la loss en fonction du learning rate après ce court apprentissage
- Malheureusement, il y a moult hyperparamètres à optimiser 😭



Optimisation de l'apprentissage

Sommaire

1. Algorithmes d'optimisation

1. Importance du learning rate
2. Schedulers
3. Warm-up
4. Optimiseurs

2. Amélioration de l'espace de recherche

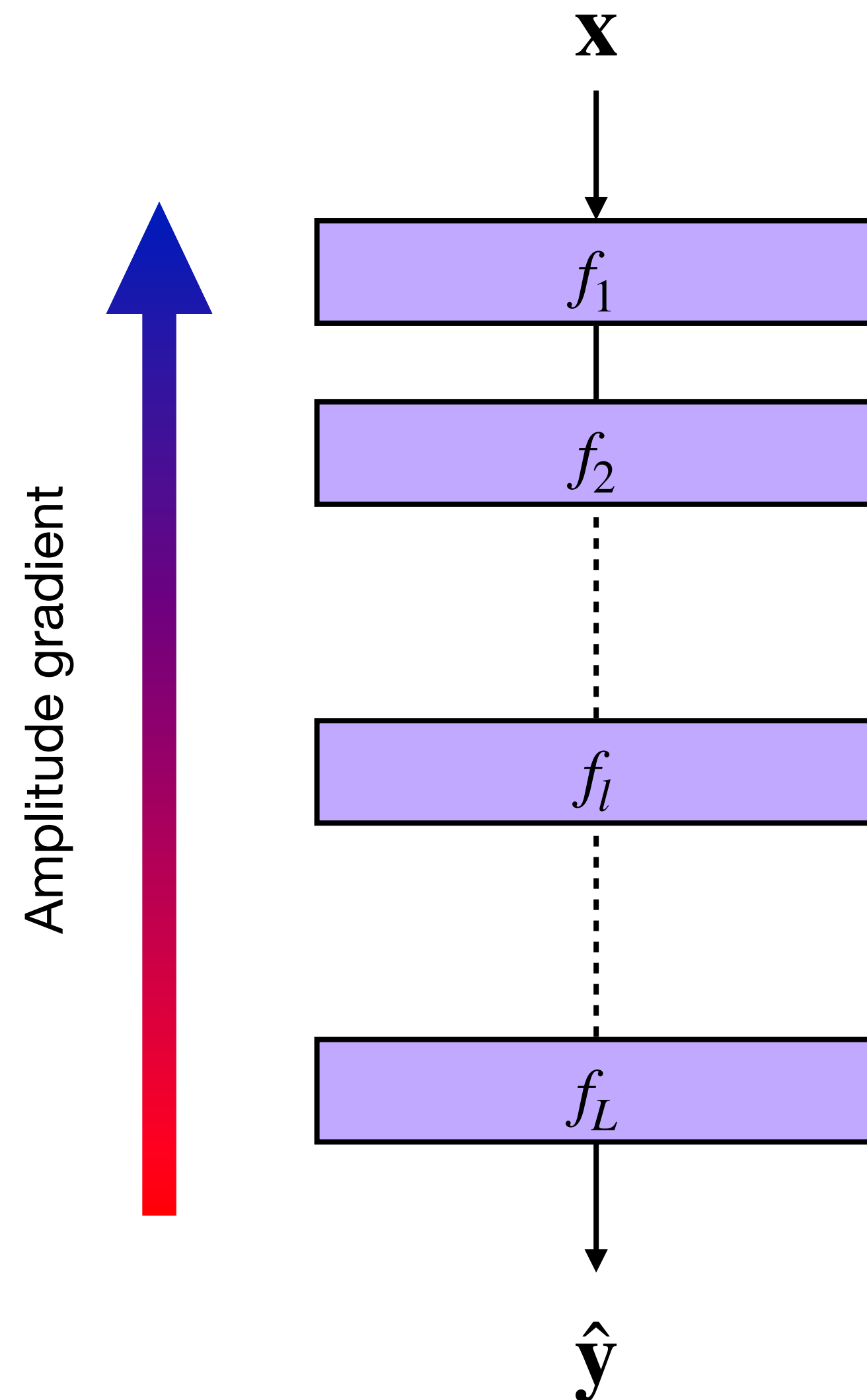
1. Connections résiduelles
2. Largeur des couches
3. Flux d'information dans les ResNet

3. Stabilisation de l'apprentissage

1. Régler la taille de batch
2. Accumuler les gradients
3. Gradient clipping

Amélioration de l'espace de recherche

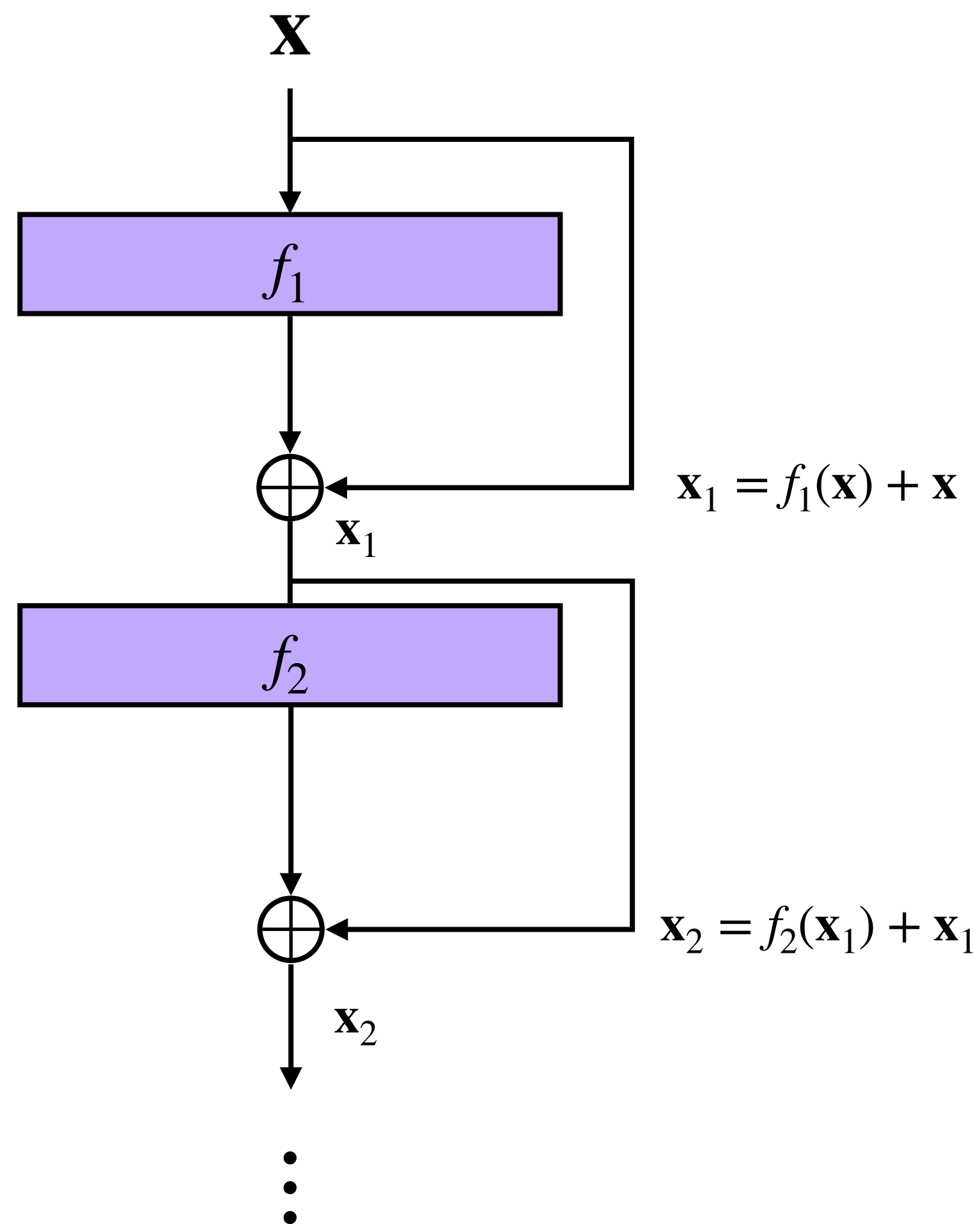
Rappels sur les réseaux de neurones profonds



- Augmenter la profondeur du réseau = améliorer les performances
- Attention : phénomène de *dégradation*
 - Trop de couches mène à la disparition du gradient (*vanishing gradient problem*)
 - Le gradient s'atténue de couche en couche
 - Les premières couches sont alors peu modifiées
 - L'apprentissage devient très long voire n'évolue plus du tout
- **Solution ?**
 - Connections résiduelles (ou *skip connexions*)
 - Raccourcis pour éviter que le gradient ne devienne trop petit

Amélioration de l'espace de recherche

Connections résiduelles (ResNet)

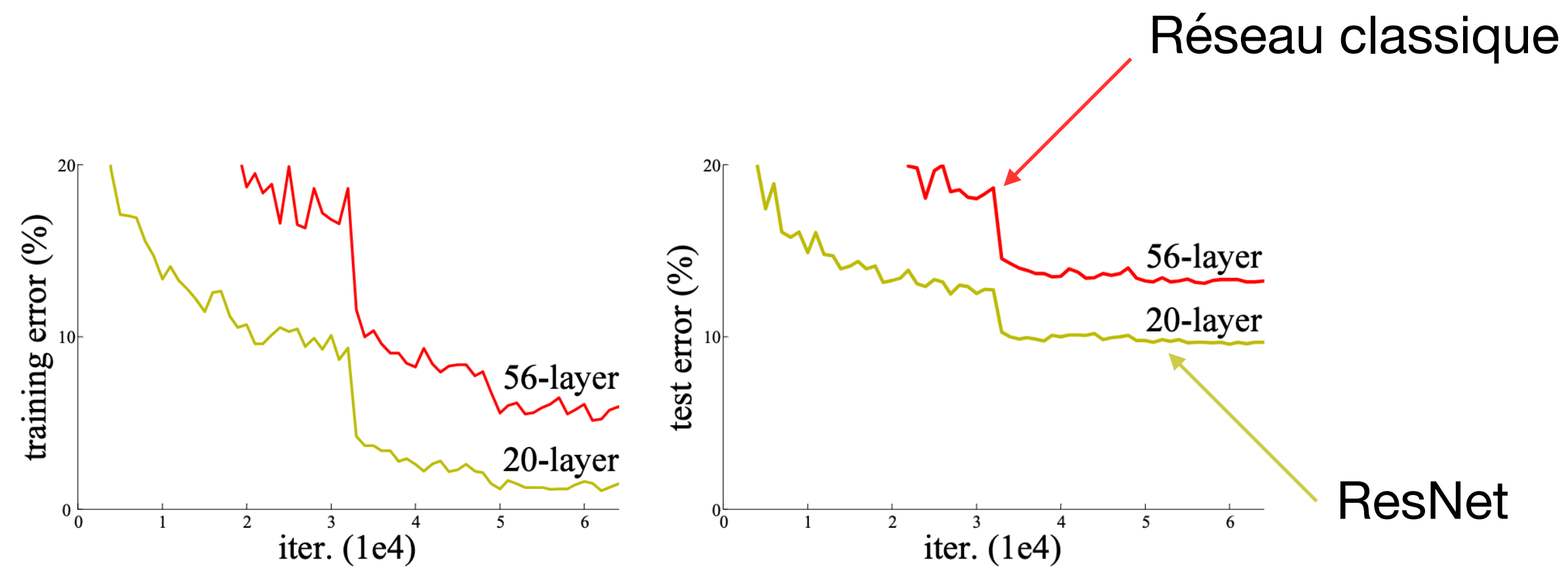


- **Idée** : ajouter à la sortie d'une couche son entrée
- Possibilité de f_i soit un sous réseau (plusieurs couches)
- Plusieurs avantages :
 - Réseaux moins profonds et plus faciles à apprendre
 - Meilleures performances car le gradient ne disparaît plus
 - Espace de recherche simplifié
- Les réseaux résiduels sont couramment appelés **ResNet**

Amélioration de l'espace de recherche

Connections résiduelles : exemples

Meilleures performances avec moins de couches
(Kaiming He et al. 2016)

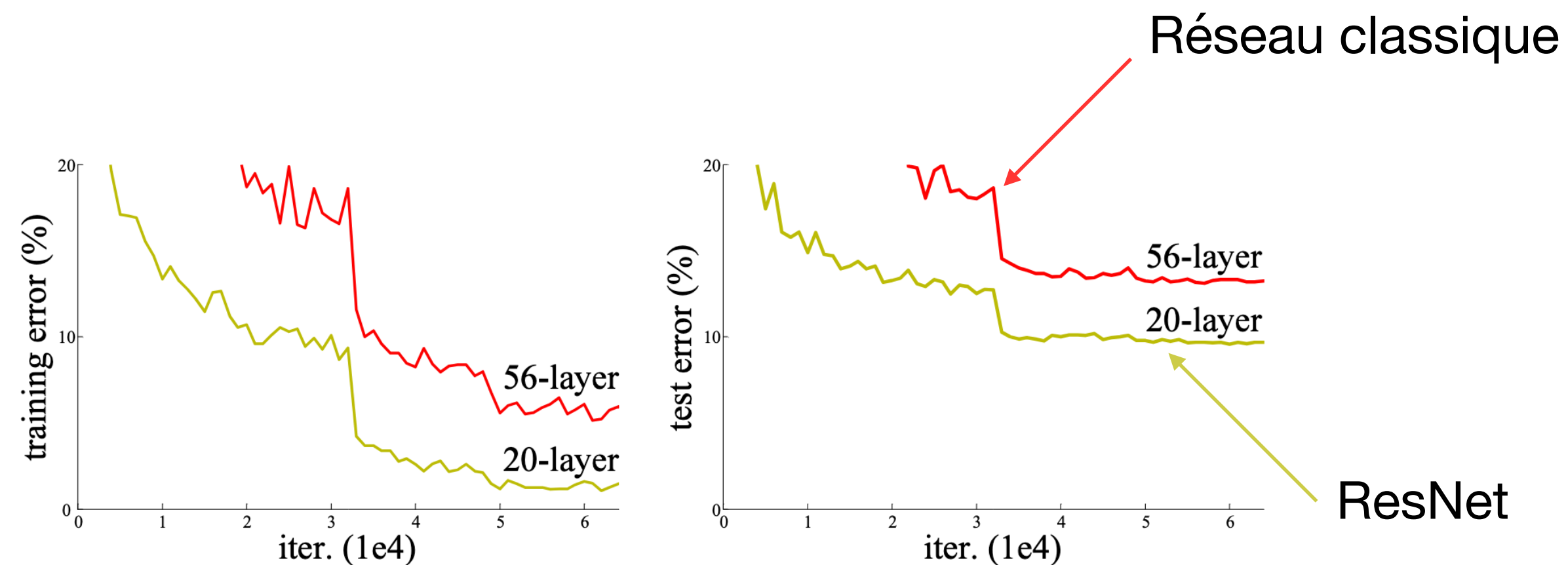


https://openaccess.thecvf.com/content_cvpr_2016/papers/He_Deep_Residual_Learning_CVPR_2016_paper.pdf

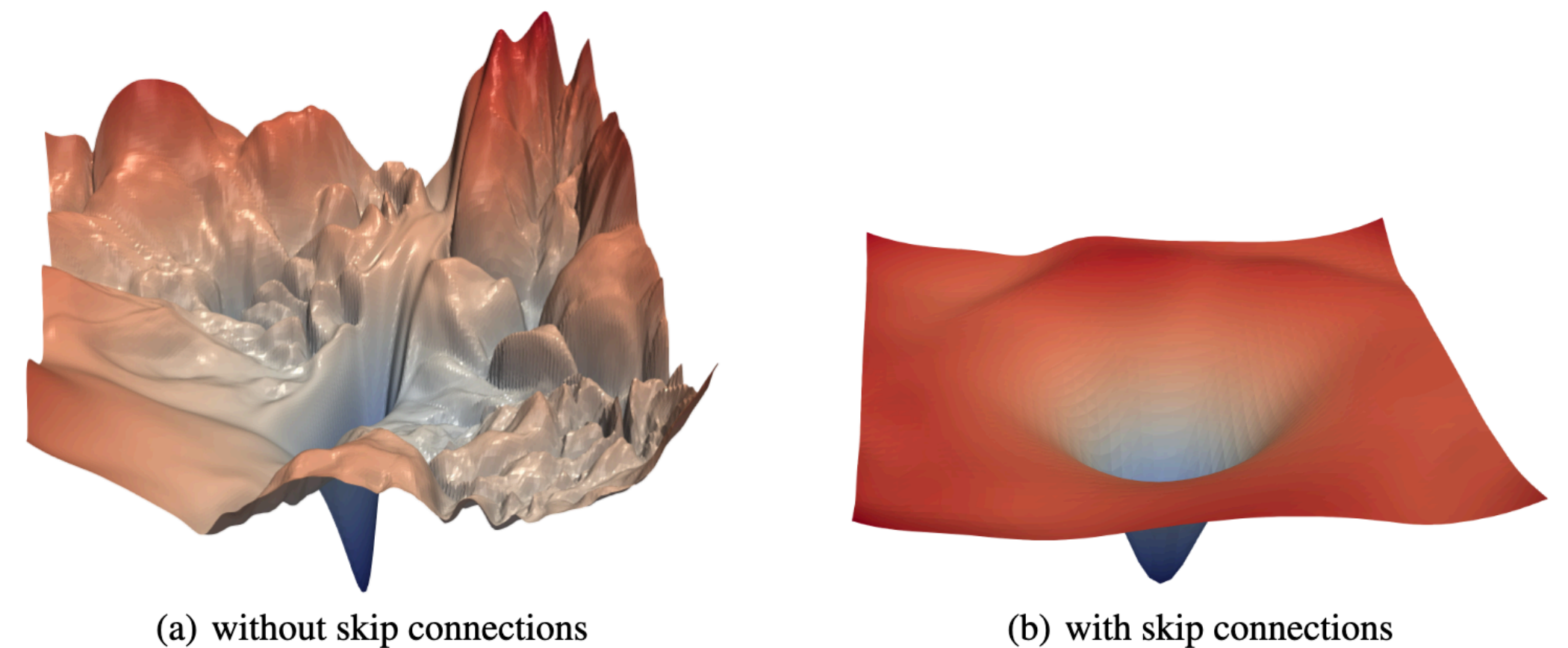
Amélioration de l'espace de recherche

Connections résiduelles : exemples

Meilleures performances avec moins de couches
(Kaiming He et al. 2016)



Espace de recherche simplifié
(Yao Li et al. 2018)



https://openaccess.thecvf.com/content_cvpr_2016/papers/He_Deep_Residual_Learning_CVPR_2016_paper.pdf

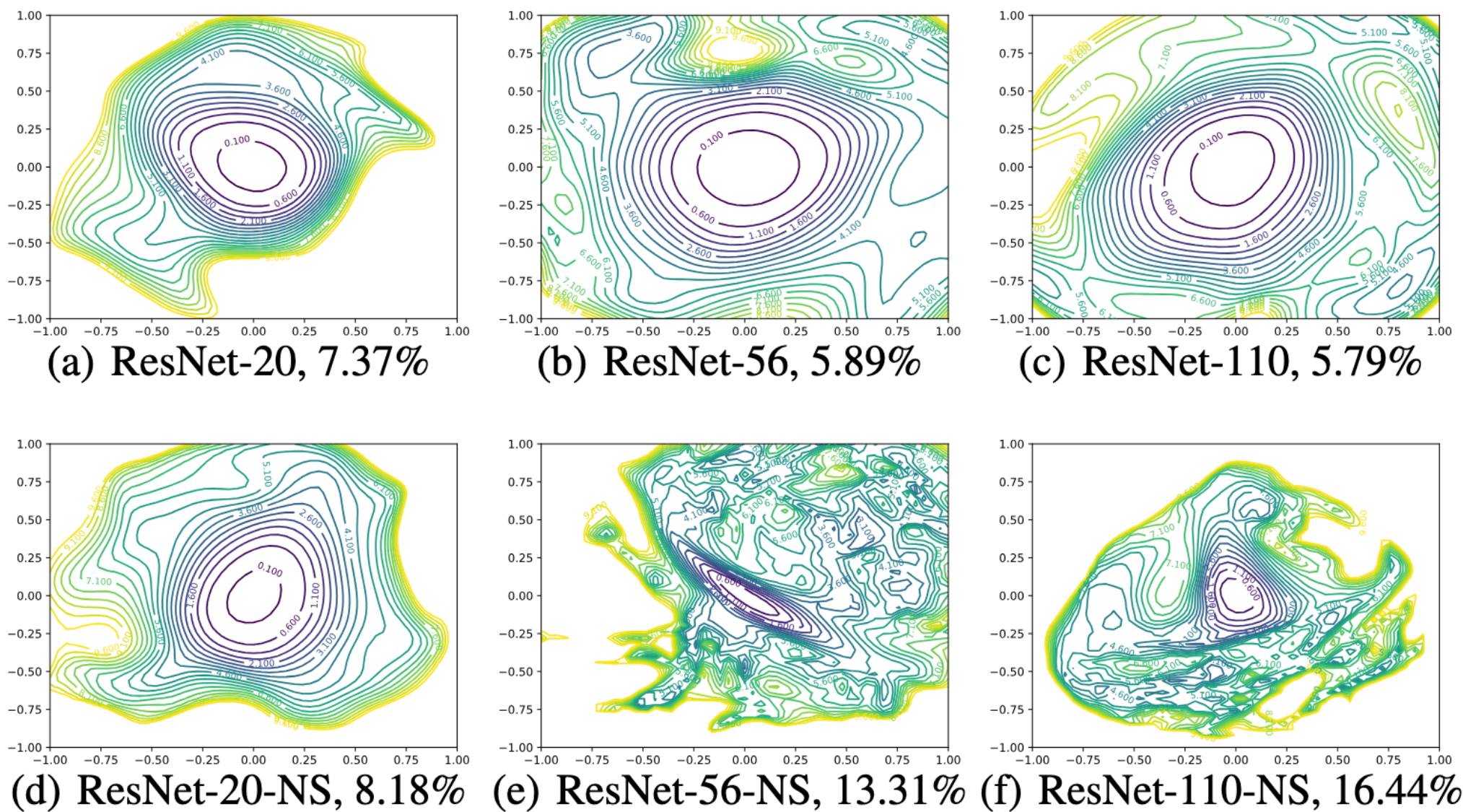
https://proceedings.neurips.cc/paper_files/paper/2018/file/a41b3bb3e6b050b6c9067c67f663b915-Paper.pdf

Amélioration de l'espace de recherche

ResNet et Élargissement des couches (Li et al. 2018)

Espace de recherche avec et
sans skip connexion

Profondeur du réseau →

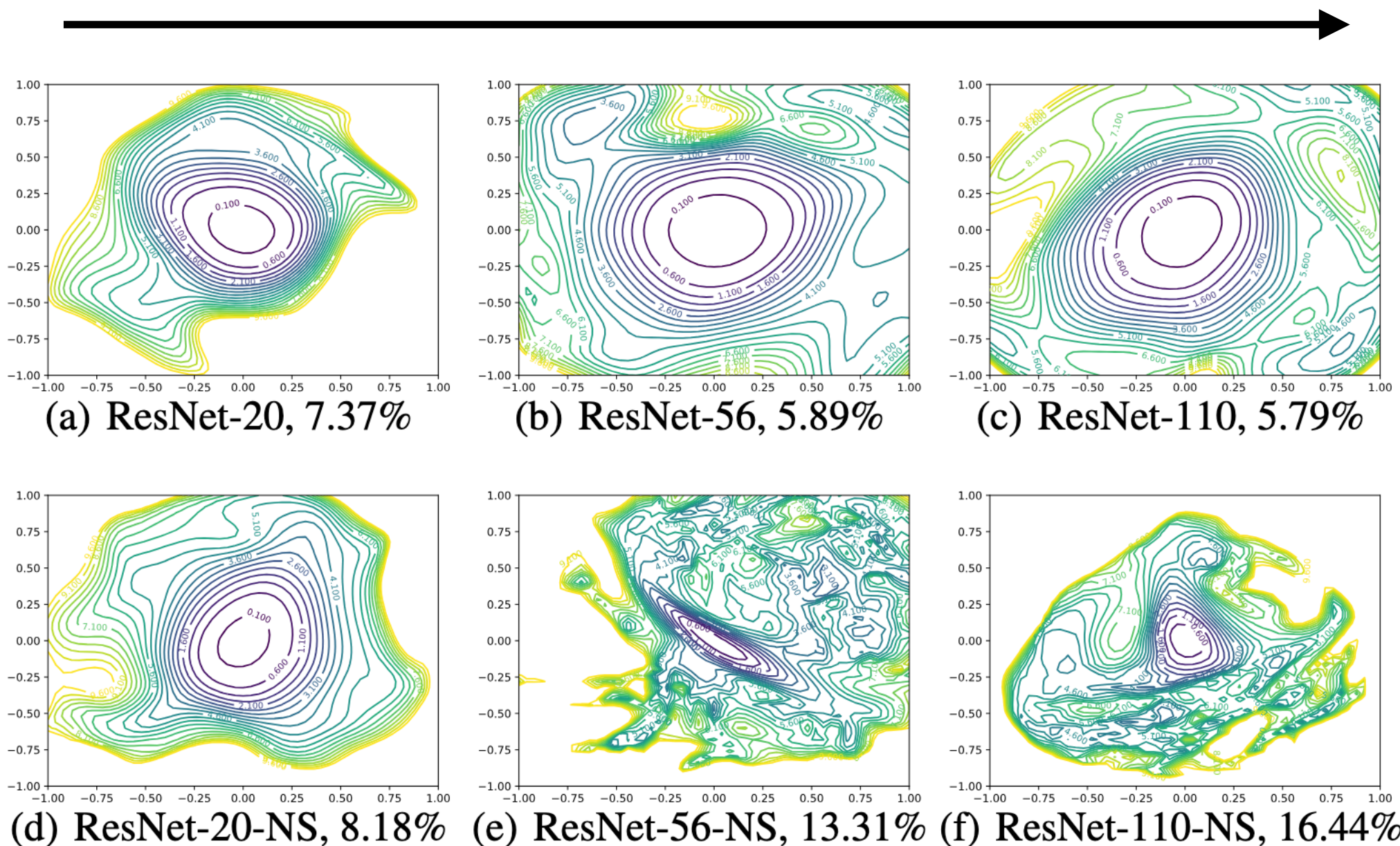


Amélioration de l'espace de recherche

ResNet et Élargissement des couches (Li et al. 2018)

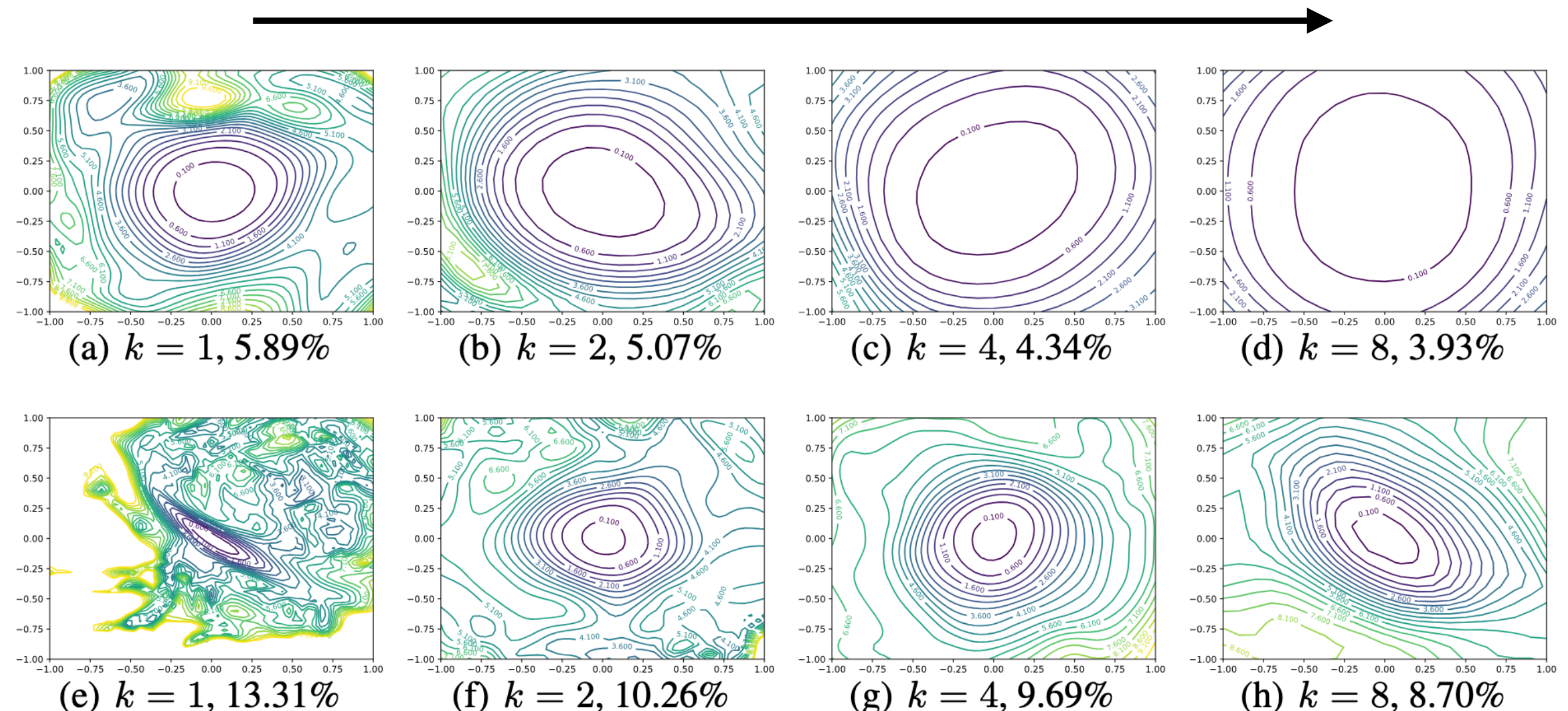
Espace de recherche avec et sans skip connexion

Profondeur du réseau



Espace de recherche en élargissant les couches

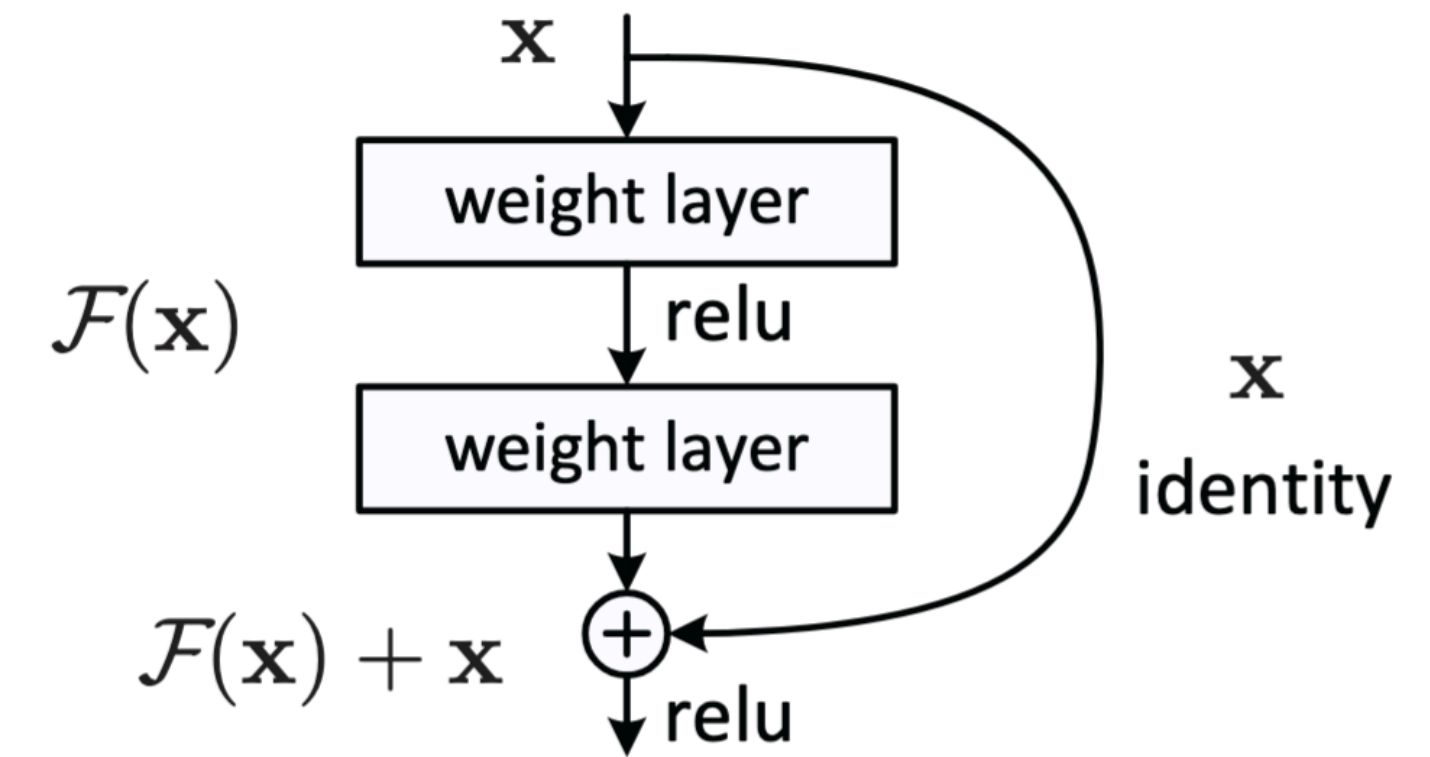
Largeur de couche (⚠)



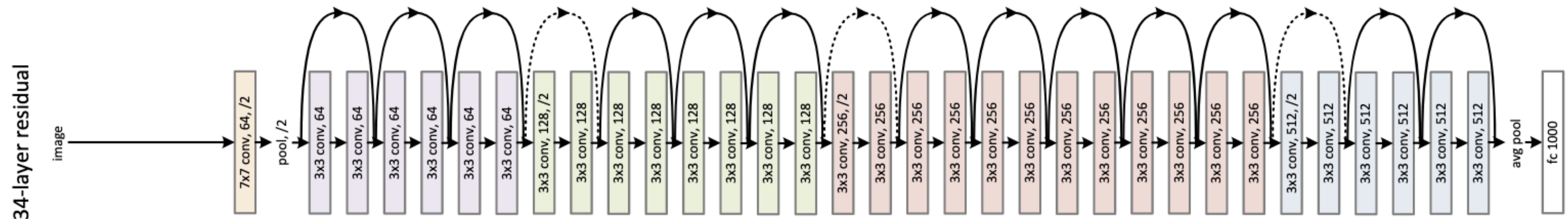
Flux d'information dans un ResNet

Complexité de l'information au sein d'un modèle de vision (Fel et al., 2024)

- Analyse réalisée dans un ResNet-50 pour la classification d'images
- ResNet-50 : modèle de classification très connu, basé sur une succession de couches résiduelles
- Objectif de l'article : analyser la complexité de l'information au sein du réseau afin de comprendre la façon dont les images sont représentées



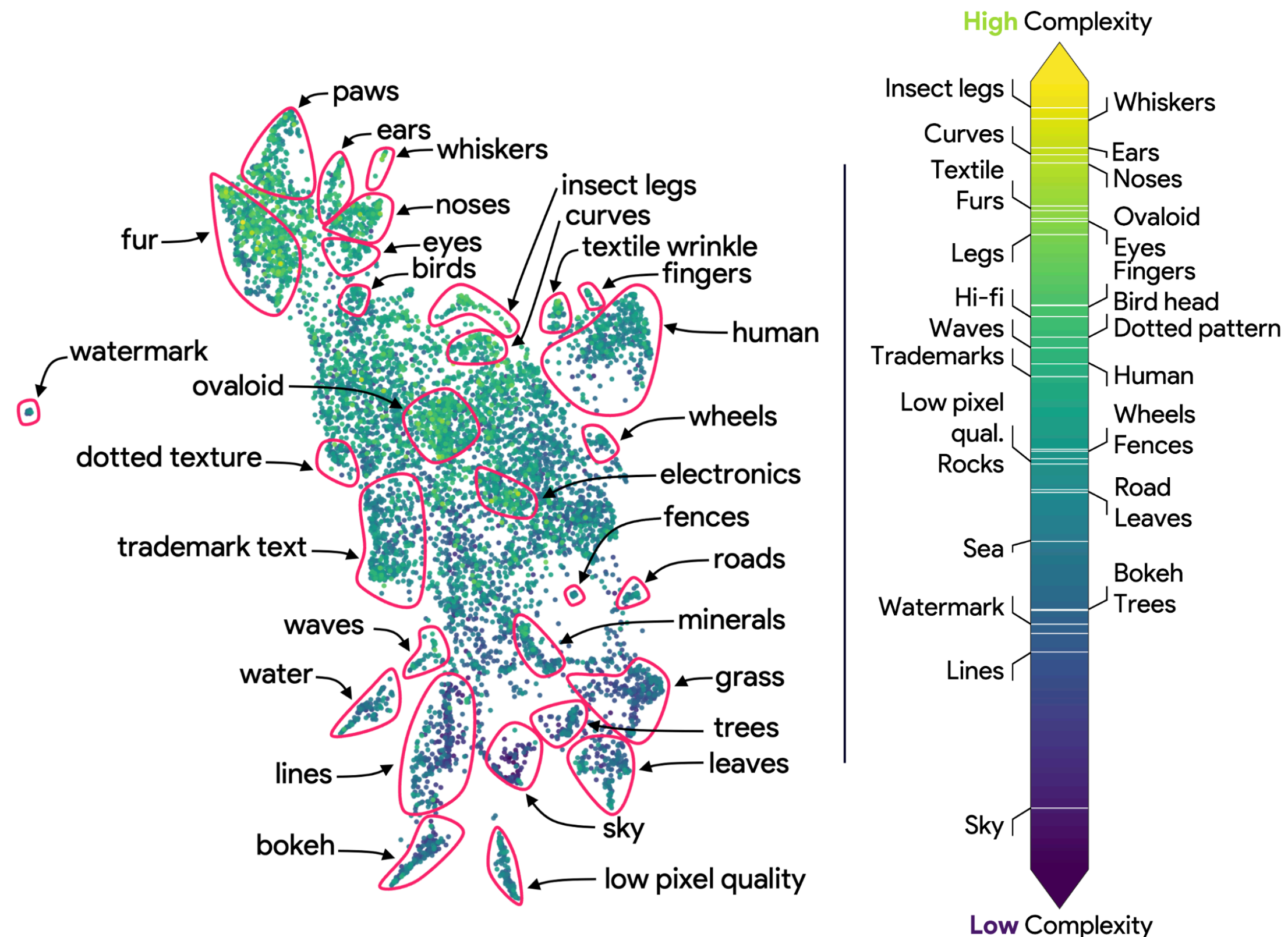
Bloc résiduel



Exemple d'un ResNet à 34 couches

Flux d'information dans un ResNet

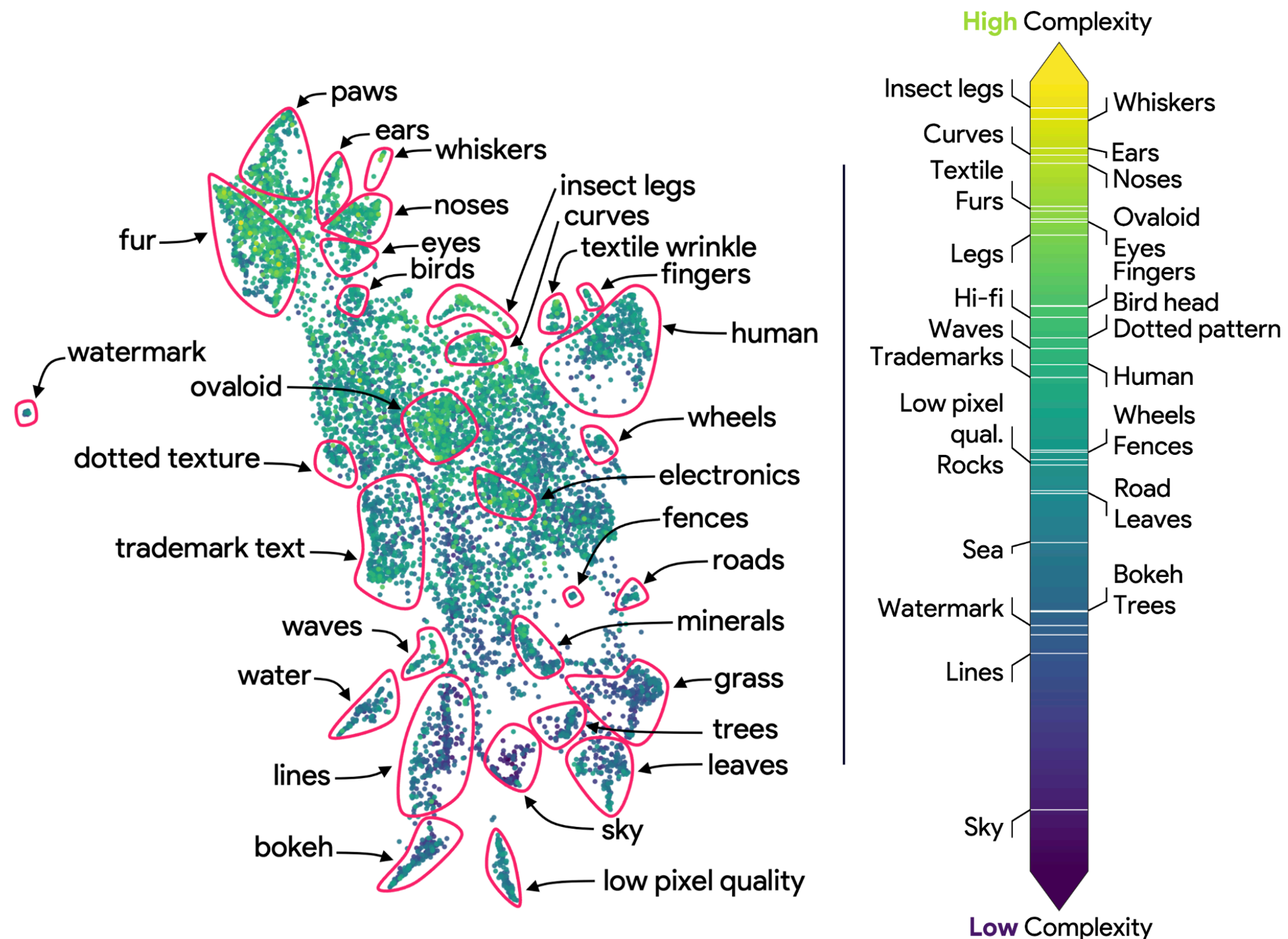
Complexité de l'information au sein d'un modèle de vision (Fel et al., 2024)



- Différents types de concepts sont représentés dans le modèle
- Concept plus « difficiles » = complexité plus élevée

Flux d'information dans un ResNet

Complexité de l'information au sein d'un modèle de vision (Fel et al., 2024)

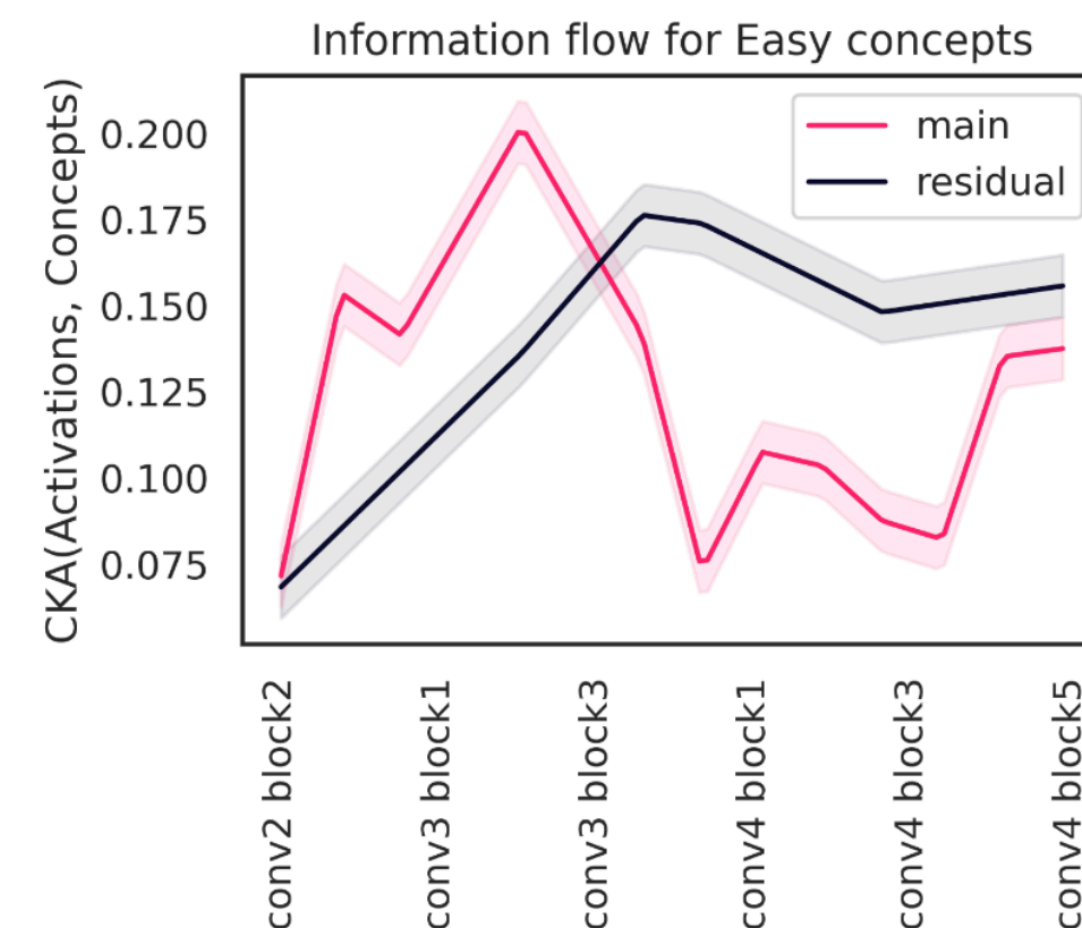
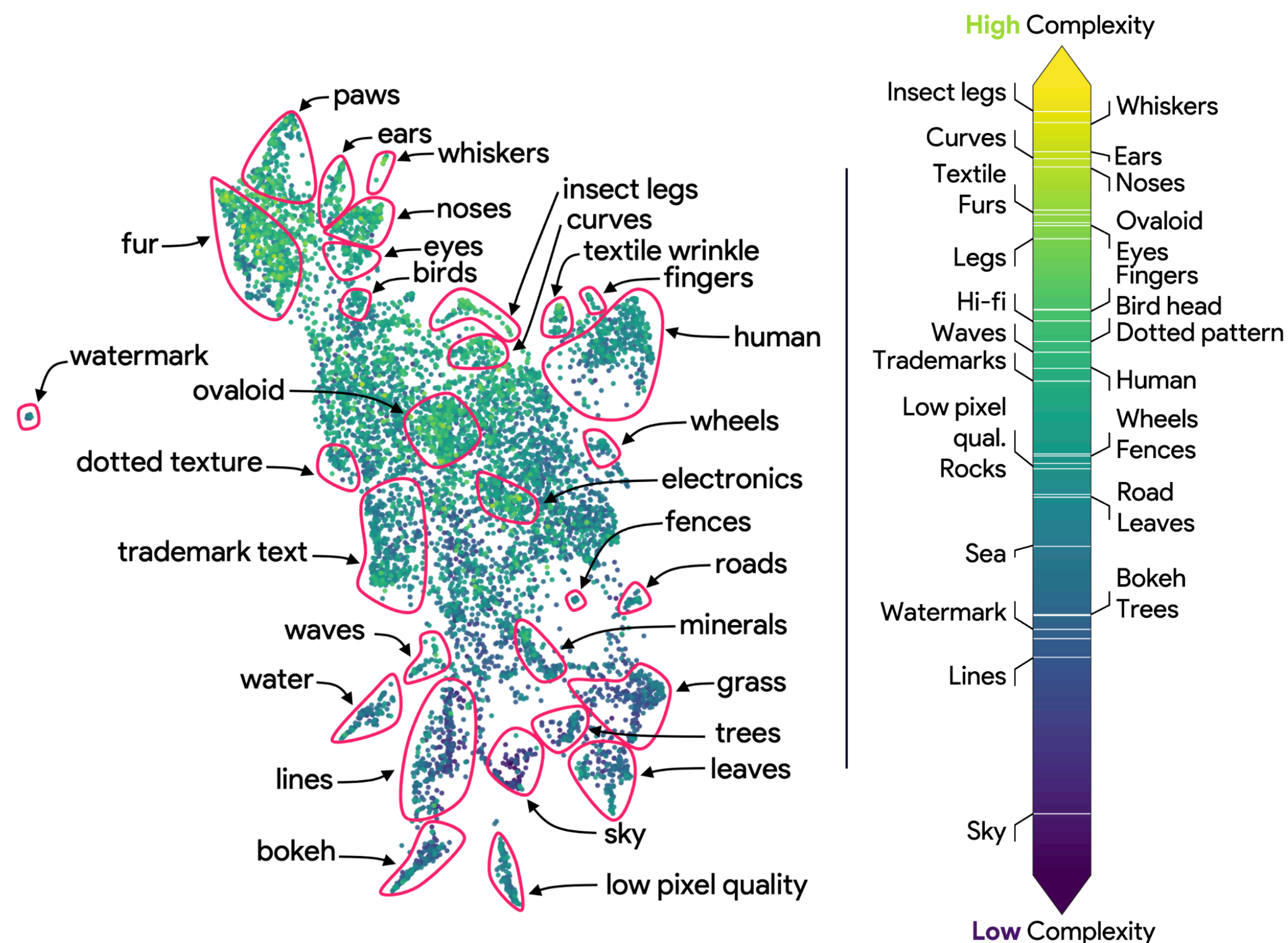


Et là, résultat de ouf !

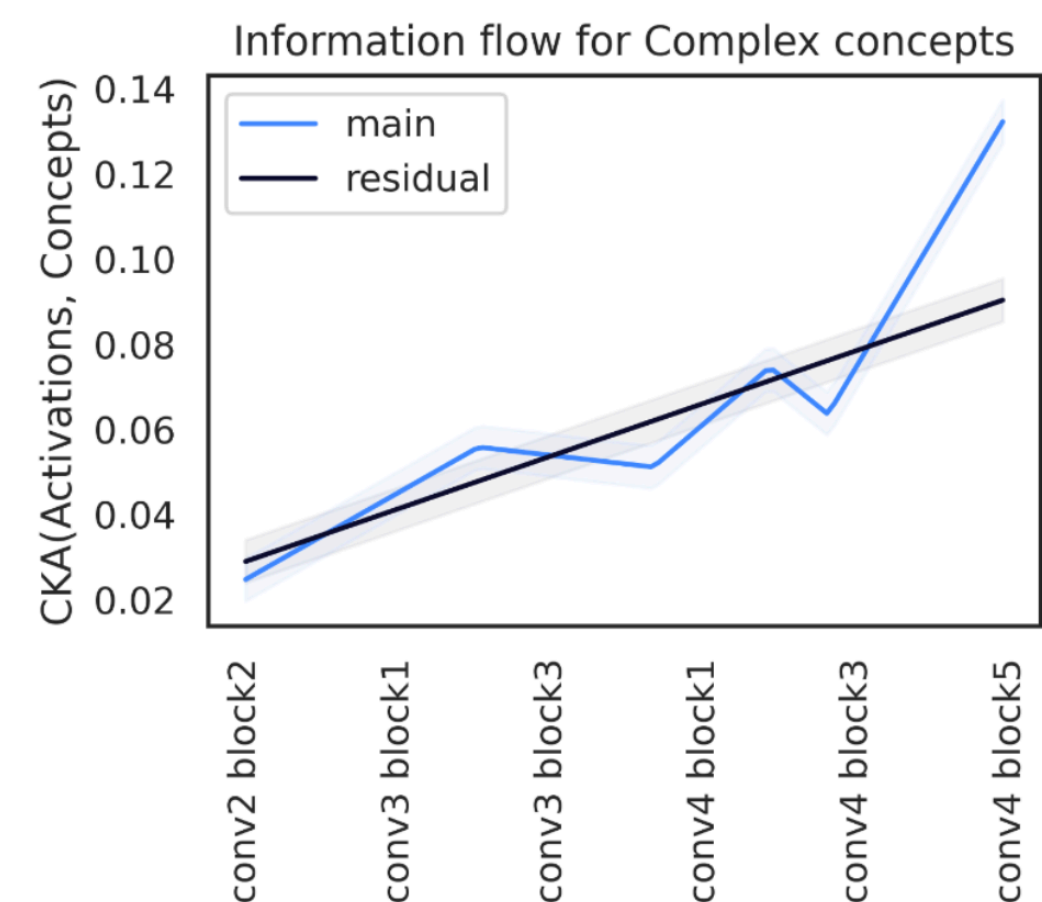
- Différents types de concepts sont représentés dans le modèle
- Concept plus « difficiles » = complexité plus élevée

Flux d'information dans un ResNet

Complexité de l'information au sein d'un modèle de vision (Fel et al., 2024)



Les couches résiduelles transportent les concepts faciles vers la sortie du modèle !

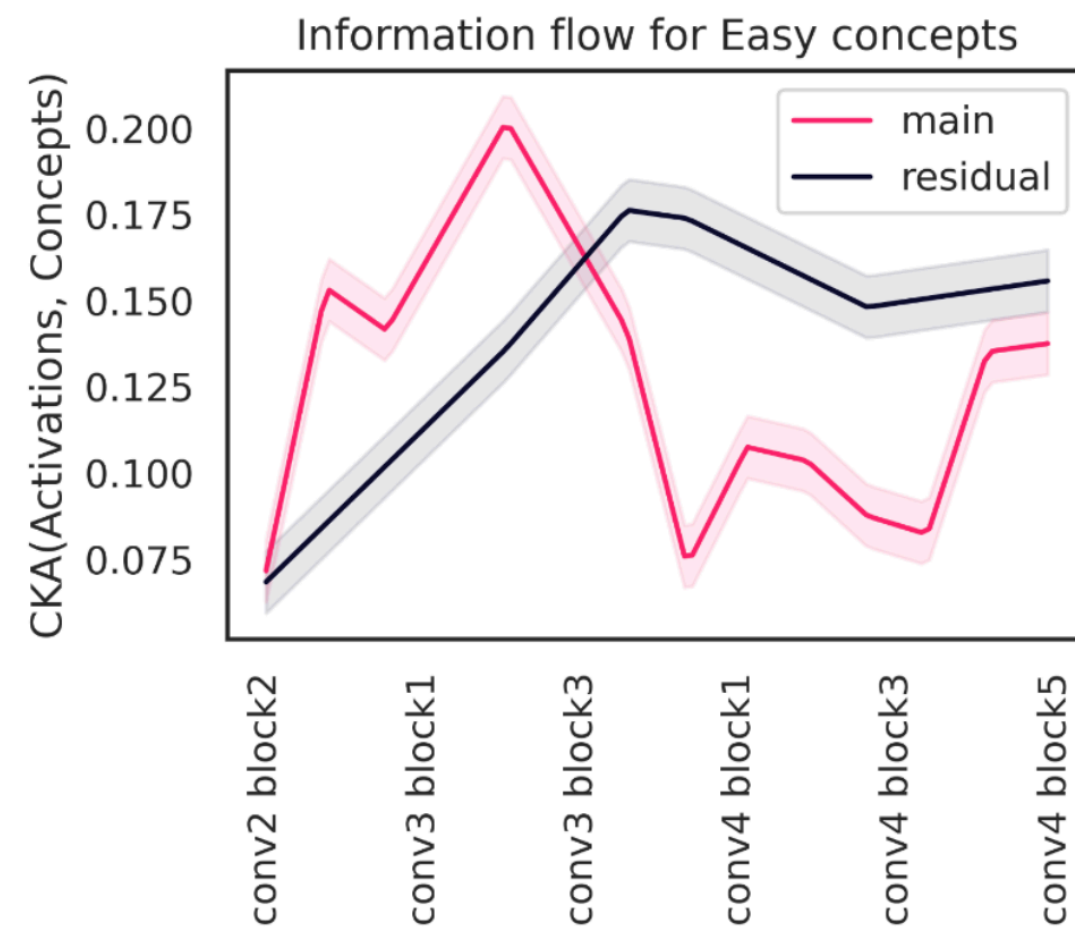


Les concepts difficiles sont représentés dans les couches profondes du réseau

- Différents types de concepts sont représentés dans le modèle
- Concept plus « difficiles » = complexité plus élevée

Flux d'information dans un ResNet

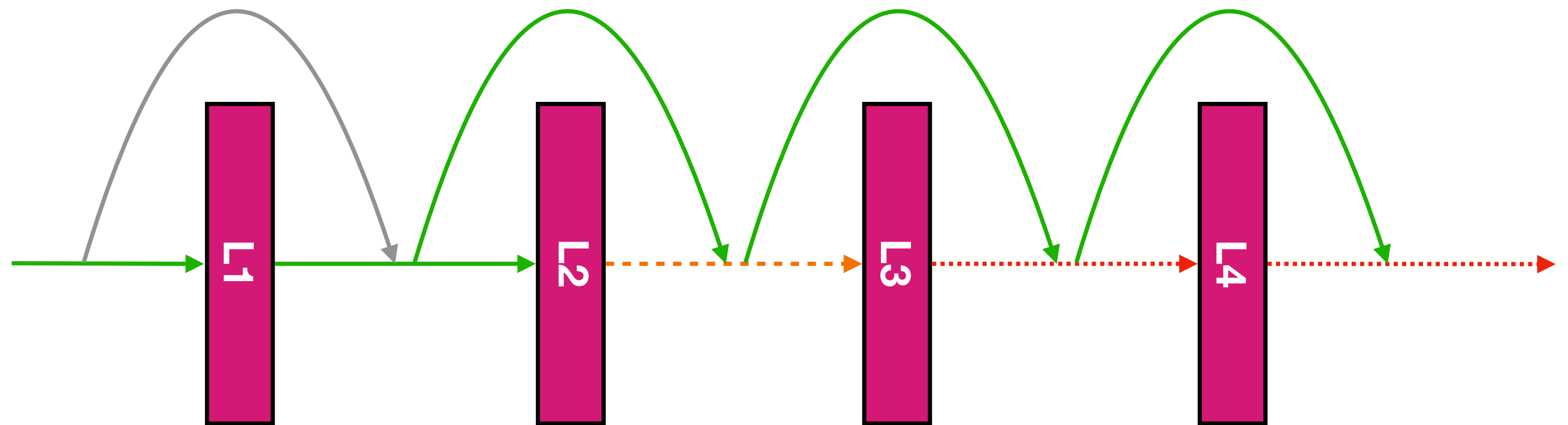
Complexité de l'information au sein d'un modèle de vision (Fel et al., 2024)



Les couches résiduelles transportent les concepts faciles vers la sortie du modèle !

→ Concepts faciles, faible complexité
→ Concepts difficiles, complexité élevée

- Dans la branche principale, il y a un gradient de complexité de l'information
- Les couches résiduelles permettent de transmettre les concepts simples vers la sortie
- Les connexions résiduelles réduisent la perte d'information, ce qui peut expliquer le fait que le gradient ne disparaît plus



Optimisation de l'apprentissage

Sommaire

1. Algorithmes d'optimisation

1. Importance du learning rate
2. Schedulers
3. Warm-up
4. Optimiseurs

2. Amélioration de l'espace de recherche

1. Connexions résiduelles
2. Largeur des couches

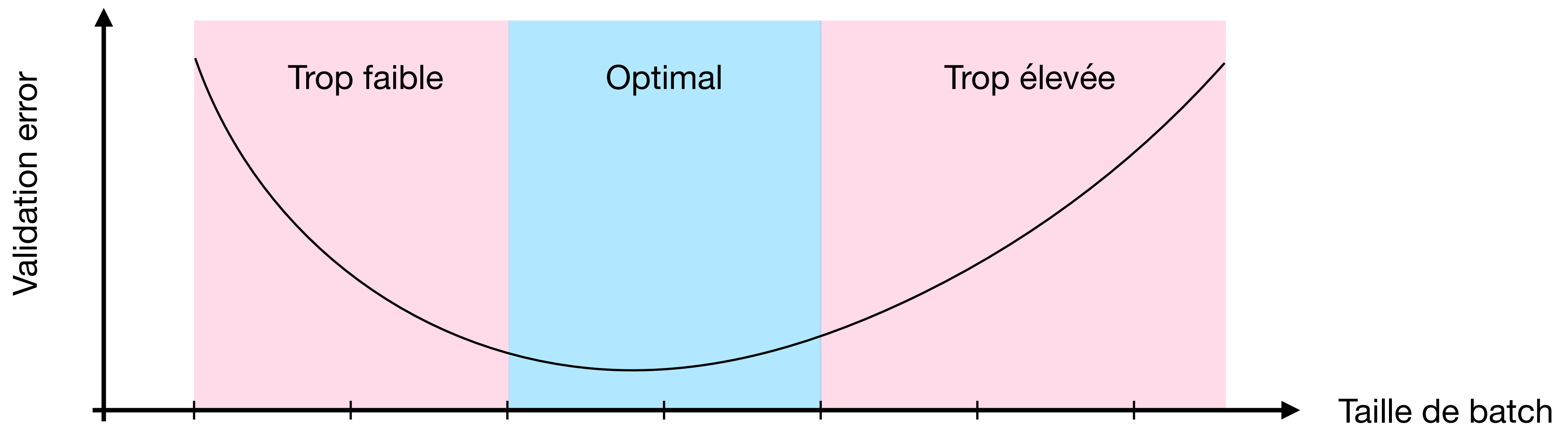
3. Stabilisation de l'apprentissage

1. Régler la taille de batch
2. Accumuler les gradients
3. Gradient clipping

Stabilisation de l'apprentissage

Régler la taille de batch

- Le deep learning bénéficie des GPUs pour paralléliser les calculs
- La rétro-propagation peut être vectorisée pour accélérer le calcul
- Intuition : « si j'augmente au max la taille de batch, j'accélère mes calculs ! » (smart)



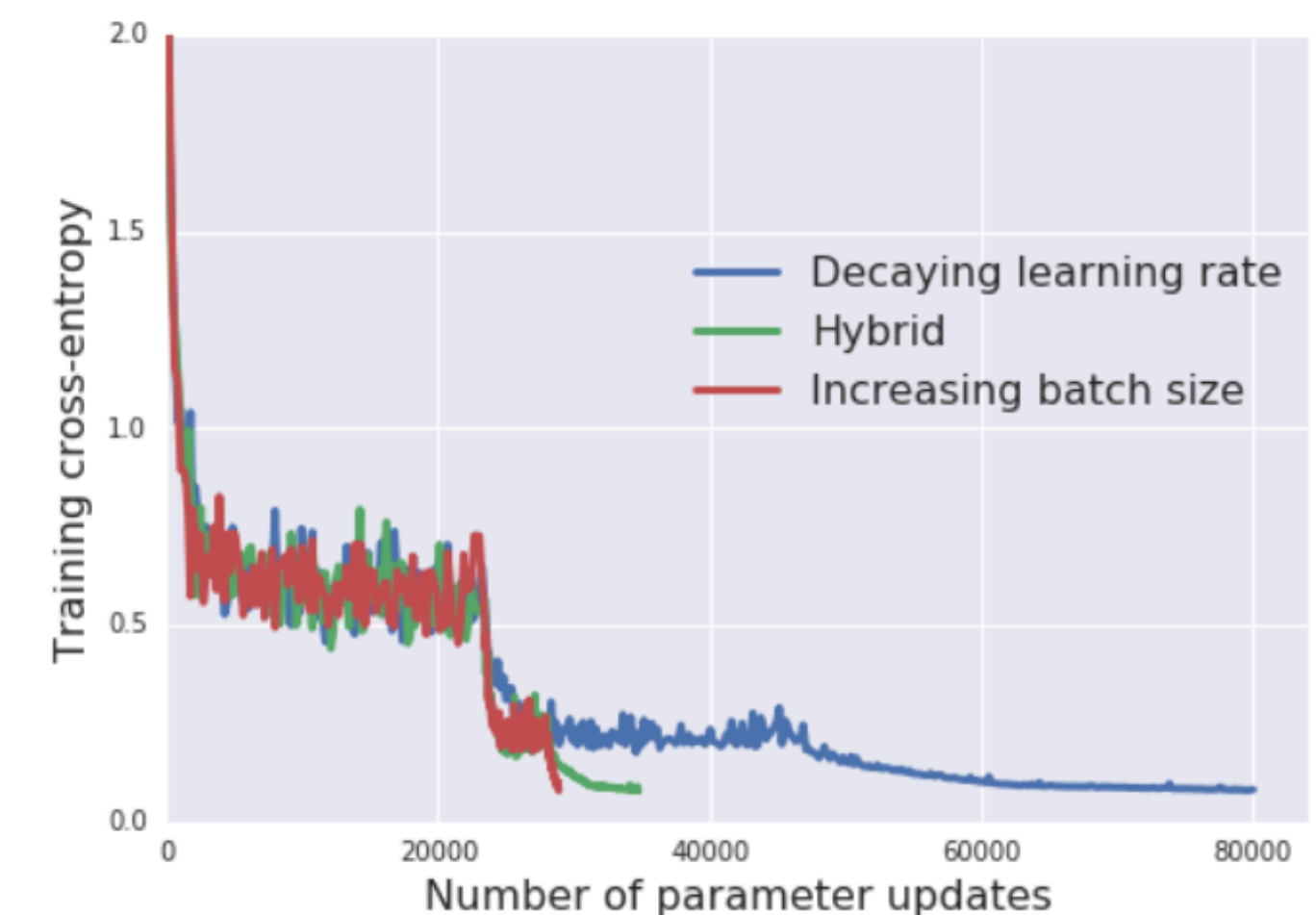
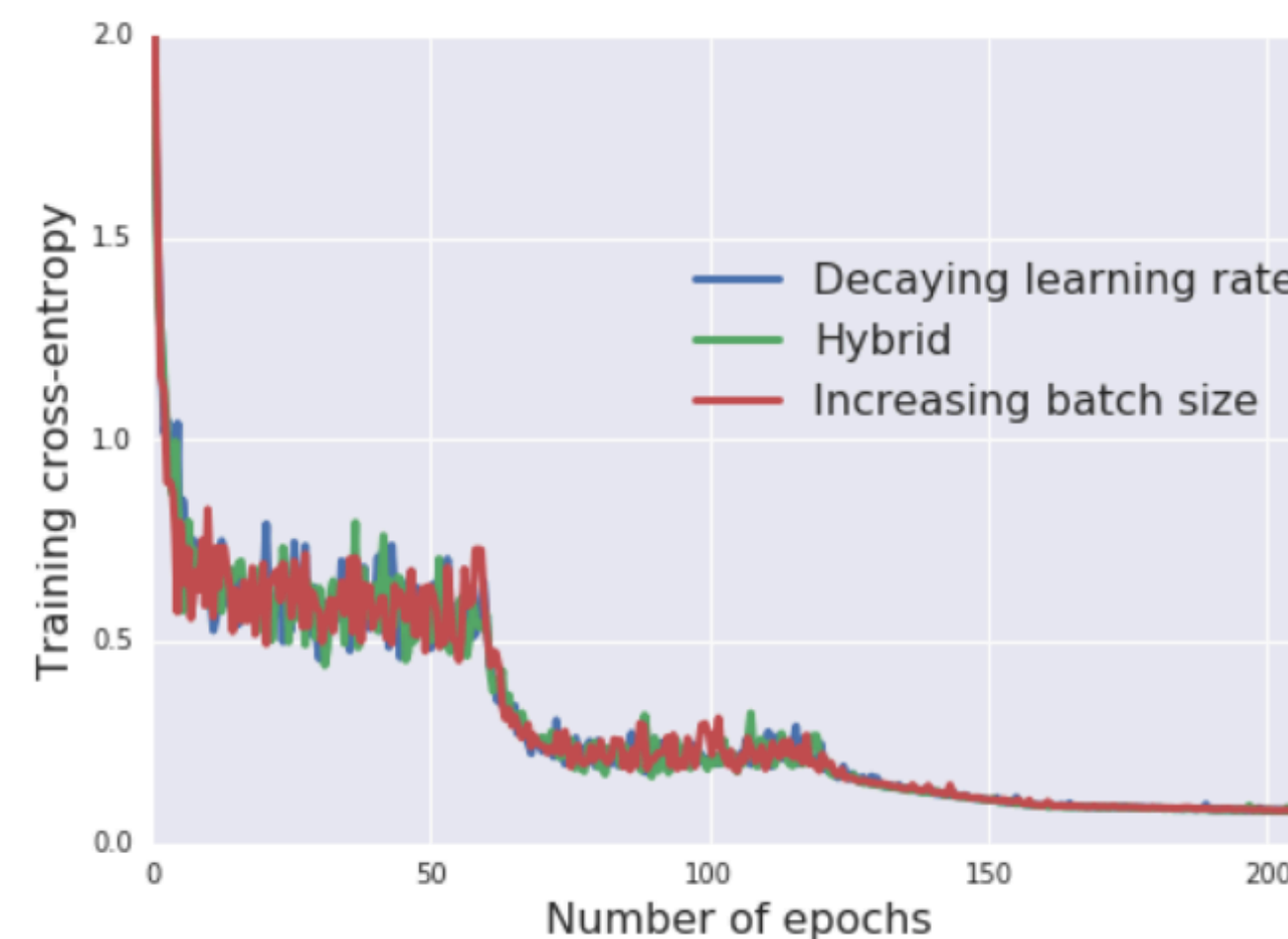
Stabilisation de l'apprentissage

Régler la taille de batch

- **Taille de batch faible** : trop de variation dans le gradient (SGD) pour stabiliser l'optimisation
- **Taille de batch trop élevée** : risque de perte de généralisation et de perte d'information
- **Taille optimale** : la plus grande taille de batch qui réduit l'erreur

Remarques sur la relation *batch size* / *learning rate*

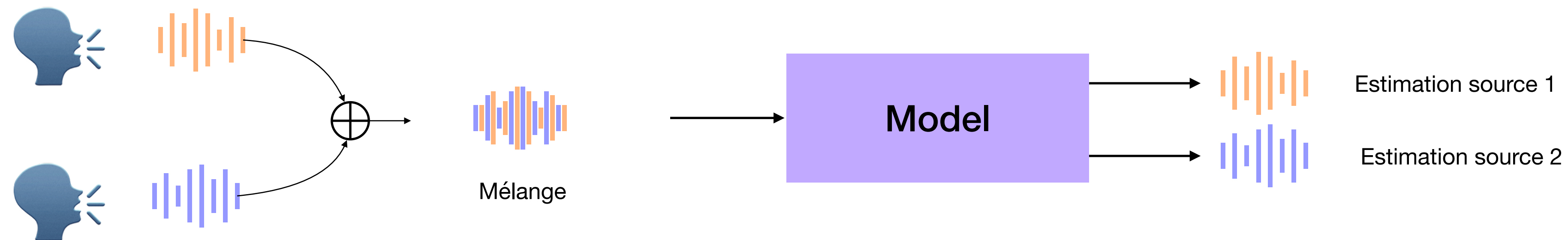
- Réduire le *learning rate* $\iff$ augmenter la *batch size*
- Beaucoup de travaux sur cette relation
- Possibilité de faire un *scheduler* de *batch size* (très avantageusement sur GPU)



Stabilisation de l'apprentissage

Exemple de la séparation de locuteurs

- La séparation de sources (ici de parole) consiste à prédire les formes d'ondes séparées de chaque locuteur à partir d'un mélange

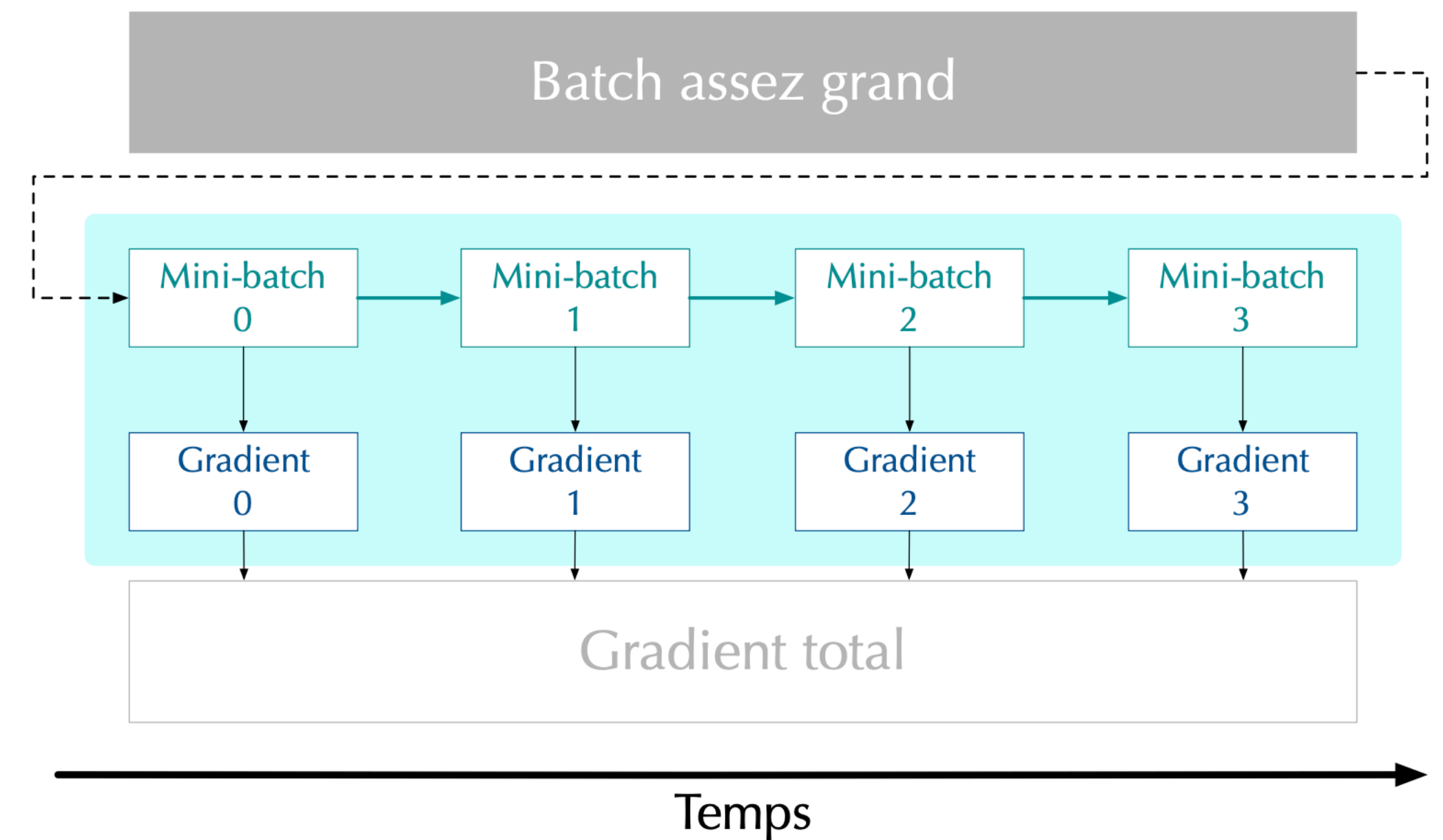


- On utilise souvent une batch size de 2 ou 4 ce qui donne les meilleures performances
- L'apprentissage de cette tâche est très sensible au moyenne des gradients !

Stabiliser l'apprentissage

Accumulation de gradient

- Sur les machines avec peu de mémoire, la *batch size* est rapidement limitée
- On peut accumuler les gradients pour créer une *batch size virtuelle*
- Les poids du modèle sont mis à jour après avoir accumulé suffisamment de gradients



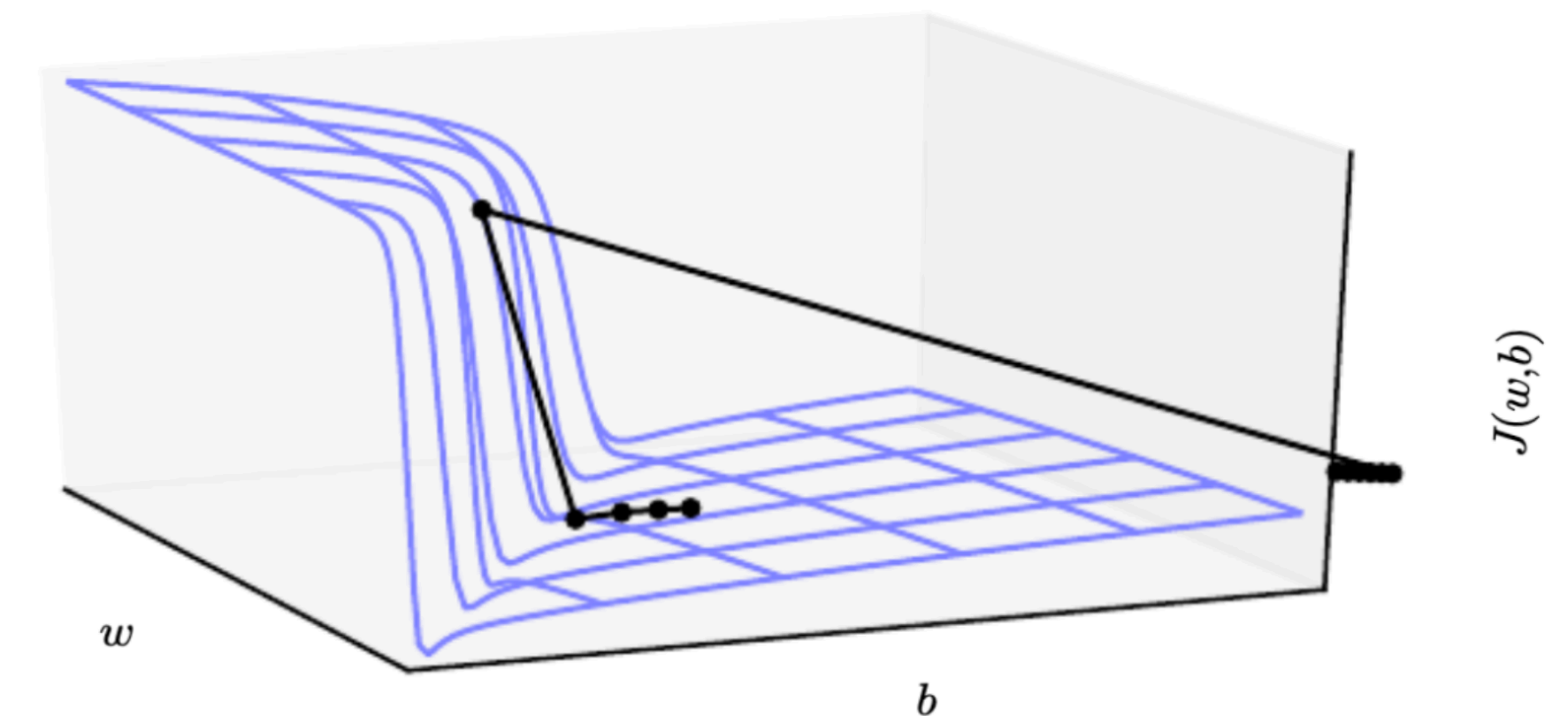
Stabiliser l'apprentissage

Gradient clipping

- Réseaux de neurones = fonctions fortement non-linéaires
- Aux abords des non-linéarités (cf. figure) le gradient peut prendre de grandes valeurs
- Les poids du modèles peuvent alors être projetés loin des valeurs précédentes
- Le gradient clipping force la valeur du gradient si celle-ci dépasse une valeur

$$\|\nabla \mathbf{f}\|_2 = \begin{cases} \|\nabla \mathbf{f}\|_2 & \text{si } \|\nabla \mathbf{f}\|_2 \leq g_{max} \\ g_{max} & \text{sinon} \end{cases}$$

- Très utile pour les réseaux récurrents (M2)



Stabiliser l'apprentissage

Gradient clipping

- Réseaux de neurones = fonctions fortement non-linéaires
- Aux abords des non-linéarités (cf. figure) le gradient peut prendre de grandes valeurs
- Les poids du modèles peuvent alors être projetés loin des valeurs précédentes
- Le gradient clipping force la valeur du gradient si celle-ci dépasse une valeur

$$\|\nabla \mathbf{f}\|_2 = \begin{cases} \|\nabla \mathbf{f}\|_2 & \text{si } \|\nabla \mathbf{f}\|_2 \leq g_{max} \\ g_{max} & \text{sinon} \end{cases} \longrightarrow \text{Projection du gradient sur une boule de rayon } \theta$$

- Très utile pour les réseaux récurrents (M2)

